



Introduction

- └ How to use PocketBase
- └ Collections
- └ API rules and filters
- └ Authentication
- └ Files upload and handling
- └ Working with relations
- └ Extending PocketBase

Going to production

Web APIs reference

Extend with
Go

Extend with
JavaScript

- JS Overview
- JS Event hooks
- JS Routing
- JS Database
- JS Record operations
- JS Collection operations
- JS Migrations
- JS Jobs scheduling
- JS Sending emails
- JS Rendering templates
- JS Console commands

Introduction



Please keep in mind that PocketBase is still under active development and full backward compatibility is not guaranteed before reaching v1.0.0. PocketBase is NOT recommended for production critical applications yet, unless you are fine with reading the [changelog](#) and applying some manual migration steps from time to time.

PocketBase is an open source backend consisting of embedded database (SQLite) with realtime subscriptions, builtin auth management, convenient dashboard UI and simple REST-ish API. It can be used both as Go framework and as standalone application.

The easiest way to get started is to download the prebuilt minimal PocketBase executable:

x64

ARM64

- [Download v0.36.7 for Linux x64](#) (~12MB zip)
- [Download v0.36.7 for Windows x64](#) (~12MB zip)
- [Download v0.36.7 for macOS x64](#) (~12MB zip)

See the [GitHub Releases page](#) for other platforms and more details.

Once you've extracted the archive, you could start the application by running `./pocketbase serve` in the extracted directory.

And that's it! The first time it will generate an installer link that should be automatically opened in the browser to set up your first superuser account

(you can also create the first superuser manually via `./pocketbase superuser create EMAIL PASS`)

JS Sending HTTP requests

JS Realtime messaging

The started web server has the following default routes:

- `http://127.0.0.1:8090` - if `pb_public` directory exists, serves the static content from it (html, css, images, etc.)
- `http://127.0.0.1:8090/_/` - superusers dashboard
- `http://127.0.0.1:8090/api/` - REST-ish API

The prebuilt PocketBase executable will create and manage 2 new directories alongside the executable:

- `pb_data` - stores your application data, uploaded files, etc. (usually should be added in `.gitignore`).
- `pb_migrations` - contains JS migration files with your collection changes (can be safely committed in your repository).

You can even write custom migration scripts. For more info check the [JS migrations docs](#).

You could find all available commands and their options by running `./pocketbase --help` or `./pocketbase [command] --help`

Next: How to use PocketBase →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



Introduction

- How to use PocketBase
- Collections**
- API rules and filters
- Authentication
- Files upload and handling
- Working with relations
- Extending PocketBase

✔ Going to production

📖 Web APIs reference

Extend with
Go

Extend with
JavaScript

- JS Overview
- JS Event hooks
- JS Routing
- JS Database
- JS Record operations
- JS Collection operations
- JS Migrations
- JS Jobs scheduling
- JS Sending emails
- JS Rendering templates
- JS Console commands
- JS Sending HTTP requests
- JS Realtime messaging
- JS Filesystem
- JS Logging
- JS Types reference

Introduction - Collections

- Overview**
 - [Base collection](#)
 - [View collection](#)
 - [Auth collection](#)
- Fields**

Overview

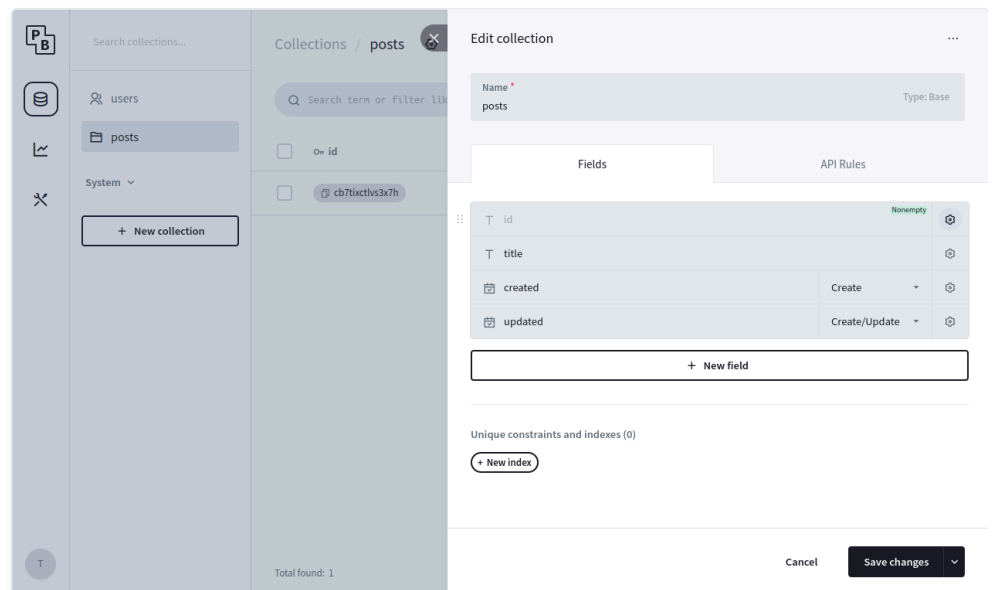
Collections represent your application data. Under the hood they are backed by plain SQLite tables that are generated automatically with the collection **name** and **fields** (columns).

Single entry of a collection is called **record** (a single row in the SQL table).

You can manage your **collections** from the Dashboard, with the Web APIs using the [client-side SDKs](#) (*superusers only*) or programmatically via the [Go/JavaScript](#) migrations.

Similarly, you can manage your **records** from the Dashboard, with the Web APIs using the [client-side SDKs](#) or programmatically via the [Go/JavaScript](#) Record operations.

Here is what a collection edit panel looks like in the Dashboard:



Currently there are 3 collection types: **Base**, **View** and **Auth**.

Base collection

Base collection is the default collection type and it could be used to store any application data (articles, products, posts, etc.).

View collection

View collection is a read-only collection type where the data is populated from a plain SQL `SELECT` statement, allowing users to perform aggregations or any other custom queries in general.

For example, the following query will create a read-only collection with 3 *posts* fields - *id*, *name* and *totalComments*:

```
SELECT
  posts.id,
  posts.name,
  count(comments.id) as totalComments
```

```
FROM posts
LEFT JOIN comments on comments.postId = posts.id
GROUP BY posts.id
```



View collections don't receive realtime events because they don't have create/update/delete operations.

Auth collection

Auth collection has everything from the **Base collection** but with some additional special fields to help you manage your app users and also provide various authentication options.

Each Auth collection has the following special system fields: `email`, `emailVisibility`, `verified`, `password` and `tokenKey`.

They cannot be renamed or deleted but can be configured using their specific field options. For example you can make the user email required or optional.

You can have as many Auth collections as you want (users, managers, staffs, members, clients, etc.) each with their own set of fields, separate login and records managing endpoints.

You can build all sort of different access controls:

- **Role (Group)**

For example, you could attach a "role" `select` field to your Auth collection with the following options: "employee" and "staff". And then in some of your other collections you could define the following rule to allow only "staff":

```
@request.auth.role = "staff"
```

- **Relation (Ownership)**

Let's say that you have 2 collections - "posts" base collection and "users" auth collection. In your "posts" collection you can create "author" `relation` field pointing to the "users" collection. To allow access to only the "author" of the record(s), you could use a rule like:

```
@request.auth.id != "" && author = @request.auth.id
```

Nested relation fields look ups, including back-relations, are also supported, for example:

```
someRelField.anotherRelField.author = @request.auth.id
```

- **Managed**

In addition to the default "List", "View", "Create", "Update", "Delete" API rules, Auth collections have also a special "Manage" API rule that could be used to allow one user (it could be even from a different collection) to be able to fully manage the data of another user (e.g. changing their email, password, etc.).

- **Mixed**

You can build a mixed approach based on your unique use-case. Multiple rules can be grouped with parenthesis `()` and combined with `&&` (AND) and `||` (OR) operators:

```
@request.auth.id != "" && (@request.auth.role = "staff" || author = @request.auth.id)
```

Fields



All collection fields (with exception of the `JSONField`) are **non-nullable** and use a **zero-default** for their respective type as fallback value when missing (empty string for `text`, 0 for `number`, etc.).

All field specific modifiers are supported both in the Web APIs and via the record Get/Set methods.

BoolField



NumberField



TextField	▼
EmailField	▼
URLField	▼
EditorField	▼
DateField	▼
AutodateField	▼
SelectField	▼
FileField	▼
RelationField	▼
JSONField	▼
GeoPoint	▼

← Prev: How to use PocketBase

Next: API rules and filters →

[FAQ](#)

[JavaScript SDK](#)

PocketBase

[Discussions](#)

[Dart SDK](#)

[Documentation](#)





Introduction

- How to use PocketBase
- Collections
- API rules and filters
- Authentication
- Files upload and handling
- Working with relations
- Extending PocketBase

Going to production

Web APIs reference

Extend with
Go

Extend with
JavaScript

- JS Overview
- JS Event hooks
- JS Routing
- JS Database
- JS Record operations
- JS Collection operations
- JS Migrations
- JS Jobs scheduling
- JS Sending emails
- JS Rendering templates
- JS Console commands

Introduction - How to use PocketBase

The easiest way to use PocketBase is by interacting with its Web APIs directly from the client-side (e.g. mobile app or browser SPA).

It was designed with this exact use case in mind and it is also the reason why there are general purpose JSON APIs for listing, pagination, sorting, filtering, etc.

The access and filter controls for your data are usually done through the [collection API rules](#).

For the cases when you need more specialized handling (sending emails, intercepting the default actions, creating new routes, etc.) you can [extend PocketBase with Go or JavaScript](#).

For interacting with the [Web APIs](#) you can make use of the official SDK clients:

- [JavaScript SDK](#) (Browser, Node.js, React Native)
- [Dart SDK](#) (Web, Mobile, Desktop, CLI)

When used on the client-side, it is safe to have a single/global SDK instance for the entire lifecycle of your application.

Web apps recommendation



Why not JS SSR



Why not htmx, Hotwire/Turbo, Unpoly, etc.



Mobile apps auth persistence



React Native file upload on Android and iOS



The next couple pages have a little bit more information about the basic PocketBase components like collections, records, authentication, relations, files handling, etc.

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation





Introduction

- └ How to use PocketBase
- └ Collections
- └ **API rules and filters**
- └ Authentication
- └ Files upload and handling
- └ Working with relations
- └ Extending PocketBase

Going to production

Web APIs reference

Extend with
Go

Extend with
JavaScript

- JS Overview
- JS Event hooks
- JS Routing
- JS Database
- JS Record operations
- JS Collection operations
- JS Migrations
- JS Jobs scheduling
- JS Sending emails
- JS Rendering templates
- JS Console commands

Introduction - API rules and filters

- [API rules](#)
- [Filters syntax](#)
- [Special identifiers and modifiers](#)
 - [@macros](#)
 - [:isset modifier](#)
 - [:changed modifier](#)
 - [:length modifier](#)
 - [:each modifier](#)
 - [:lower modifier](#)
 - [geoDistance\(lonA, latA, lonB, latB\)](#)
 - [strftime\(format, \[time-value, modifiers...\]\)](#)
- [Examples](#)

API rules

API Rules are your collection access controls and data filters.

Each collection has **5 rules**, corresponding to the specific API action:

- `listRule`
- `viewRule`
- `createRule`
- `updateRule`
- `deleteRule`

Auth collections have an additional `options.manageRule` used to allow one user (it could be even from a different collection) to be able to fully manage the data of another user (ex. changing their email, password, etc.).

Each rule could be set to:

- **"locked"** - aka. `null`, which means that the action could be performed only by an authorized superuser (**this is the default**)

JS Sending HTTP requests

JS Realtime messaging

- **Empty string** - anyone will be able to perform the action (superusers, authorized users and guests)
- **Non-empty string** - only users (authorized or not) that satisfy the rule filter expression will be able to perform this action

PocketBase API Rules act also as records filter!

Or in other words, you could for example allow listing only the "active" records of your collection, by using a simple filter expression such as: `status = "active"` (where "status" is a field defined in your Collection).



Because of the above, the API will return 200 empty items response in case a request doesn't satisfy a `listRule`, 400 for unsatisfied

`createRule` and 404 for unsatisfied `viewRule`, `updateRule` and `deleteRule`.

All rules will return 403 in case they were "locked" (aka. superuser only) and the request client is not a superuser.

The API Rules are ignored when the action is performed by an authorized superuser (**superusers can access everything**)!

Filters syntax

You can find information about the available fields in your collection API rules tab:

Edit collection

...

Name *

posts

Type: Base

Fields

API Rules

All rules follow the [PocketBase filter syntax and operators](#).

Show available fields

List/Search rule

Set Superusers only

@request.auth.id != "" && status = "active"

View rule - Superusers only

Create rule - Superusers only

Update rule - Superusers only

Cancel

Save changes

There is autocomplete to help guide you while typing the rule filter expression, but in general you have access to **3 groups of fields**:

- **Your Collection schema fields**

This includes all nested relation fields too, ex.

```
someRelField.status != "pending"
```

- **@request.***

Used to access the current request data, such as query parameters, body/form fields, authorized user state, etc.

- **@request.context** - the context where the rule is used (ex. `@request.context != "oauth2"`)
The currently supported context values are `default`, `oauth2`, `otp`, `password`, `realtime`, `protectedFile`.
- **@request.method** - the HTTP request method (ex. `@request.method = "GET"`)
- **@request.headers.*** - the request headers as string values (ex. `@request.headers.x_token = "test"`)
Note: All header keys are normalized to lowercase and "-" is replaced with "_" (for example "X-Token" is "x_token").
- **@request.query.*** - the request query parameters as string values (ex. `@request.query.page = "1"`)
- **@request.auth.*** - the current authenticated model (ex. `@request.auth.id != ""`)
- **@request.body.*** - the submitted body parameters (ex. `@request.body.title != ""`)

Note: Uploaded files are not part of the `@request.body` because they are evaluated separately (*this behavior may change in the future*).

- **@collection.***

This filter could be used to target other collections that are not directly related to the current one (aka. there is no relation field pointing to it) but both shares a common field value, like for example a category id:

```
@collection.news.categoryId ?= categoryId &&  
@collection.news.author ?= @request.auth.id
```

In case you want to join the same collection multiple times but based on different criteria, you can define an alias by appending `:alias` suffix to the collection name.

```
// see  
https://github.com/pocketbase/pocketbase/discussions/3805#discussioncomment-7634791  
@request.auth.id != "" &&  
@collection.courseRegistrations.user ?= id &&  
@collection.courseRegistrations:auth.user ?=  
@request.auth.id &&  
@collection.courseRegistrations.courseGroup ?=  
@collection.courseRegistrations:auth.courseGroup
```

The syntax basically follows the format

`OPERAND OPERATOR OPERAND`, where:

- **OPERAND** - could be any field literal, string (single or double quoted), number, null, true, false
- **OPERATOR** - is one of:
 - `=` Equal
 - `!=` NOT equal
 - `>` Greater than
 - `>=` Greater than or equal
 - `<` Less than
 - `<=` Less than or equal
 - `~` Like/Contains (if not specified auto wraps the right string OPERAND in a "%" for wildcard match)
 - `!~` NOT Like/Contains (if not specified auto wraps the right string OPERAND in a "%" for wildcard match)

- `?=` *Any/At least one of Equal*
- `?!=` *Any/At least one of NOT equal*
- `?>` *Any/At least one of Greater than*
- `?>=` *Any/At least one of Greater than or equal*
- `?<` *Any/At least one of Less than*
- `?<=` *Any/At least one of Less than or equal*
- `?~` *Any/At least one of Like/Contains (if not specified auto wraps the right string OPERAND in a "%" for wildcard match)*
- `?!~` *Any/At least one of NOT Like/Contains (if not specified auto wraps the right string OPERAND in a "%" for wildcard match)*

To group and combine several expressions you can use parenthesis `(...)`, `&&` (AND) and `||` (OR) tokens.

Single line comments are also supported:

```
// Example comment.
```

Field expressions with array-like value or nested fields that originate from a source with multiple records will apply a **match-all** constraint by default. If you want **any/at-least-one-of** type of constraint for such fields you'll have to prefix your operator with `?` (e.g. `multiRelation.title ?= "test"`).

Special identifiers and modifiers

@ macros

The following datetime macros are available and can be used as part of the filter expression:

```
// all macros are UTC based
// (for more complex date operation check the
// strftime() function)
@now          - the current datetime as string
@second       - @now second number (0-59)
@minute       - @now minute number (0-59)
@hour         - @now hour number (0-23)
@weekday      - @now weekday number (0-6)
@day          - @now day number
@month        - @now month number
@year         - @now year number
@yesterday    - the yesterday datetime relative to
@now as string
```

```
@tomorrow - the tomorrow datetime relative to
@now as string
@todayStart - beginning of the current day as
datetime string
@todayEnd - end of the current day as datetime
string
@monthStart - beginning of the current month as
datetime string
@monthEnd - end of the current month as datetime
string
@yearStart - beginning of the current year as
datetime string
@yearEnd - end of the current year as datetime
string
```

For example:

```
@request.body.publicDate >= @now
```

:isset modifier

The `:isset` field modifier is available only for the `@request.*` fields and can be used to check whether the client submitted a specific data with the request. Here is for example a rule that disallows submitting a "role" field:

```
@request.body.role:isset = false
```

Note that `@request.body.*:isset` at the moment doesn't support checking for new uploaded files because they are evaluated separately and cannot be serialized (*this behavior may change in the future*).

:changed modifier

The `:changed` field modifier is available only for the `@request.body.*` fields and can be used to check whether the client submitted AND changed a specific record field with the request. Here is for example a rule that disallows changing a "role" field:

```
// the same as: (@request.body.role:isset = false
|| @request.body.role = role)
@request.body.role:changed = false
```

Note that `@request.body.*:changed` at the moment doesn't support checking for new uploaded files because they are evaluated separately and cannot be serialized (*this behavior may change in the future*).

:length modifier

The `:length` field modifier could be used to check the number of items in an array field (multiple `file`, `select`, `relation`). Could be used with both the collection schema fields and the `@request.body.*` fields. For example:

```
// check example submitted data:
{"someSelectField": ["val1", "val2"]}
@request.body.someSelectField:length > 1

// check existing record field length
someRelationField:length = 2
```

Note that `@request.body.*:length` at the moment doesn't support checking for new uploaded files because they are evaluated separately and cannot be serialized (*this behavior may change in the future*).

:each modifier

The `:each` field modifier works only with multiple `select`, `file` and `relation` type fields. It could be used to apply a condition on each item from the field array. For example:

```
// check if all submitted select options contain
the "create" text
@request.body.someSelectField:each ~ "create"

// check if all existing someSelectField has "pb_"
prefix
someSelectField:each ~ "pb_"
```

Note that `@request.body.*:each` at the moment doesn't support checking for new uploaded files because they are evaluated separately and cannot be serialized (*this behavior may change in the future*).

:lower modifier

The `:lower` field modifier could be used to perform lower-case string comparisons. For example:

```
// check if the submitted lower-cased body "title"
field is equal to "test" ("Test", "tEsT", etc.)
@request.body.title:lower = "test"

// match existing records with lower-cased "title"
equal to "test" ("Test", "tEsT", etc.)
title:lower ~ "test"
```

Under the hood it uses the [SQLite LOWER](#) scalar function and by default works only for ASCII characters, unless the ICU extension is loaded.

geoDistance(lonA, latA, lonB, latB)

The `geoDistance(lonA, latA, lonB, latB)` function could be used to calculate the Haversine distance between 2 geographic points in kilometres.

The function is intended to be used primarily with the `geoPoint` field type, but the accepted arguments could be any plain number or collection field identifier. If the identifier cannot be resolved and converted to a numeric value, it resolves to `null`. Note that the `geoDistance` function always results in a single row/record value meaning that "any/at-least-one-of" type of constraint will be applied even if some of its arguments originate from a multiple relation field.

For example:

```
// offices that are less than 25km from my
location (address is a geoPoint field in the
offices collection)
geoDistance(address.lon, address.lat, 23.32,
42.69) < 25
```

strftime(format, [time-value, modifiers...])

The `strftime(format, [time-value, modifiers...])` returns a date string formatted according to the specified format argument.

The function is similar to the builtin SQLite `strftime` with the main difference that NULL results will be normalized for consistency with the non-nullable PocketBase `text` and `date` fields.

The function accepts 1, 2 or 3+ arguments.

1. The first (**format**) argument must be a formatting string with valid substitution characters as listed in https://sqlite.org/lang_datefunc.html.
2. The second (**time-value**) argument is optional and must be either a date **string**, **number** or **collection field identifier** with value matching one of the formats listed in https://sqlite.org/lang_datefunc.html#time_values. If not set the function fallbacks to the current datetime.
3. The remaining (**modifiers**) optional arguments are expected to be string literals matching the listed modifiers

in https://sqlite.org/lang_datefunc.html#modifiers (up to 8 max).

A match-all constraint will be also applied in case the time-value is an identifier as a result of a multi-value relation field. For example:

```
// requires ALL multiRel records to have "created"
// that match the formatted string "2026-01"
strftime('%Y-%m', multiRel.created) = "2026-01"

// requires ANY/AT-LEAST-ONE-OF multiRel records
// to have "created" that match the formatted string
// "2026-01"
strftime('%Y-%m', multiRel.created) ?= "2026-01"
```

Examples

- Allow only registered users:

```
@request.auth.id != ""
```

- Allow only registered users and return records that are either "active" or "pending":

```
@request.auth.id != "" && (status = "active"
|| status = "pending")
```

- Allow only registered users who are listed in an *allowed_users* multi-relation field value:

```
@request.auth.id != "" && allowed_users.id ?=
@request.auth.id
```

- Allow access by anyone and return only the records where the *title* field value starts with "Lorem" (ex. "Lorem ipsum"):

```
title ~ "Lorem%"
```


FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation





Introduction

- How to use PocketBase
- Collections
- API rules and filters
- Authentication**
- Files upload and handling
- Working with relations
- Extending PocketBase

Going to production

Web APIs reference

Extend with
Go

Extend with
JavaScript

- JS Overview
- JS Event hooks
- JS Routing
- JS Database
- JS Record operations
- JS Collection operations
- JS Migrations
- JS Jobs scheduling
- JS Sending emails
- JS Rendering templates
- JS Console commands

Introduction - Authentication

- [Overview](#)
- [Authenticate with password](#)
- [Authenticate with OTP](#)
- [Authenticate with OAuth2](#)
- [Multi-factor authentication](#)
- [Users impersonation](#)
- [API keys](#)
- [Auth token verification](#)

Overview

A single client is considered authenticated as long as it sends valid `Authorization:YOUR_AUTH_TOKEN` header with the request.

The PocketBase Web APIs are fully stateless and there are no sessions in the traditional sense (even the tokens are not stored in the database).

Because there are no sessions and we don't store the tokens on the server there is also no logout endpoint. To "logout" a user you can simply disregard the token from your local state (aka. `pb.authStore.clear()` if you use the SDKs).

The auth token could be generated either through the specific auth collection Web APIs or programmatically via Go/JS.

All allowed auth collection methods can be configured individually from the specific auth collection options.

Note that PocketBase admins (aka. `_superusers`) are similar to the regular auth collection records with 2 caveats:



- OAuth2 is not supported as auth method for the `_superusers` collection

JS Sending HTTP requests

JS Realtime messaging

- Superusers can access and modify anything (collection API rules are ignored)

Authenticate with password

To authenticate with password you must enable the *Identity/Password* auth collection option (see also [Web API reference](#)).

The default identity field is the `email` but you can configure any other unique field like "username" (it must have a UNIQUE index).

JavaScript

Dart

```
import PocketBase from 'pocketbase';

const pb = new
PocketBase('http://127.0.0.1:8090');

...

const authData = await
pb.collection("users").authWithPassword('test@example.com', '1234567890');

// after the above you can also access the auth
data from the authStore
console.log(pb.authStore.isValid);
console.log(pb.authStore.token);
console.log(pb.authStore.record.id);

// "logout" the last authenticated record
pb.authStore.clear();
```

Authenticate with OTP

To authenticate with email code you must enable the *One-time password (OTP)* auth collection option (see also [Web API reference](#)).

The usual flow is the user typing manually the received password from their email but you can also adjust the default email template from the collection options and add a url

containing the OTP and its id as query parameters (*you have access to `{OTP}` and `{OTP_ID}` placeholders*).

Note that when requesting an OTP we return an `otpId` even if a user with the provided email doesn't exist as a very rudimentary enumeration protection (it doesn't create or send anything).

On successful OTP validation, by default the related user email will be automatically marked as "verified".



Keep in mind that OTP as a standalone authentication method could be less secure compared to the other methods because the generated password is usually 0-9 digits and there is a risk of it being guessed or enumerated (especially when a longer duration time is configured).

For security critical applications OTP is recommended to be used in combination with the other auth methods and the Multi-factor authentication option.

JavaScript

Dart

```
import PocketBase from 'pocketbase';

const pb = new
PocketBase('http://127.0.0.1:8090');

...

// send OTP email to the provided auth record
const result = await
pb.collection('users').requestOTP('test@example.co
m');

// ... show a screen/popup to enter the password
from the email ...

// authenticate with the requested OTP id and the
email password
const authData = await
pb.collection('users').authWithOTP(result.otpId,
"YOUR_OTP");

// after the above you can also access the auth
```

```
data from the authStore
console.log(pb.authStore.isValid);
console.log(pb.authStore.token);
console.log(pb.authStore.record.id);

// "logout"
pb.authStore.clear();
```

Authenticate with OAuth2

You can also authenticate your users with an OAuth2 provider (Google, GitHub, Microsoft, etc.). See the section below for example integrations.

Before starting, you'll need to create an OAuth2 app in the provider's dashboard in order to get a **Client Id** and **Client Secret**, and register a redirect URL [?](#).



Once you have obtained the **Client Id** and **Client Secret**, you can enable and configure the provider from your PocketBase auth collection options (*PocketBase > Collections > {YOUR_COLLECTION} > Edit collection (settings cogwheel) > Options > OAuth2*).

All in one (*recommended*)

Manual code exchange

This method handles everything within a single call without having to define custom redirects, deeplinks or even page reload.

When creating your OAuth2 app, for a callback/redirect URL you have to use the

`https://yourdomain.com/api/oauth2-redirect` (or when testing locally -

`http://127.0.0.1:8090/api/oauth2-redirect`).

JavaScript

Dart

```
import PocketBase from 'pocketbase';

const pb = new
PocketBase('https://pocketbase.io');
```

...

```
// This method initializes a one-off realtime
subscription and will
// open a popup window with the OAuth2 vendor page
to authenticate.
//
// Once the external OAuth2 sign-in/sign-up flow
is completed, the popup
// window will be automatically closed and the
OAuth2 data sent back
// to the user through the previously established
realtime connection.
//
// If the popup is being blocked on Safari, make
sure that your click handler is not using
async/await.
pb.collection('users').authWithOAuth2({
  provider: 'google'
}).then((authData) => {
  console.log(authData)

  // after the above you can also access the
auth data from the authStore
  console.log(pb.authStore.isValid);
  console.log(pb.authStore.token);
  console.log(pb.authStore.record.id);

  // "logout" the last authenticated record
  pb.authStore.clear();
});
```

Multi-factor authentication

PocketBase v0.23+ introduced optional Multi-factor authentication (MFA).

If enabled, it requires the user to authenticate with any 2 different auth methods from above (the order doesn't matter). The expected flow is:

1. User authenticates with "Auth method A".
2. On success, a 401 response is sent with `{"mfaId": "..."}` as JSON body (the MFA "session" is stored in the `_mfas` system collection).
3. User authenticates with "Auth method B" as usual **but adds the `mfaId` from the previous step as body or query parameter.**

4. On success, a regular auth response is returned, aka. token + auth record data.

Below is an example for email/password + OTP authentication:

JavaScript

Dart

```
import PocketBase from 'pocketbase';

const pb = new
PocketBase('http://127.0.0.1:8090');

...

try {
  await
pb.collection('users').authWithPassword('test@example.com', '1234567890');
} catch (err) {
  const mfaId = err.response?.mfaId;
  if (!mfaId) {
    throw err; // not mfa -> rethrow
  }

  // the user needs to authenticate again with
  another auth method, for example OTP
  const result = await
pb.collection('users').requestOTP('test@example.com');
  // ... show a modal for users to check their
  email and to enter the received code ...
  await
pb.collection('users').authWithOTP(result.otpId,
'EMAIL_CODE', { 'mfaId': mfaId });
}
```

Users impersonation

Superusers have the option to generate tokens and authenticate as anyone else via the [Impersonate endpoint](#).

The generated impersonate auth tokens can have custom duration but are not renewable!

For convenience the official SDKs creates and returns a standalone client that keeps the token state in memory, aka. only for the duration of the impersonate client instance.

JavaScript

Dart

```
import PocketBase from 'pocketbase';

const pb = new
PocketBase('http://127.0.0.1:8090');

...

// authenticate as superuser
await
pb.collection("_superusers").authWithPassword("tes
t@example.com", "1234567890");

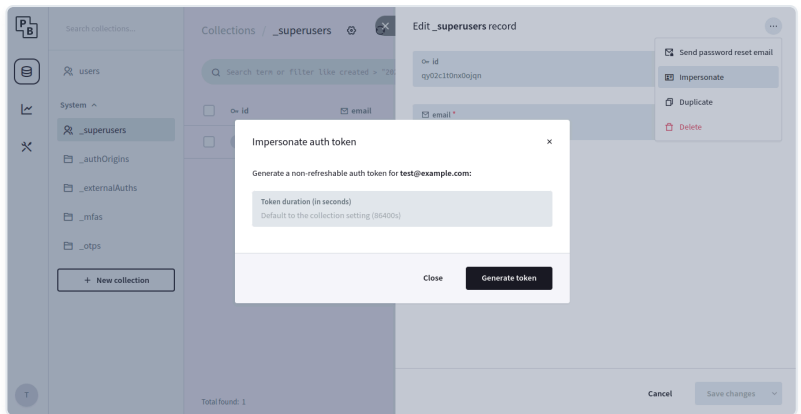
// impersonate
// (the custom token duration is in seconds and it
is optional)
const impersonateClient = await
pb.collection("users").impersonate("USER_RECORD_ID
", 3600)

// log the impersonate token and user data
console.log(impersonateClient.authStore.token);
console.log(impersonateClient.authStore.record);

// send requests as the impersonated user
const items = await
impersonateClient.collection("example").getFullList();
```

API keys

While PocketBase doesn't have "API keys" in the traditional sense, as a side effect of the support for users impersonation, for such cases you can use instead the generated nonrenewable `_superusers` impersonate auth token. You can generate such token via the above impersonate API or from the *Dashboard > Collections > _superusers > {select superuser} > "Impersonate" dropdown option*:



Because of the security implications (superusers can execute, access and modify anything), use the generated `_superusers` tokens with extreme care and only for internal **server-to-server** communication.



To invalidate already issued tokens, you need to change the individual superuser account password (or if you want to reset the tokens for all superusers - change the shared auth token secret from the `_superusers` collection options).

Auth token verification

PocketBase doesn't have a dedicated token verification endpoint, but if you want to verify an existing auth token from a 3rd party app you can send an Auth refresh call, aka.

```
pb.collection("users").authRefresh()
```

On valid token - it returns a new token with refreshed `exp` claim and the latest user data.

Otherwise - returns an error response.

Note that calling `authRefresh` doesn't invalidate previously issued tokens and you can safely disregard the new one if you don't need it (as mentioned in the beginning - PocketBase doesn't store the tokens on the server).

Performance wise, the used `HS256` algorithm for generating the JWT has very little to no impact and it is essentially the same in terms of response time as calling `getOne("USER_ID")` (see benchmarks).

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



Introduction

- How to use PocketBase
- Collections
- API rules and filters
- Authentication
- Files upload and handling**
- Working with relations
- Extending PocketBase

Going to production

Web APIs reference

Extend with
Go

Extend with
JavaScript

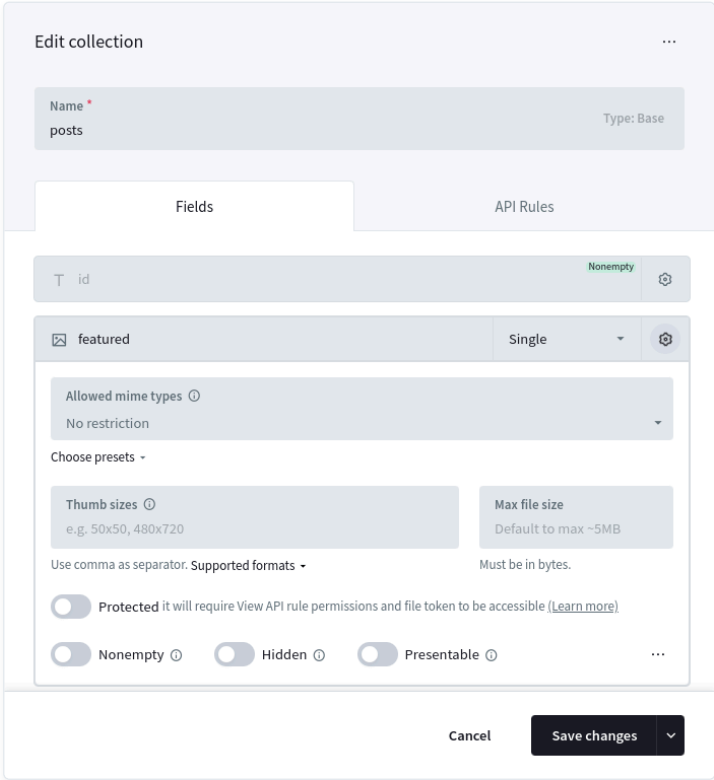
- JS Overview
- JS Event hooks
- JS Routing
- JS Database
- JS Record operations
- JS Collection operations
- JS Migrations
- JS Jobs scheduling
- JS Sending emails
- JS Rendering templates
- JS Console commands
- JS Sending HTTP requests
- JS Realtime messaging
- JS Filesystem
- JS Logging
- JS Types reference

Introduction - Files upload and handling

- [Uploading files](#)
- [Deleting files](#)
- [File URL](#)
- [Protected files](#)
- [Storage options](#)

Uploading files

To upload files, you must first add a `file` field to your collection:



Edit collection ...

Name * posts Type: Base

Fields API Rules

`id` Nonempty ⚙️

`featured` Single ⚙️

Allowed mime types ⓘ
No restriction ▼

Choose presets -

Thumb sizes ⓘ
e.g. 50x50, 480x720

Max file size
Default to max ~5MB

Use comma as separator. Supported formats - Must be in bytes.

☐ Protected ⓘ it will require View API rule permissions and file token to be accessible ([Learn more](#))

☐ Nonempty ⓘ ☐ Hidden ⓘ ☐ Presentable ⓘ ⋮

Cancel Save changes ▼

Once added, you could create/update a Record and upload "documents" files by sending a `multipart/form-data` request using the *Records create/update APIs*.

Each uploaded file will be stored with the original filename (sanitized) and suffixed with a random part (usually 10 characters). For example `test_52iwbgds7l.png`.

ⓘ All `file` fields by default has a max allowed file size up to ~5MB (you can adjust it from the collection field options but keep in mind that allowing to upload and serve large files could degrade the performance of your application).

Here is an example how to create a new record and upload multiple files to the example "documents" `file` field using the SDKs:

JavaScript

Dart

```
import PocketBase from 'pocketbase';

const pb = new PocketBase('http://127.0.0.1:8090');

...
```

```
// create a new record and upload multiple files
// (files must be Blob or File instances)
const createdRecord = await pb.collection('example').create({
  title: 'Hello world!', // regular text field
  'documents': [
    new File(['content 1...'], 'file1.txt'),
    new File(['content 2...'], 'file2.txt'),
  ]
});

// -----
// Alternative FormData + plain HTML file input example
// <input type="file" id="fileInput" />
// -----

const fileInput = document.getElementById('fileInput');

const formData = new FormData();

// set regular text field
formData.append('title', 'Hello world!');

// listen to file input changes and add the selected files to the form data
fileInput.addEventListener('change', function () {
  for (let file of fileInput.files) {
    formData.append('documents', file);
  }
});

...

// upload and create new record
const createdRecord = await pb.collection('example').create(formData);
```

If your `file` field supports uploading multiple files (aka. **Max Files option is >= 2**) you can use the `+` prefix/suffix field name modifier to respectively prepend/append new files alongside the already uploaded ones. For example:

JavaScript

Dart

```
import PocketBase from 'pocketbase';

const pb = new PocketBase('http://127.0.0.1:8090');

...

const createdRecord = await pb.collection('example').update('RECORD_ID', {
  "documents+": new File(["content 3..."], "file3.txt")
});
```

Deleting files

To delete uploaded file(s), you could either edit the Record from the Dashboard, or use the API and set the file field to a zero-value (empty string, `[]`).

If you want to **delete individual file(s) from a multiple file upload field**, you could suffix the field name with `-` and specify the filename(s) you want to delete. Here are some examples using the SDKs:

JavaScript

Dart

```
import PocketBase from 'pocketbase';

const pb = new PocketBase('http://127.0.0.1:8090');

...

// delete all "documents" files
await pb.collection('example').update('RECORD_ID', {
  'documents': [],
});

// delete individual files
await pb.collection('example').update('RECORD_ID', {
  'documents-': ["file1.pdf", "file2.txt"],
});
```

The above examples use the JSON object data format, but you could also use `FormData` instance for *multipart/form-data* requests. If using `FormData` set the file field to an empty string.

File URL

Each uploaded file could be accessed by requesting its file url:

`http://127.0.0.1:8090/api/files/COLLECTION_ID_OR_NAME/RECORD_ID/FILENAME`

If your file field has the **Thumb** sizes option, you can get a thumb of the image file by adding the `thumb` query parameter to the url like this:

`http://127.0.0.1:8090/api/files/COLLECTION_ID_OR_NAME/RECORD_ID/FILENAME?thumb=100x300`

Currently limited to jpg, png, gif (its first frame) and partially webp (stored as png).

The following thumb formats are currently supported:

- **WxH** (e.g. 100x300) - crop to WxH viewbox (from center)
- **WxHt** (e.g. 100x300t) - crop to WxH viewbox (from top)
- **WxHb** (e.g. 100x300b) - crop to WxH viewbox (from bottom)
- **WxHf** (e.g. 100x300f) - fit inside a WxH viewbox (without cropping)
- **0xH** (e.g. 0x300) - resize to H height preserving the aspect ratio
- **Wx0** (e.g. 100x0) - resize to W width preserving the aspect ratio

The original file would be returned, if the requested thumb size is not found or the file is not an image!

If you already have a Record model instance, the SDKs provide a convenient method to generate a file url by its name.

JavaScript

Dart

```
import PocketBase from 'pocketbase';

const pb = new PocketBase('http://127.0.0.1:8090');

...

const record = await pb.collection('example').getOne('RECORD_ID');

// get only the first filename from "documents"
//
// note:
// "documents" is an array of filenames because
// the "documents" field was created with "Max Files" option > 1;
// if "Max Files" was 1, then the result property would be just a string
const firstFilename = record.documents[0];
```

```
// returns something like:  
// http://127.0.0.1:8090/api/files/example/kfzjt5oy8r34hvn/test_52iWbGinWd.png?  
thumb=100x250  
const url = pb.files.getURL(record, firstFilename, {'thumb': '100x250'});
```

Additionally, to instruct the browser to always download the file instead of showing a preview when accessed directly, you can append the `?download=1` query parameter to the file url.

Protected files

By default all files are publicly accessible if you know their full url.

For most applications this is fine and reasonably safe because all files have a random part appended to their name, but in some cases you may want an extra security to prevent unauthorized access to sensitive files like ID card or Passport copies, contracts, etc.

To do this you can mark the `file` field as *Protected* from its field options in the Dashboard and then request the file with a special **short-lived file token**.



Only requests that satisfy the **View API rule** of the record collection will be able to access or download the protected file(s).

JavaScript

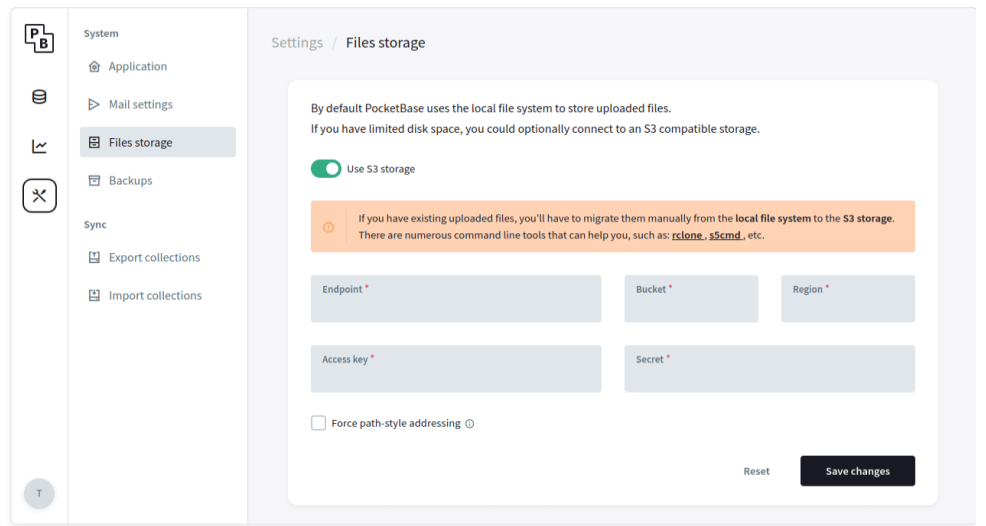
Dart

```
import PocketBase from 'pocketbase';  
  
const pb = new PocketBase('http://127.0.0.1:8090');  
  
...  
  
// authenticate  
await pb.collection('users').authWithPassword('test@example.com', '1234567890');  
  
// generate a file token  
const fileToken = await pb.files.getToken();  
  
// retrieve an example protected file url (will be valid ~2min)  
const record = await pb.collection('example').getOne('RECORD_ID');  
const url = pb.files.getURL(record, record.myPrivateFile, {'token': fileToken});
```

Storage options

By default PocketBase stores uploaded files in the `pb_data/storage` directory on the local file system. For the majority of cases this is usually the recommended storage option because it is very fast, easy to work with and backup.

But if you have limited disk space you could switch to an external S3 compatible storage (AWS S3, MinIO, Wasabi, DigitalOcean Spaces, Vultr Object Storage, etc.). The easiest way to set up the connection settings is from the *Dashboard > Settings > Files storage*:



← Prev: Authentication

Next: Working with relations →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation





Introduction

- How to use PocketBase
- Collections
- API rules and filters
- Authentication
- Files upload and handling
- Working with relations**
- Extending PocketBase

Going to production

Web APIs reference

Extend with Go

Extend with JavaScript

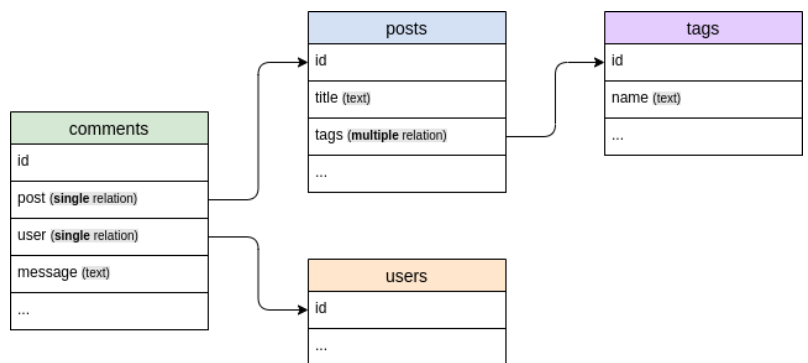
- JS Overview
- JS Event hooks
- JS Routing
- JS Database
- JS Record operations
- JS Collection operations
- JS Migrations
- JS Jobs scheduling
- JS Sending emails
- JS Rendering templates
- JS Console commands

Introduction - Working with relations

- Overview
- Prepend/Append to multiple relation
- Remove from multiple relation
- Expanding relations
- Back-relations
 - Back-relation caveats

Overview

Let's assume that we have the following collections structure:



The **relation** fields follow the same rules as any other collection field and can be set/modified by directly updating the field value - with a record id or array of ids, in case a **multiple relation** is used.

Below is an example that shows creating a new **posts** record with 2 assigned tags.

JavaScript

Dart

```
import PocketBase from 'pocketbase';

const pb = new
PocketBase('http://127.0.0.1:8090');

...

const post = await pb.collection('posts').create({
  'title': 'Lorem ipsum...',
```


JS Sending HTTP requests

JS Realtime messaging

```
'tags': ['TAG_ID1', 'TAG_ID2'],  
});
```

Prepend/Append to multiple relation

To prepend/append a single or multiple relation id(s) to an existing value you can use the **+** field modifier:

JavaScript

Dart

```
import PocketBase from 'pocketbase';  
  
const pb = new  
PocketBase('http://127.0.0.1:8090');  
  
...  
  
const post = await  
pb.collection('posts').update('POST_ID', {  
  // prepend single tag  
  '+tags': 'TAG_ID1',  
  
  // append multiple tags at once  
  'tags+': ['TAG_ID1', 'TAG_ID2'],  
})
```

Remove from multiple relation

To remove a single or multiple relation id(s) from an existing value you can use the **-** field modifier:

JavaScript

Dart

```
import PocketBase from 'pocketbase';  
  
const pb = new  
PocketBase('http://127.0.0.1:8090');  
  
...  
  
const post = await  
pb.collection('posts').update('POST_ID', {  
  // remove single tag  
  'tags-': 'TAG_ID1',  
  
  // remove multiple tags at once
```

```
'tags-': ['TAG_ID1', 'TAG_ID2'],
})
```

Expanding relations

You can also expand record relation fields directly in the returned response without making additional requests by using the `expand` query parameter, e.g. `?expand=user,post.tags`



Only the relations that the request client can **View** (aka. satisfies the relation collection's **View API Rule**) will be expanded.

Nested relation references in `expand`, `filter` or `sort` are supported via dot-notation and up to 6-levels depth.

For example, to list all **comments** with their **user** relation expanded, we can do the following:

JavaScript

Dart

```
await pb.collection("comments").getList(1, 30, {
  expand: "user" })
```

```
{
  "page": 1,
  "perPage": 30,
  "totalPages": 1,
  "totalItems": 20,
  "items": [
    {
      "id": "lmPJt4Z9CkLW36z",
      "collectionId": "BHKW36mJl3ZPt6z",
      "collectionName": "comments",
      "created": "2022-01-01 01:00:00.456Z",
      "updated": "2022-01-01 02:15:00.456Z",
      "post": "WyAw4bDrvws6gGl",
      "user": "FtHAW9feB5rze7D",
      "message": "Example message...",
      "expand": {
        "user": {
          "id": "FtHAW9feB5rze7D",
          "collectionId":
"srmAo0hLxEqYF7F",
```

```

        "collectionName": "users",
        "created": "2022-01-01
00:00:00.000Z",
        "updated": "2022-01-01
00:00:00.000Z",
        "username": "users54126",
        "verified": false,
        "emailVisibility": false,
        "name": "John Doe"
    }
  },
  ...
]
}

```

Back-relations

PocketBase supports also `filter`, `sort` and `expand` for **back-relations** - relations where the associated `relation` field is not in the main collection.

The following notation is used:

`referenceCollection_via_relField` (ex. `comments_via_post`).

For example, let's list the **posts** that have at least one **comments** record containing the word *"hello"*:

JavaScript

Dart

```

await pb.collection("posts").getList(1, 30, {
  filter: "comments_via_post.message ?~ 'hello'",
  expand: "comments_via_post.user",
})

```

```

{
  "page": 1,
  "perPage": 30,
  "totalPages": 2,
  "totalItems": 45,
  "items": [
    {
      "id": "WyAw4bDrvws6gGl",
      "collectionId": "1rAwHJatkTNCUIN",
      "collectionName": "posts",
      "created": "2022-01-01 01:00:00.456Z",
      "updated": "2022-01-01 02:15:00.456Z",

```

```

        "title": "Lorem ipsum dolor sit...",
        "expand": {
            "comments_via_post": [
                {
                    "id": "lmPJt4Z9CkLW36z",
                    "collectionId":
"BHKW36mJl3ZPt6z",
                    "collectionName":
"comments",
                    "created": "2022-01-01
01:00:00.456Z",
                    "updated": "2022-01-01
02:15:00.456Z",
                    "post": "WyAw4bDrvws6gGl",
                    "user": "FtHAW9feB5rze7D",
                    "message": "lorem
ipsum...",
                    "expand": {
                        "user": {
                            "id":
"FtHAW9feB5rze7D",
                            "collectionId":
"srmAo0hLxEqYF7F",
                            "collectionName":
"users",
                            "created": "2022-
01-01 00:00:00.000Z",
                            "updated": "2022-
01-01 00:00:00.000Z",
                            "username":
"users54126",
                            "verified": false,
                            "emailVisibility":
false,
                            "name": "John Doe"
                        }
                    }
                },
                {
                    "id": "tu4Z9CkLW36mPJz",
                    "collectionId":
"BHKW36mJl3ZPt6z",
                    "collectionName":
"comments",
                    "created": "2022-01-01
01:10:00.123Z",
                    "updated": "2022-01-01
02:39:00.456Z",
                    "post": "WyAw4bDrvws6gGl",
                    "user": "FtHAW9feB5rze7D",
                    "message": "hello...",
                    "expand": {

```

```

    "FtHAW9feB5rZe7D",
    "srmAo0hLxEqYF7F",
    "users",
    "01-01 00:00:00.000Z",
    "01-01 00:00:00.000Z",
    "users54126",
    false,
    "user": {
      "id":
      "collectionId":
      "collectionName":
      "created": "2022-
      "updated": "2022-
      "username":
      "verified": false,
      "emailVisibility":
      "name": "John Doe"
    },
    ...
  ],
  ...
]
}

```

Back-relation caveats

- By default the back-relation reference is resolved as a dynamic *multiple* relation field, even when the back-relation field itself is marked as *single*.

This is because the main record could have more than one *single* back-relation reference (see in the above example that the `comments_via_post` expand is returned as array, although the original `comments.post` field is a *single* relation).

The only case where the back-relation will be treated as a *single* relation field is when there is `UNIQUE` index constraint defined on the relation field.

- Back-relation `expand` is limited to max 1000 records per relation field. If you need to fetch

larger number of back-related records a better approach could be to send a separate paginated `getList()` request to the back-related collection to avoid transferring large JSON payloads and to reduce the memory usage.

← Prev: Files upload and handling

Next: Extending PocketBase →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation





Introduction

- How to use PocketBase
- Collections
- API rules and filters
- Authentication
- Files upload and handling
- Working with relations

Extending PocketBase

Going to production

Web APIs reference

Extend with
Go

Extend with
JavaScript

- JS Overview
- JS Event hooks
- JS Routing
- JS Database
- JS Record operations
- JS Collection operations
- JS Migrations
- JS Jobs scheduling
- JS Sending emails
- JS Rendering templates
- JS Console commands

Introduction - Extending PocketBase

One of the main features of PocketBase is that **it can be used as a framework** which enables you to write your own custom app business logic in Go or JavaScript and still have a portable backend at the end.

Choose **Extend with Go** if you are already familiar with the language or have the time to learn it. As the primary PocketBase language, the Go APIs are better documented and you'll be able to integrate with any 3rd party Go library since you'll have more control over the application flow. The only drawback is that the Go APIs are slightly more verbose and it may require some time to get used to, especially if this is your first time working with Go.

Choose **Extend with JavaScript** if you don't intend to write too much custom code and want a quick way to explore the **PocketBase capabilities**. The embedded JavaScript engine is a pluggable wrapper around the existing Go APIs, so most of the time the slight performance penalty will be negligible because it'll invoke the Go functions under the hood.

As a bonus, because the JS VM mirrors the Go APIs, you would be able migrate gradually without much code changes from JS -> Go at later stage in case you hit a bottleneck or want more control over the execution flow.

With both Go and JavaScript, you can:

- Register custom routes:

Go

JavaScript

```
routerAdd("GET", "/hello", (e) => {  
    return e.string(200, "Hello world!")  
})
```

- Bind to event hooks and intercept responses:

Go

JavaScript

JS Sending HTTP requests

JS Realtime messaging

```
onRecordCreateRequest((e) => {  
  // if not superuser, overwrite the newly  
  submitted "posts" record status to pending  
  if (!e.hasSuperuserAuth()) {  
    e.record.set("status", "pending")  
  }  
  
  e.next()  
}, "posts")
```

- Register custom console commands:

Go

JavaScript

```
$app.rootCmd.addCommand(new Command({  
  use: "hello",  
  run: (cmd, args) => {  
    console.log("Hello world!")  
  },  
}))
```

- and many more...

For further info, please check the related [Extend with Go](#) or [Extend with JavaScript](#) guides.

← [Prev: Working with relations](#)

[FAQ](#)

[JavaScript SDK](#)

PocketBase

[Discussions](#)

[Dart SDK](#)

[Documentation](#)



[Introduction](#)[Going to production](#)[Web APIs reference](#)Extend with
GoExtend with
JavaScript

- JS Overview
- JS Event hooks
- JS Routing
- JS Database
- JS Record operations
- JS Collection operations
- JS Migrations
- JS Jobs scheduling
- JS Sending emails
- JS Rendering templates
- JS Console commands
- JS Sending HTTP requests
- JS Realtime messaging
- JS Filesystem
- JS Logging
- JS Types reference

Going to production

- [Deployment strategies](#)
 - [Minimal setup](#)
 - [Using reverse proxy](#)
 - [Using Docker](#)
- [Backup and Restore](#)
- [Recommendations](#)
 - [Use SMTP mail server](#)
 - [Enable MFA for superusers](#)
 - [Enable rate limiter](#)
 - [Increase the open file descriptors limit](#)
 - [Set GOMEMLIMIT](#)
 - [Enable settings encryption](#)

Deployment strategies

Minimal setup

One of the best PocketBase features is that it's completely portable. This means that it doesn't require any external dependency and **could be deployed by just uploading the executable on your server.**

Here is an example of starting a production HTTPS server (auto managed TLS with Let's Encrypt) on a clean Ubuntu 22.04 installation.

0. Consider the following app directory structure:

```
myapp/  
  pb_migrations/  
  pb_hooks/  
  pocketbase
```

1. Upload the binary and anything else required by your application to your remote server, for example using **rsync**:

```
rsync -avz -e ssh /local/path/to/myapp  
root@YOUR_SERVER_IP:/root/pb
```

2. Start a SSH session with your server:

```
ssh root@YOUR_SERVER_IP
```

3. Start the executable (specifying a domain name will issue a Let's encrypt certificate for it)

```
[root@dev ~]$ /root/pb/pocketbase serve yourdomain.com
```

*Notice that in the above example we are logged in as **root** which allows us to bind to the **privileged 80 and 443 ports**.*

*For **non-root** users usually you'll need special privileges to be able to do that. You have several options depending on your OS -*

*authbind, setcap, iptables, sysctl, etc. Here is an example using **setcap**:*

```
[myuser@dev ~]$ sudo setcap  
'cap_net_bind_service=+ep' /root/pb/pocketbase
```

4. (Optional) Systemd service

You can skip step 3 and create a **Systemd service** to allow your application to start/restart on its own.

Here is an example service file (usually created in

`/lib/systemd/system/pocketbase.service`):

```
[Unit]  
Description = pocketbase  
  
[Service]  
Type = simple  
User = root  
Group = root  
LimitNOFILE = 4096  
Restart = always  
RestartSec = 5s  
StandardOutput = append:/root/pb/std.log  
StandardError = append:/root/pb/std.log  
WorkingDirectory = /root/pb  
ExecStart = /root/pb/pocketbase serve  
yourdomain.com  
  
[Install]  
WantedBy = multi-user.target
```

After that we just have to enable it and start the service using **systemctl**:

```
[root@dev ~]$ systemctl enable pocketbase.service  
[root@dev ~]$ systemctl start pocketbase
```

You can find a link to the Web UI installer in the `/root/pb/std.log`, but alternatively you can also create the first superuser explicitly via the `superuser` PocketBase command:

```
[root@dev ~]$ /root/pb/pocketbase superuser create  
EMAIL PASS
```

Using reverse proxy

If you plan on hosting multiple applications on a single server or need finer network controls, you can always put PocketBase behind a reverse proxy such as *NGINX*, *Apache*, *Caddy*, etc.

Just note that when using a reverse proxy you may need to set up the "User IP proxy headers" in the PocketBase settings so that the application can extract and log the actual visitor/client IP (the headers are usually `X-Real-IP`, `X-Forwarded-For`).

Here is a minimal *NGINX* example configuration:

```
server {  
    listen 80;  
    server_name example.com;  
    client_max_body_size 10M;  
  
    location / {  
        # check  
        http://nginx.org/en/docs/http/nginx_http_upstream_module.html#  
        keepalive  
        proxy_set_header Connection '';  
        proxy_http_version 1.1;  
        proxy_read_timeout 360s;  
  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For  
        $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
  
        # enable if you are serving under a subpath location  
        #  
        # note that it is better to use a subdomain when  
        possible because of  
        # the same-origin isolation for localStorage and  
        other resources  
        # rewrite /yourSubpath/(.*) /$1 break;  
  
        proxy_pass http://127.0.0.1:8090;  
    }  
}
```

Corresponding *Caddy* configuration is:

```
example.com {
  request_body {
    max_size 10MB
  }
  reverse_proxy 127.0.0.1:8090 {
    transport http {
      read_timeout 360s
    }
  }
}
```

Using Docker

Some hosts (e.g. fly.io) use Docker for deployments. PocketBase doesn't have an official Docker image, but you could use the below minimal Dockerfile as an example:

```
FROM alpine:latest

ARG PB_VERSION=0.36.7

RUN apk add --no-cache \
    unzip \
    ca-certificates

# download and unzip PocketBase
ADD
https://github.com/pocketbase/pocketbase/releases/download/v
${PB_VERSION}/pocketbase_${PB_VERSION}_linux_amd64.zip
/tmp/pb.zip
RUN unzip /tmp/pb.zip -d /pb/

# uncomment to copy the local pb_migrations dir into the
image
# COPY ./pb_migrations /pb/pb_migrations

# uncomment to copy the local pb_hooks dir into the image
# COPY ./pb_hooks /pb/pb_hooks

EXPOSE 8080

# start PocketBase
CMD ["/pb/pocketbase", "serve", "--http=0.0.0.0:8080"]
```

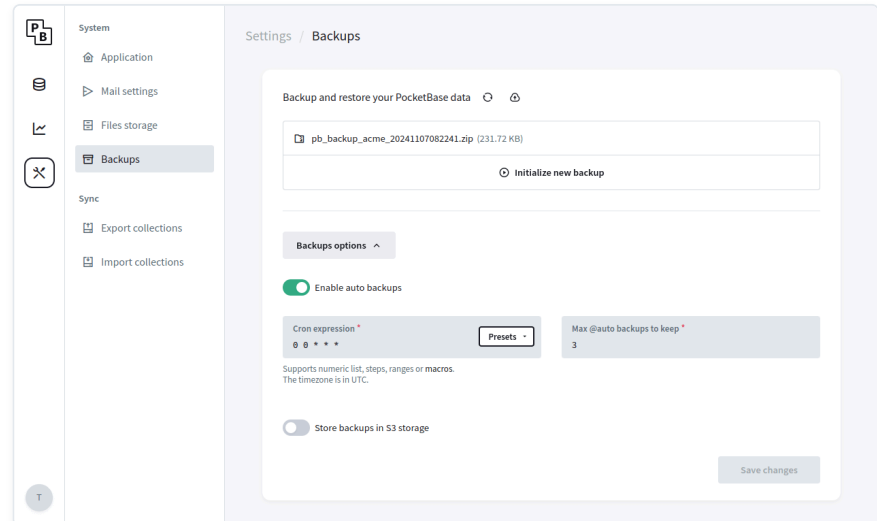
To persist your data you need to mount a volume at `/pb/pb_data`.

For a full example you could check the ["Host for free on Fly.io" guide](#).

Backup and Restore

To backup/restore your application it is enough to manually copy/replace your `pb_data` directory (*for transactional safety make sure that the application is not running*).

To make things slightly easier, PocketBase v0.16+ comes with builtin backups and restore APIs that could be accessed from the Dashboard (*Settings > Backups*):



Backups can be stored locally (default) or in a S3 compatible storage (*it is recommended to use a separate bucket only for the backups*). The generated backup represents a full snapshot as ZIP archive of your `pb_data` directory (including the locally stored uploaded files but excluding any local backups or files uploaded to S3).

During the backup's ZIP generation the application will be temporary set in read-only mode.



Depending on the size of your `pb_data` this could be a very slow operation and it is advised in case of large `pb_data` (e.g. 2GB+) to consider a different backup strategy (*see an example [backup.sh script](#) that combines `sqlite3 .backup` + `rsync`*).

Recommendations

highly recommended

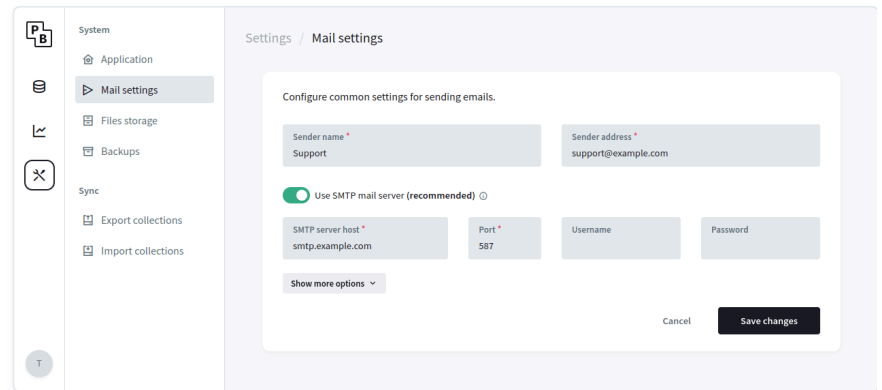
Use SMTP mail server

By default, PocketBase uses the internal Unix `sendmail` command for sending emails.

While it's OK for development, it's not very useful for production, because your emails most likely will get marked as spam or even fail to deliver.

To avoid deliverability issues, consider using a local SMTP server or an external mail service like [MailerSend](#), [Brevo](#), [SendGrid](#), [Mailgun](#), [AWS SES](#), etc.

Once you've decided on a mail service, you could configure the PocketBase SMTP settings from the *Dashboard > Settings > Mail settings* :



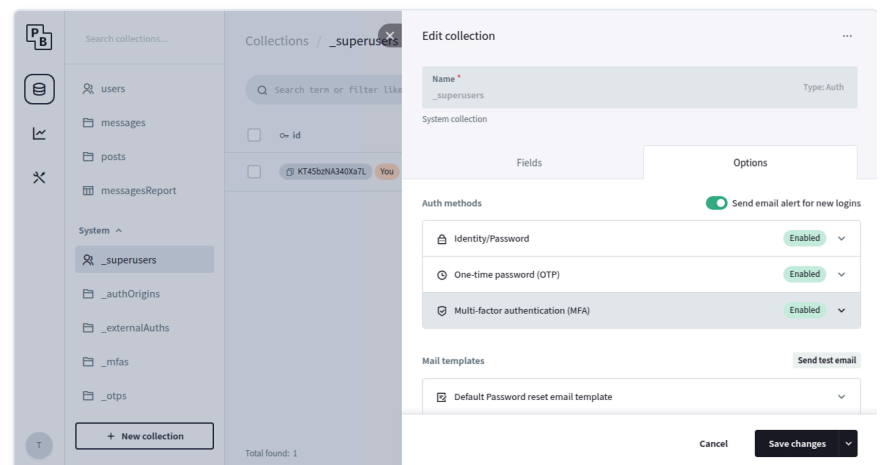
The screenshot shows the 'Mail settings' configuration page in the PocketBase dashboard. On the left is a sidebar with navigation options: System, Application, Mail settings (selected), Files storage, Backups, Sync, Export collections, and Import collections. The main content area is titled 'Settings / Mail settings' and contains a form to 'Configure common settings for sending emails.' The form includes fields for 'Sender name' (filled with 'Support'), 'Sender address' (filled with 'support@example.com'), a toggle for 'Use SMTP mail server (recommended)' which is turned on, 'SMTP server host' (filled with 'smtp.example.com'), 'Port' (filled with '587'), 'Username', and 'Password'. There is a 'Show more options' dropdown and 'Cancel' and 'Save changes' buttons at the bottom right.

highly recommended Enable MFA for superusers

As an additional layer of security you can enable the MFA and OTP options for the `_superusers` collection, which will enforce an additional one-time password (email code) requirement when authenticating as superuser.

In case of email deliverability issues, you can also generate an OTP manually using the

`./pocketbase superuser otp yoursuperuser@example.com` command.



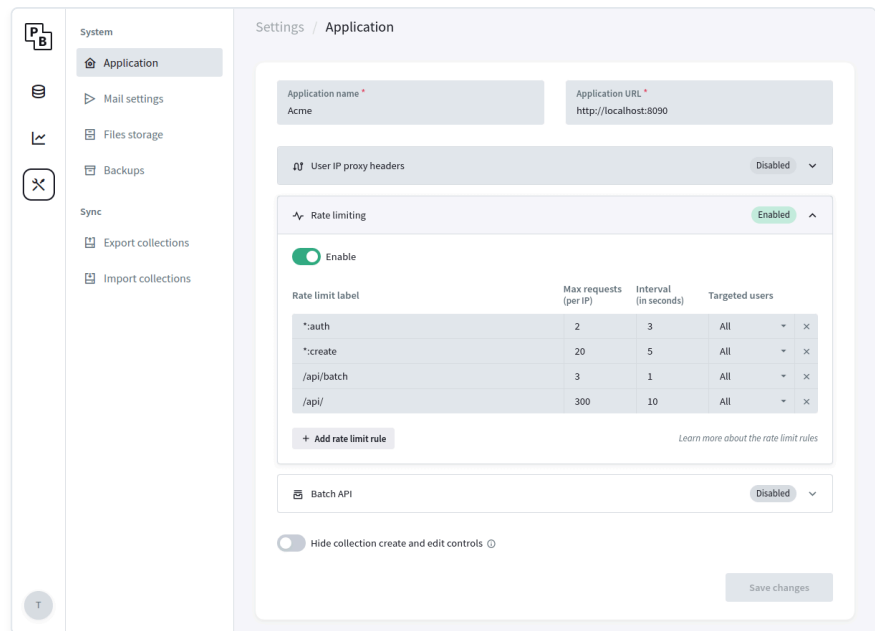
The screenshot shows the 'Edit collection' page for the `_superusers` collection in the PocketBase dashboard. The left sidebar shows the 'Collections' list with `_superusers` selected. The main area is titled 'Edit collection' and shows the collection name `_superusers` and its type 'Auth'. Below this are tabs for 'Fields' and 'Options'. The 'Options' tab is active, showing 'Auth methods' with three options: 'Identity/Password' (Enabled), 'One-time password (OTP)' (Enabled), and 'Multi-factor authentication (MFA)' (Enabled). There is also a 'Send email alert for new logins' toggle which is turned on. At the bottom, there are 'Mail templates' with a 'Send test email' button and a 'Default Password reset email template' dropdown. 'Cancel' and 'Save changes' buttons are at the bottom right.

highly recommended Enable rate limiter

To minimize the risk of API abuse (e.g. excessive auth or record create requests) it is recommended to set up a rate limiter.

PocketBase v0.23.0+ comes with a simple builtin rate limiter that should cover most of the cases but you are also free to use any external one via reverse proxy if you need more advanced options.

You can configure the builtin rate limiter from the *Dashboard > Settings > Application*:



optional Increase the open file descriptors limit

The below instructions are for Linux but other operating systems have similar mechanism.

Unix uses "file descriptors" also for network connections and most systems have a default limit of ~ 1024.

If your application has a lot of concurrent realtime connections, it is possible that at some point you would get an error such as:

Too many open files.

One way to mitigate this is to check your current account resource limits by running `ulimit -a` and find the parameter you want to change. For example, if you want to increase the open files limit (`-n`), you could run `ulimit -n 4096` before starting PocketBase.

optional Set GOMEMLIMIT

If you are running in a memory constrained environment and/or allow upload of large files, defining the `GOMEMLIMIT` environment variable could help preventing out-of-memory (OOM) termination of your process. It is a "soft limit" meaning that the memory usage could still exceed it in some situations, but it instructs the GC to be more "aggressive" and run more often if needed. For example: `GOMEMLIMIT=512MiB`.

If after `GOMEMLIMIT` you are still experiencing OOM errors, you can try to enable swap partitioning (if not already) or open a [Q&A discussion](#) with some steps to reproduce the error in case it is something that we can improve in PocketBase.

optional Enable settings encryption

It is fine to ignore the below if you are not sure whether you need it.

By default, PocketBase stores the applications settings in the database as plain JSON text, including the SMTP password and S3 storage credentials.

While this is not a security issue on its own (PocketBase applications live entirely on a single server and it is expected only authorized users to have access to your server and application data), in some situations it may be a good idea to store the settings encrypted in case someone get their hands on your database file (e.g. from an external stored backup).

To store your PocketBase settings encrypted:

1. Create a new environment variable and **set a random 32 characters** string as its value.

e.g. add

```
export PB_ENCRYPTION_KEY="bLGosYu5k8izvY0hCcR009PgCkKit8VH"
```

in your shell profile file

2. Start the application with `--encryptionEnv=YOUR_ENV_VAR` flag.

e.g. `pocketbase serve --encryptionEnv=PB_ENCRYPTION_KEY`

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



[Introduction](#)[Going to production](#)[Web APIs reference](#)Extend with
GoExtend with
JavaScript[Go Overview](#)[Go Event hooks](#)[Go Routing](#)[Go Database](#)[Go Record operations](#)[Go Collection operations](#)[Go Migrations](#)[Go Jobs scheduling](#)[Go Sending emails](#)[Go Rendering templates](#)[Go Console commands](#)[Go Realtime messaging](#)[Go Filesystem](#)[Go Logging](#)[Go Testing](#)[Go Miscellaneous](#)

Extend with Go - Overview

- [Getting started](#)
- [Custom SQLite driver](#)

Getting started

PocketBase can be used as regular Go package that exposes various helpers and hooks to help you implement your own custom portable application.

A new PocketBase instance is created via `pocketbase.New()` or `pocketbase.NewWithConfig(config)`.

Once created you can register your custom business logic via the available [event hooks](#) and call `app.Start()` to start the application.

Below is a minimal example:

0. [Install Go 1.23+](#)

1. Create a new project directory with `main.go` file inside it.

As a reference, you can also explore the prebuilt executable `example/base/main.go` file.

```
package main

import (
    "log"
    "os"

    "github.com/pocketbase/pocketbase"
    "github.com/pocketbase/pocketbase/apis"
    "github.com/pocketbase/pocketbase/core"
)

func main() {
    app := pocketbase.New()

    app.OnServe().BindFunc(func(se
*core.ServeEvent) error {
    // serves static files from the
    provided public dir (if exists)
    se.Router.GET("/{path...}",
```

```
apis.Static(os.DirFS("./pb_public"), false))

    return se.Next()
})

if err := app.Start(); err != nil {
    log.Fatal(err)
}
}
```

2. To init the dependencies, run

```
go mod init myapp && go mod tidy.
```

3. To start the application, run `go run . serve`.

4. To build a statically linked executable, run `go build` and then you can start the created executable with

```
./myapp serve.
```

Custom SQLite driver



The general recommendation is to use the builtin SQLite setup but if you need more advanced configuration or extensions like ICU, FTS5, etc. you'll have to specify a custom driver/build.

Note that PocketBase by default doesn't require CGO because it uses the pure Go SQLite port modernc.org/sqlite, but this may not be the case when using a custom SQLite driver!

PocketBase v0.23+ added support for defining a `DBConnect` function as app configuration to load custom SQLite builds and drivers compatible with the standard Go `database/sql`.

The `DBConnect` function is called twice - once for `pb_data/data.db` (the main database file) and second time for `pb_data/auxiliary.db` (used for logs and other ephemeral system meta information).

If you want to load your custom driver conditionally and fallback to the default handler, then you can call

```
core.DefaultDBConnect
```

As a side-note, if you are not planning to use `core.DefaultDBConnect` fallback as part of your custom driver registration you can exclude the default pure Go driver with `go build -tags no_default_driver` to reduce the binary size a little (~4MB).

Below are some minimal examples with commonly used external SQLite drivers:

github.com/matttn/go-sqlite3



github.com/ncruces/go-sqlite3



Next: Event hooks →

[FAQ](#)

[JavaScript SDK](#)

PocketBase

[Discussions](#)

[Dart SDK](#)

[Documentation](#)



[Introduction](#)[Going to production](#)[Web APIs reference](#)[API Records](#)[API Realtime](#)[API Files](#)[API Collections](#)[API Settings](#)[API Logs](#)[API Crons](#)[API Backups](#)[API Health](#)[Extend with
Go](#)[Extend with
JavaScript](#)[JS Overview](#)[JS Event hooks](#)[JS Routing](#)[JS Database](#)[JS Record operations](#)[JS Collection operations](#)[JS Migrations](#)[JS Jobs scheduling](#)[JS Sending emails](#)[JS Rendering templates](#)

Web APIs reference - API Records

CRUD actions

[List/Search records](#)[View record](#)[Create record](#)[Update record](#)[Delete record](#)[Batch create/update/upsert/delete records](#)

Auth record actions

[List auth methods](#)[Auth with password](#)[Auth with OAuth2](#)[Auth with OTP](#)[Auth refresh](#)[Verification](#)[Password reset](#)[Email change](#)

JS

Console commands

Impersonate



Next: API Realtime →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



[Introduction](#)[Going to production](#)[Web APIs reference](#)[Extend with
Go](#)[Extend with
JavaScript](#)[JS Overview](#)[JS Event hooks](#)[JS Routing](#)[JS Database](#)[JS Record operations](#)[JS Collection operations](#)[JS Migrations](#)[JS Jobs scheduling](#)[JS Sending emails](#)[JS Rendering templates](#)[JS Console commands](#)[JS Sending HTTP requests](#)[JS Realtime messaging](#)[JS Filesystem](#)[JS Logging](#)

Extend with JavaScript - Overview

- [JavaScript engine](#)
 - [Global objects](#)
- [TypeScript declarations and code completion](#)
- [Caveats and limitations](#)
 - [Handlers scope](#)
 - [Relative paths](#)
 - [Loading modules](#)
 - [Performance](#)
 - [Engine limitations](#)

JavaScript engine

The prebuilt PocketBase v0.17+ executable comes with embedded ES5 JavaScript engine ([goja](#)) which enables you to write custom server-side code using plain JavaScript.

You can start by creating `*.pb.js` file(s) inside a `pb_hooks` directory next to your executable.

```
// pb_hooks/main.pb.js

routerAdd("GET", "/hello/{name}", (e) => {
  let name = e.request.pathValue("name")

  return e.json(200, { "message": "Hello " +
    name })
})

onRecordAfterUpdateSuccess((e) => {
  console.log("user updated...",
    e.record.get("email"))

  e.next()
}, "users")
```

For convenience, when making changes to the files inside `pb_hooks`, the process will automatically restart/reload itself

(currently supported only on UNIX based platforms). The `*.pb.js` files are loaded per their filename sort order.

For most parts, the JavaScript APIs are derived from [Go](#) with 2 main differences:

- Go exported method and field names are converted to camelCase, for example:
`app.FindRecordById("example", "RECORD_ID")`
 becomes
`$app.findRecordById("example", "RECORD_ID")`.
- Errors are thrown as regular JavaScript exceptions and not returned as Go values.

Global objects

Below is a list with some of the commonly used global objects that are accessible from everywhere:

- `__hooks` - The absolute path to the app `pb_hooks` directory.
- `$app` - The current running PocketBase application instance.
- `$apis.*` - API routing helpers and middlewares.
- `$os.*` - OS level primitives (deleting directories, executing shell commands, etc.).
- `$security.*` - Low level helpers for creating and parsing JWTs, random string generation, AES encryption, etc.
- And many more - for all exposed APIs, please refer to the [JSVM reference docs](#).

TypeScript declarations and code completion

While you can't use directly TypeScript (*without transpiling it to JS on your own*), PocketBase comes with builtin **ambient TypeScript declarations** that can help providing information and documentation about the available global variables, methods and arguments, code completion, etc. as long as your editor has TypeScript LSP support (*most editors either have it builtin or available as plugin*).

The types declarations are stored in `pb_data/types.d.ts` file. You can point to those declarations using the [reference triple-](#)

slash directive at the top of your JS file:

```
/// <reference path="../../pb_data/types.d.ts" />

onBootstrap((e) => {
  e.next()

  console.log("App initialized!")
})
```

If after referencing the types your editor still doesn't perform linting, then you can try to rename your file to have `.pb.ts` extension.

Caveats and limitations

Handlers scope

Each handler function (hook, route, middleware, etc.) is **serialized and executed in its own isolated context as a separate "program"**. This means that you don't have access to custom variables and functions declared outside of the handler scope. For example, the below code will fail:

```
const name = "test"

onBootstrap((e) => {
  e.next()

  console.log(name) // <-- name will be
  undefined inside the handler
})
```

The above serialization and isolation context is also the reason why error stack trace line numbers may not be accurate.

One possible workaround for sharing/reusing code across different handlers could be to move and export the reusable code portion as local module and load it with `require()` inside the handler but keep in mind that the loaded modules use a shared registry and mutations should be avoided when possible to prevent concurrency issues:

```
onBootstrap((e) => {
  e.next()
```



```
const config = require(`${__hooks}/config.js`)
console.log(config.name)
})
```

Relative paths

Relative file paths are relative to the current working directory (CWD) and not to the `pb_hooks`.

To get an absolute path to the `pb_hooks` directory you can use the global `__hooks` variable.

Loading modules



Please note that the embedded JavaScript engine is not a Node.js or browser environment, meaning that modules that rely on APIs like *window*, *fs*, *fetch*, *buffer* or any other runtime specific API not part of the ES5 spec may not work!

You can load modules either by specifying their local filesystem path or by using their name, which will automatically search in:

- the current working directory (*affects also relative paths*)
- any `node_modules` directory
- any parent `node_modules` directory

Currently only CommonJS (CJS) modules are supported and can be loaded with `const x = require(...)`.

ECMAScript modules (ESM) can be loaded by first precompiling and transforming your dependencies with a bundler like [rollup](#), [webpack](#), [browserify](#), etc.

A common usage of local modules is for loading shared helpers or configuration parameters, for example:

```
// pb_hooks/utils.js
module.exports = {
  hello: (name) => {
    console.log("Hello " + name)
  }
}
```

```
// pb_hooks/main.pb.js
onBootstrap((e) => {
  e.next()

  const utils = require(`${__hooks}/utils.js`)
  utils.hello("world")
})
```



Loaded modules use a shared registry and mutations should be avoided when possible to prevent concurrency issues.

Performance

The prebuilt executable comes with a **prewarmed pool of 15 JS runtimes**, which helps maintaining the handlers execution times on par with the Go equivalent code (see [benchmarks](#)).

You can adjust the pool size manually with the

`--hooksPool=50` flag (*increasing the pool size may improve the performance in high concurrent scenarios but also will increase the memory usage*).

Note that the handlers performance may degrade if you have heavy computational tasks in pure JavaScript (encryption, random generators, etc.). For such cases prefer using the exposed [Go bindings](#) (e.g. `$security.randomString(10)`).

Engine limitations

We inherit some of the limitations and caveats of the embedded JavaScript engine ([goja](#)):

- Has most of ES6 functionality already implemented but it is not fully spec compliant yet.
- No concurrent execution inside a single handler (aka. no `setTimeout` / `setInterval`).
- Wrapped Go structural types (such as maps, slices) comes with some peculiarities and do not behave the exact same way as native ECMAScript values (for more details see [goja ToValue](#)).
- In relation to the above, DB `json` field values require the use of `get()` and `set()` helpers (*this may change in the future*).

Next: Event hooks →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



[Introduction](#)[Going to production](#)[Web APIs reference](#)[Extend with Go](#)[Extend with JavaScript](#)[Go Overview](#)[Go Event hooks](#)[Go Routing](#)[Go Database](#)[Go Record operations](#)[Go Collection operations](#)[Go Migrations](#)[Go Jobs scheduling](#)[Go Sending emails](#)[Go Rendering templates](#)[Go Console commands](#)[Go Realtime messaging](#)[Go Filesystem](#)[Go Logging](#)[Go Testing](#)[Go Miscellaneous](#)

Extend with Go - Event hooks

The standard way to modify PocketBase is through **event hooks** in your Go code.

All hooks have 3 main methods:

- **Bind(handler)** ↗ adds a new handler to the specified event hook. A handler has 3 fields:
 - **Id** (*optional*) - the name of the handler (could be used as argument for **Unbind**)
 - **Priority** (*optional*) - the execution order of the handler (if empty fallbacks to the order of registration in the code).
 - **Func** (*required*) - the handler function.
- **BindFunc(func)** ↗ is similar to **Bind** but registers a new handler from just the provided function. The registered handler is added with a default 0 priority and the id is autogenerated (the returned string value).
- **Trigger(event, oneOffHandlerFuncs...)** ↗ triggers the event hook.

This method rarely has to be called manually by users.

To remove an already registered hook handler, you can use the handler id and pass it to **Unbind(id)** or remove all handlers with **UnbindAll()** (!including system handlers).

All hook handler functions share the same **func(e T) error** signature and expect the user to call **e.Next()** if they want to proceed with the execution chain.



If you need to access the app instance from inside a hook handler, prefer using the **e.App** field instead of reusing a parent scope app variable because the hook could be part of a DB transaction and can cause deadlock.

Also avoid using global mutex locks inside a hook handler because it could be invoked recursively

(e.g. cascade delete) and can cause deadlock.

You can explore all available hooks below:

- [App hooks](#)
- [Mailer hooks](#)
- [Realtime hooks](#)
- [Record model hooks](#)
- [Collection model hooks](#)
- [Request hooks](#)
- [Base model hooks](#)

App hooks

OnBootstrap	▼
OnServe	▼
OnSettingsReload	▼
OnBackupCreate	▼
OnBackupRestore	▼
OnTerminate	▼

Mailer hooks

OnMailerSend	▼
OnMailerRecordAuthAlertSend	▼
OnMailerRecordPasswordResetSend	▼
OnMailerRecordVerificationSend	▼

OnMailerRecordEmailChangeSend	▼
-------------------------------	---

OnMailerRecordOTPSend	▼
-----------------------	---

Realtime hooks

OnRealtimeConnectRequest	▼
--------------------------	---

OnRealtimeSubscribeRequest	▼
----------------------------	---

OnRealtimeMessageSend	▼
-----------------------	---

Record model hooks



These are lower level Record model hooks and could be triggered from anywhere (custom console command, scheduled cron job, when calling `e.Save(record)`, etc.) and therefore they have no access to the request context!

If you want to intercept the builtin Web APIs and to access their request body, query parameters, headers or the request auth state, then please use the designated Record ***Request** hooks.

OnRecordEnrich	▼
----------------	---

OnRecordValidate	▼
------------------	---

Record model create hooks

OnRecordCreate	▼
----------------	---

OnRecordCreateExecute	▼
-----------------------	---

OnRecordAfterCreateSuccess	▼
----------------------------	---

OnRecordAfterCreateError



Record model update hooks

OnRecordUpdate



OnRecordUpdateExecute



OnRecordAfterUpdateSuccess



OnRecordAfterUpdateError



Record model delete hooks

OnRecordDelete



OnRecordDeleteExecute



OnRecordAfterDeleteSuccess



OnRecordAfterDeleteError



Collection model hooks



These are lower level Collection model hooks and could be triggered from anywhere (custom console command, scheduled cron job, when calling `e.Save(collection)`, etc.) and therefore they have no access to the request context!

If you want to intercept the builtin Web APIs and to access their request body, query parameters, headers or the request auth state, then please use the designated Collection ***Request** hooks .

OnCollectionValidate



Collection mode create hooks

OnCollectionCreate



OnCollectionCreateExecute



OnCollectionAfterCreateSuccess



OnCollectionAfterCreateError



Collection mode update hooks

OnCollectionUpdate



OnCollectionUpdateExecute



OnCollectionAfterUpdateSuccess



OnCollectionAfterUpdateError



Collection mode delete hooks

OnCollectionDelete



OnCollectionDeleteExecute



OnCollectionAfterDeleteSuccess



OnCollectionAfterDeleteError



Request hooks

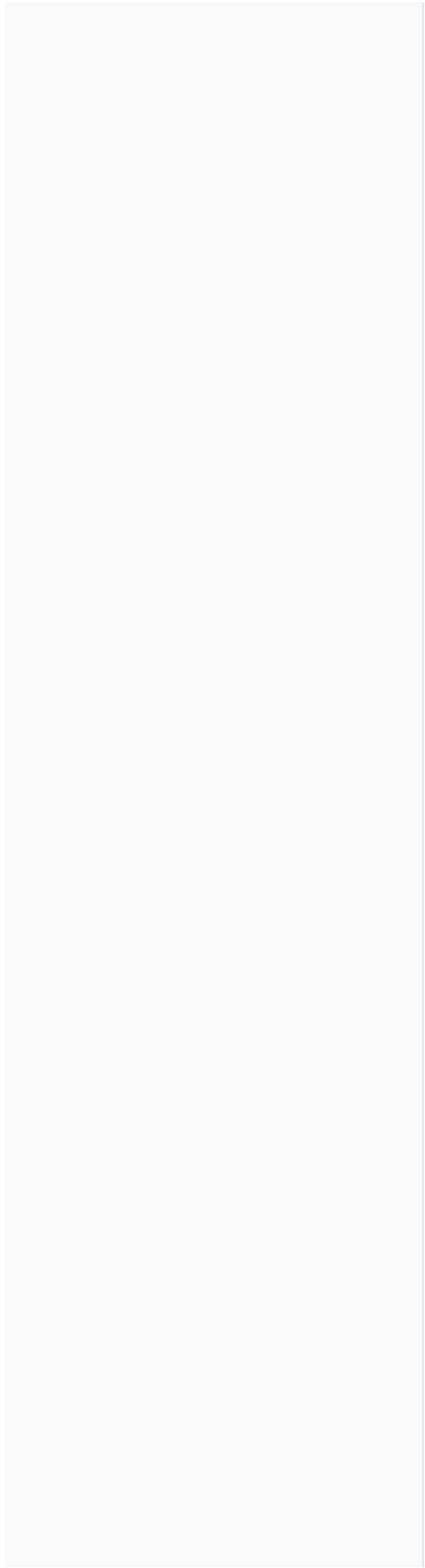


The request hooks are triggered only when the corresponding API request endpoint is accessed.

Record CRUD request hooks

OnRecordsListRequest





OnRecordViewRequest	▼
OnRecordCreateRequest	▼
OnRecordUpdateRequest	▼
OnRecordDeleteRequest	▼

Record auth request hooks

OnRecordAuthRequest	▼
OnRecordAuthRefreshRequest	▼
OnRecordAuthWithPasswordRequest	▼
OnRecordAuthWithOAuth2Request	▼
OnRecordRequestPasswordResetRequest	▼
OnRecordConfirmPasswordResetRequest	▼
OnRecordRequestVerificationRequest	▼
OnRecordConfirmVerificationRequest	▼
OnRecordRequestEmailChangeRequest	▼
OnRecordConfirmEmailChangeRequest	▼
OnRecordRequestOTPRequest	▼
OnRecordAuthWithOTPRequest	▼

Batch request hooks

OnBatchRequest	▼
----------------	---

File request hooks

OnFileDownloadRequest	▼
OnFileTokenRequest	▼

Collection request hooks

OnCollectionsListRequest	▼
OnCollectionViewRequest	▼
OnCollectionCreateRequest	▼
OnCollectionUpdateRequest	▼
OnCollectionDeleteRequest	▼
OnCollectionsImportRequest	▼

Settings request hooks

OnSettingsListRequest	▼
OnSettingsUpdateRequest	▼

Base model hooks

The Model hooks are fired for all PocketBase structs that implements the Model DB interface - Record, Collection, Log, etc.



For convenience, if you want to listen to only the Record or Collection DB model events without doing manual type assertion, you can use the **OnRecord*** and **OnCollection*** proxy hooks above.

OnModelValidate



Base model create hooks

OnModelCreate



OnModelCreateExecute



OnModelAfterCreateSuccess



OnModelAfterCreateError



Base model update hooks

OnModelUpdate



OnModelUpdateExecute



OnModelAfterUpdateSuccess



OnModelAfterUpdateError



Base model delete hooks

OnModelDelete



OnModelDeleteExecute



OnModelAfterDeleteSuccess



OnModelAfterDeleteError



← Prev: Overview

Next: Routing →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



[Introduction](#)[Going to production](#)[Web APIs reference](#)[Extend with
Go](#)[Extend with
JavaScript](#)[Go Overview](#)[Go Event hooks](#)[Go Routing](#)[Go Database](#)[Go Record operations](#)[Go Collection operations](#)[Go Migrations](#)[Go Jobs scheduling](#)[Go Sending emails](#)[Go Rendering templates](#)[Go Console commands](#)[Go Realtime messaging](#)[Go Filesystem](#)[Go Logging](#)[Go Testing](#)[Go Miscellaneous](#)[Go Record proxy](#)

Extend with Go - Routing

PocketBase routing is built on top of the standard Go

`net/http.ServeMux`. The router can be accessed via the

`app.OnServe()` hook allowing you to register custom endpoints and middlewares.

- **Routes**

- [Registering new routes](#)
- [Path parameters and matching rules](#)
- [Reading path parameters](#)
- [Retrieving the current auth state](#)
- [Reading query parameters](#)
- [Reading request headers](#)
- [Writing response headers](#)
- [Retrieving uploaded files](#)
- [Reading request body](#)
- [Writing response body](#)
- [Reading the client IP](#)
- [Request store](#)

- **Middleware**

- [Registering middlewares](#)
- [Removing middlewares](#)
- [Builtin middlewares](#)
- [Default globally registered middlewares](#)

- **Error response**

- **Helpers**

- [Serving static directory](#)
- [Auth response](#)
- [Enrich record\(s\)](#)
- [Go http.Handler wrappers](#)

- **Sending request to custom routes using the SDKs**

Routes

Registering new routes

Every route has a path, handler function and eventually middlewares attached to it. For example:

```
app.OnServe().BindFunc(func(se *core.ServeEvent) error {
    // register "GET /hello/{name}" route (allowed for
    everyone)
    se.Router.GET("/hello/{name}", func(e
    *core.RequestEvent) error {
        name := e.Request.PathValue("name")

        return e.String(http.StatusOK, "Hello " +
        name)
    })

    // register "POST /api/myapp/settings" route
    (allowed only for authenticated users)
    se.Router.POST("/api/myapp/settings", func(e
    *core.RequestEvent) error {
        // do something ...
        return e.JSON(http.StatusOK,
        map[string]bool{"success": true})
    }).Bind(apis.RequireAuth())

    return se.Next()
})
```

There are several routes registration methods available, but the most common ones are:

```
se.Router.GET(path, action)
se.Router.POST(path, action)
se.Router.PUT(path, action)
se.Router.PATCH(path, action)
se.Router.DELETE(path, action)

// If you want to handle any HTTP method define only a
path (e.g. "/example")
// OR if you want to specify a custom one add it as
prefix to the path (e.g. "TRACE /example")
se.Router.Any(pattern, action)
```

The router also supports creating groups for routes that share the same base path and middlewares. For example:

```
g :=
se.Router.Group("/api/
myapp")

// group middleware
```

The example registers the following endpoints (all require authenticated user access):

```
g.Bind(apis.RequireAuth())
```

```
// group routes
g.GET("", action1)
g.GET("/example/{id}", action2)
g.PATCH("/example/{id}", action3).BindFunc(
    /* custom route specific middleware
    func */
)
```

```
// nested group
sub := g.Group("/sub")
sub.GET("/sub1", action4)
```

- GET /api/myapp -> action1
- GET /api/myapp/example/{id} -> action2
- PATCH /api/myapp/example/{id} -> action3
- GET /api/myapp/example/sub/sub1 -> action4

Each router group and route could define middlewares in a similar manner to the regular app hooks via the `Bind/BindFunc` methods, allowing you to perform various BEFORE or AFTER action operations (e.g. inspecting request headers, custom access checks, etc.).

Path parameters and matching rules

Because PocketBase routing is based on top of the Go standard router mux, we follow the same pattern matching rules. Below you could find a short overview but for more details please refer to [net/http.ServeMux](https://golang.org/pkg/net/http/httpmux/).

In general, a route pattern looks like `[METHOD][HOST]/[PATH]` (the *METHOD* prefix is added automatically when using the designated `GET()`, `POST()`, etc. methods).

Route paths can include parameters in the format `{paramName}`. You can also use `{paramName...}` format to specify a parameter that targets more than one path segment.

A pattern ending with a trailing slash `/` acts as anonymous wildcard and matches any requests that begins with the defined route. If you want to have a trailing slash but to indicate the end of the URL then you need to end the path with the special `{$}` parameter.

If your route path starts with `/api/` consider combining it with your unique app name like



`/api/myapp/...` to avoid collisions with system routes.

Here are some examples:

```
// match "GET example.com/index.html"
se.Router.GET("example.com/index.html")

// match "GET /index.html" (for any host)
se.Router.GET("/index.html")

// match "GET /static/", "GET /static/a/b/c", etc.
se.Router.GET("/static/")

// match "GET /static/", "GET /static/a/b/c", etc.
// (similar to the above but with a named wildcard
parameter)
se.Router.GET("/static/{path...}")

// match only "GET /static/" (if no "/static" is
registered, it is 301 redirected)
se.Router.GET("/static/{${}")

// match "GET /customers/john", "GET /customers/jane",
etc.
se.Router.GET("/customers/{name}")
```

In the following examples `e` is usually `*core.RequestEvent` value.

Reading path parameters

```
id := e.Request.PathValue("id")
```

Retrieving the current auth state

The request auth state can be accessed (or set) via the `RequestEvent.Auth` field.

```
authRecord := e.Auth

isGuest := e.Auth == nil
```



```
// the same as "e.Auth != nil && e.Auth.IsSuperuser()"
isSuperuser := e.HasSuperuserAuth()
```

Alternatively you could also access the request data from the summarized request info instance *(usually used in hooks like the `OnRecordEnrich` where there is no direct access to the request)*.

```
info, err := e.RequestInfo()

authRecord := info.Auth

isGuest := info.Auth == nil

// the same as "info.Auth != nil &&
info.Auth.IsSuperuser()"
isSuperuser := info.HasSuperuserAuth()
```

Reading query parameters

```
// retrieve the first value of the "search" query
param
search := e.Request.URL.Query().Get("search")

// or via the parsed request info
info, err := e.RequestInfo()
search := info.Query["search"]

// in case of array query params (e.g.
search=123&search=456)
arr := e.Request.URL.Query()["search"] //
[]string{"123", "456"}
```

Reading request headers

```
token := e.Request.Header.Get("Some-Header")

// or via the parsed request info
// (the header value is always normalized per the
@request.headers.* API rules format)
info, err := e.RequestInfo()
token := info.Headers["some_header"]
```

Writing response headers

```
e.Response.Header().Set("Some-Header", "123")
```

Retrieving uploaded files

```
// retrieve the uploaded files and parse the found
multipart data into a ready-to-use []*filesystem.File
files, err := e.FindUploadedFiles("document")

// or retrieve the raw single multipart/form-data file
and header
mf, mh, err := e.Request.FormFile("document")
```

Reading request body

Body parameters can be read either via `e.BindBody` OR through the parsed request info (*requires manual type assertions*).

The `e.BindBody` argument must be a pointer to a struct or `map[string]any`.

The following struct tags are supported (*the specific binding rules and which one will be used depend on the request Content-Type*):

- `json` (json body)- uses the builtin Go JSON package for unmarshaling.
- `xml` (xml body) - uses the builtin Go XML package for unmarshaling.
- `form` (form data) - utilizes the custom `router.UnmarshalRequestData` method.

NB! When binding structs make sure that they don't have public fields that shouldn't be bindable and it is advisable such fields to be unexported or define a separate struct with just the safe bindable fields.

```
// read/scan the request body fields into a typed
struct
data := struct {
    // unexported to prevent binding
    somethingPrivate string

    Title      string `json:"title" form:"title"`
    Description string `json:"description"
form:"description"`
    Active     bool  `json:"active" form:"active"`
}{}
if err := e.BindBody(&data); err != nil {
    return e.BadRequestError("Failed to read request
data", err)
}

// alternatively, read the body via the parsed request
info
```

```
info, err := e.RequestInfo()
title, ok := info.Body["title"].(string)
```

Writing response body

For all supported methods, you can refer to `router.Event` .

```
// send response with JSON body
// (it also provides a generic response fields
// picker/filter if the "fields" query parameter is set)
e.JSON(http.StatusOK, map[string]any{"name": "John"})

// send response with string body
e.String(http.StatusOK, "Lorem ipsum...")

// send response with HTML body
// (check also the "Rendering templates" section)
e.HTML(http.StatusOK, "<h1>Hello!</h1>")

// redirect
e.Redirect(http.StatusTemporaryRedirect,
    "https://example.com")

// send response with no body
e.NoContent(http.StatusNoContent)

// serve a single file
e.FileFS(os.DirFS("."), "example.txt")

// stream the specified reader
e.Stream(http.StatusOK, "application/octet-stream",
    reader)

// send response with blob (bytes slice) body
e.Blob(http.StatusOK, "application/octet-stream",
    []byte{ ... })
```

Reading the client IP

```
// The IP of the last client connecting to your
// server.
// The returned IP is safe and can be always trusted.
// When behind a reverse proxy (e.g. nginx) this
// method returns the IP of the proxy.
//
// https://pkg.go.dev/github.com/pocketbase/pocketbase/tools/router#Event.RemoteIP
ip := e.RemoteIP()

// The "real" IP of the client based on the configured
// Settings.TrustedProxy header(s).
```

```
// If such headers are not set, it fallbacks to
e.RemoteIP().
//
https://pkg.go.dev/github.com/pocketbase/pocketbase/co
re#RequestEvent.RealIP
ip := e.RealIP()
```

Request store

The `core.RequestEvent` comes with a local store that you can use to share custom data between middlewares and the route action.

```
// store for the duration of the request
e.Set("someKey", 123)

// retrieve later
val := e.Get("someKey").(int) // 123
```

Middlewares

Registering middlewares

Middlewares allow inspecting, intercepting and filtering route requests.

All middleware functions share the same signature with the route actions (aka. `func(e *core.RequestEvent) error`) but expect the user to call `e.Next()` if they want to proceed with the execution chain.

Middlewares can be registered *globally*, on *group* and on *route* level using the `Bind` and `BindFunc` methods.

Here is a minimal example of what a global middleware looks like:

```
app.OnServe().BindFunc(func(se *core.ServeEvent) error {
    // register a global middleware
    se.Router.BindFunc(func(e *core.RequestEvent)
error {
        if e.Request.Header.Get("Something") == "" {
            return e.BadRequestError("Something header
value is missing!", nil)
        }
        return e.Next()
    })

    return se.Next()
})
```

`RouterGroup.Bind(middlewares...) ↗ /`

`Route.Bind(middlewares...) ↗` registers one or more middleware handlers.

Similar to the other app hooks, a middleware handler has 3 fields:

- `Id` (*optional*) - the name of the middleware (could be used as argument for `Unbind`)
- `Priority` (*optional*) - the execution order of the middleware (if empty fallbacks to the order of registration in the code)
- `Func` (*required*) - the middleware handler function

Often you don't need to specify the `Id` or `Priority` of the middleware and for convenience you can instead use directly

`RouterGroup.BindFunc(funcs...) ↗ /`

`Route.BindFunc(funcs...) ↗` .

Below is a slightly more advanced example showing all options and the execution sequence (2,0,1,3,4):

```
app.OnServe().BindFunc(func(se *core.ServeEvent) error {
    // attach global middleware
    se.Router.BindFunc(func(e *core.RequestEvent)
error {
    println(0)
    return e.Next()
})

    g := se.Router.Group("/sub")

    // attach group middleware
    g.BindFunc(func(e *core.RequestEvent) error {
        println(1)
        return e.Next()
    })

    // attach group middleware with an id and custom
    priority
    g.Bind(&hook.Handler[*core.RequestEvent]{
        Id: "something",
        Func: func(e *core.RequestEvent) error {
            println(2)
            return e.Next()
        },
        Priority: -1,
    })

    // attach middleware to a single route
    //
    // "GET /sub/hello" should print the sequence:
    2,0,1,3,4
```

```

g.GET("/hello", func(e *core.RequestEvent) error {
    println(4)
    return e.String(200, "Hello!")
}).BindFunc(func(e *core.RequestEvent) error {
    println(3)
    return e.Next()
})

return se.Next()
})

```

Removing middlewares

To remove a registered middleware from the execution chain for a specific group or route you can make use of the `Unbind(id)` method.

Note that only middlewares that have a non-empty `Id` can be removed.

```

app.OnServe().BindFunc(func(se *core.ServeEvent) error {
    // global middleware
    se.Router.Bind(&hook.Handler[*core.RequestEvent]{
        Id: "test",
        Func: func(e *core.RequestEvent) error {
            // ...
            return e.Next()
        },
    })

    // "GET /A" invokes the "test" middleware
    se.Router.GET("/A", func(e *core.RequestEvent)
error {
    return e.String(200, "A")
})

    // "GET /B" doesn't invoke the "test" middleware
    se.Router.GET("/B", func(e *core.RequestEvent)
error {
    return e.String(200, "B")
}).Unbind("test")

    return se.Next()
})

```

Builtin middlewares

The `apis` package exposes several middlewares that you can use as part of your application.

```

// Require the request client to be unauthenticated
(aka. guest).
// Example: Route.Bind(apis.RequireGuestOnly())
apis.RequireGuestOnly()

// Require the request client to be authenticated
// (optionally specify a list of allowed auth
collection names, default to any).
// Example: Route.Bind(apis.RequireAuth())
apis.RequireAuth(optCollectionNames...)

// Require the request client to be authenticated as
superuser
// (this is an alias for
apis.RequireAuth(core.CollectionNameSuperusers)).
// Example: Route.Bind(apis.RequireSuperuserAuth())
apis.RequireSuperuserAuth()

// Require the request client to be authenticated as
superuser OR
// regular auth record with id matching the specified
route parameter (default to "id").
// Example:
Route.Bind(apis.RequireSuperuserOrOwnerAuth(""))
apis.RequireSuperuserOrOwnerAuth(ownerIdParam)

// Changes the global 32MB default request body size
limit (set it to 0 for no limit).
// Note that system record routes have dynamic body
size limit based on their collection field types.
// Example: Route.Bind(apis.BodyLimit(10 <= 20))
apis.BodyLimit(limitBytes)

// Compresses the HTTP response using Gzip compression
scheme.
// Example: Route.Bind(apis.Gzip())
apis.Gzip()

// Instructs the activity logger to log only requests
that have failed/returned an error.
// Example: Route.Bind(apis.SkipSuccessActivityLog())
apis.SkipSuccessActivityLog()

```

Default globally registered middlewares

The below list is mostly useful for users that may want to plug their own custom middlewares before/after the priority of the default global ones, for example: registering a custom auth loader before the rate limiter with `apis.DefaultRateLimitMiddlewarePriority - 1` so that the rate limit can be applied properly based on the loaded auth state.

All PocketBase applications have the below internal middlewares registered out of the box (*sorted by their priority*):

- **WWW redirect**

`apis.DefaultWWWRedirectMiddlewareId` ↗

`apis.DefaultWWWRedirectMiddlewarePriority` ↗

Performs www -> non-www redirect(s) if the request host matches with one of the values in certificate host policy.

- **CORS**

`apis.DefaultCorsMiddlewareId` ↗

`apis.DefaultCorsMiddlewarePriority` ↗

By default all origins are allowed (PocketBase is stateless and doesn't rely on cookies) and can be configured with the `--origins` flag but for more advanced customization it can be also replaced entirely by binding with `apis.CORS(config)` middleware or registering your own custom one in its place.

- **Activity logger**

`apis.DefaultActivityLoggerMiddlewareId` ↗

`apis.DefaultActivityLoggerMiddlewarePriority` ↗

Saves request information into the logs auxiliary database.

- **Auto panic recover**

`apis.DefaultPanicRecoverMiddlewareId` ↗

`apis.DefaultPanicRecoverMiddlewarePriority` ↗

Default panic-recover handler.

- **Auth token loader**

`apis.DefaultLoadAuthTokenMiddlewareId` ↗

`apis.DefaultLoadAuthTokenMiddlewarePriority` ↗

Loads the auth token from the `Authorization` header and populates the related auth record into the request event (aka. `e.Auth`).

- **Security response headers**

`apis.DefaultSecurityHeadersMiddlewareId` ↗

`apis.DefaultSecurityHeadersMiddlewarePriority` ↗

Adds default common security headers (`X-XSS-Protection` , `X-Content-Type-Options` , `X-Frame-Options`) to the response (can be overwritten by other middlewares or from inside the route action).

- **Rate limit**

`apis.DefaultRateLimitMiddlewareId` ↗

`apis.DefaultRateLimitMiddlewarePriority` ↗

Rate limits client requests based on the configured app settings (it does nothing if the rate limit option is not enabled).

- **Body limit**

`apis.DefaultBodyLimitMiddlewareId` ↗

`apis.DefaultBodyLimitMiddlewarePriority` ↗

Applies a default max ~32MB request body limit for all custom routes (system record routes have dynamic body size limit based on their collection field types). Can be overwritten on

group/route level by simply rebinding the `apis.BodyLimit(limitBytes)` middleware.

Error response

PocketBase has a global error handler and every returned error from a route or middleware will be safely converted by default to a generic `ApiError` to avoid accidentally leaking sensitive information (the original raw error message will be visible only in the *Dashboard > Logs* or when in `--dev` mode).

To make it easier returning formatted JSON error responses, the request event provides several `ApiError` methods.

Note that `ApiError.RawData()` will be returned in the response only if it is a map of `router.SafeErrorItem` / `validation.Error` items.

```
import validation "github.com/go-ozzo/ozzo-validation/v4"

se.Router.GET("/example", func(e *core.RequestEvent)
error {
    ...

    // construct ApiError with custom status code and
    validation data error
    return e.Error(500, "something went wrong",
map[string]validation.Error{
    "title": validation.NewError("invalid_title",
    "Invalid or missing title"),
    })

    // if message is empty string, a default one will
    be set
    return e.BadRequestError(optMessage, optData)
// 400 ApiError
    return e.UnauthorizedError(optMessage, optData)
// 401 ApiError
    return e.ForbiddenError(optMessage, optData)
// 403 ApiError
    return e.NotFoundError(optMessage, optData)
// 404 ApiError
    return e.TooManyRequestsError(optMessage, optData)
// 429 ApiError
    return e.InternalServerError(optMessage, optData)
// 500 ApiError
})
```

This is not very common but if you want to return `ApiError` outside of request related handlers, you can use the below `apis.*` factories:

```
import (
    validation "github.com/go-ozzo/ozzo-validation/v4"
    "github.com/pocketbase/pocketbase/apis"
)

app.OnRecordCreate().BindFunc(func(e
*core.RecordEvent) error {
    ...

    // construct ApiError with custom status code and
    validation data error
    return apis.NewApiError(500, "something went
wrong", map[string]validation.Error{
        "title": validation.NewError("invalid_title",
        "Invalid or missing title"),
    })

    // if message is empty string, a default one will
    be set
    return apis.NewBadRequestError(optMessage,
optData) // 400 ApiError
    return apis.NewUnauthorizedError(optMessage,
optData) // 401 ApiError
    return apis.NewForbiddenError(optMessage, optData)
// 403 ApiError
    return apis.NewNotFoundError(optMessage, optData)
// 404 ApiError
    return apis.NewTooManyRequestsError(optMessage,
optData) // 429 ApiError
    return apis.NewInternalServerError(optMessage,
optData) // 500 ApiError
})
```

Helpers

Serving static directory

`apis.Static()` serves static directory content from `fs.FS` instance.

Expects the route to have a `{path...}` wildcard parameter.

```
app.OnServe().BindFunc(func(se *core.ServeEvent) error {
    // serves static files from the provided dir (if
```

```

exists)
    se.Router.GET("/{path...}",
apis.Static(os.DirFS("/path/to/public"), false))

    return se.Next()
})

```

Auth response

apis.RecordAuthResponse() [↗] writes standardized JSON record auth response (aka. token + record data) into the specified request body. Could be used as a return result from a custom auth route.

```

app.OnServe().BindFunc(func(se *core.ServeEvent) error
{
    se.Router.POST("/phone-login", func(e
*core.RequestEvent) error {
        data := struct {
            Phone    string `json:"phone"
form:"phone"`
            Password string `json:"password"
form:"password"`
        }{}
        if err := e.BindBody(&data); err != nil {
            return e.BadRequestError("Failed to read
request data", err)
        }

        record, err :=
e.App.FindFirstRecordByData("users", "phone",
data.Phone)
        if err != nil ||
!record.ValidatePassword(data.Password) {
            // return generic 400 error as a basic
enumeration protection
            return e.BadRequestError("Invalid
credentials", err)
        }

        return apis.RecordAuthResponse(e, record,
"phone", nil)
    })

    return se.Next()
})

```

Enrich record(s)

apis.EnrichRecord() [↗] and **apis.EnrichRecords()** [↗] helpers parses the request context and enrich the provided record(s) by:

- expands relations (if `defaultExpands` and/or `?expand` query parameter is set)
- ensures that the emails of the auth record and its expanded auth relations are visible only for the current logged superuser, record owner or record with manage access

```
app.OnServe().BindFunc(func(se *core.ServeEvent) error {
    se.Router.GET("/custom-article", func(e
    *core.RequestEvent) error {
        records, err :=
e.App.FindRecordsByFilter("article", "status =
'active'", "-created", 40, 0)
        if err != nil {
            return e.NotFoundError("No active
articles", err)
        }

        // enrich the records with the "categories"
relation as default expand
        err = apis.EnrichRecords(e, records,
"categories")
        if err != nil {
            return err
        }

        return e.JSON(http.StatusOK, records)
    })

    return se.Next()
})
```

Go http.Handler wrappers

If you want to register standard Go `http.Handler` function and middlewares, you can use `apis.WrapStdHandler(handler) ↗` and `apis.WrapStdMiddleware(func) ↗` functions.

Sending request to custom routes using the SDKs

The official PocketBase SDKs expose the internal `send()` method that could be used to send requests to your custom route(s).

JavaScript

Dart

```
import PocketBase from 'pocketbase';

const pb = new PocketBase('http://127.0.0.1:8090');
```

```
await pb.send("/hello", {  
  // for other options check  
  // https://developer.mozilla.org/en-US/docs/Web/API/fetch#options  
  query: { "abc": 123 },  
});
```

← Prev: Event hooks

Next: Database →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



[Introduction](#)[Going to production](#)[Web APIs reference](#)Extend with
GoExtend with
JavaScript[Go Overview](#)[Go Event hooks](#)[Go Routing](#)[Go Database](#)[Go Record operations](#)[Go Collection operations](#)[Go Migrations](#)[Go Jobs scheduling](#)[Go Sending emails](#)[Go Rendering templates](#)[Go Console commands](#)[Go Realtime messaging](#)[Go Filesystem](#)[Go Logging](#)[Go Testing](#)[Go Miscellaneous](#)

Extend with Go - Database

`core.App` is the main interface to interact with the database.

`App.DB()` returns a `dbx.Builder` that can run all kinds of SQL statements, including raw queries.

Most of the common DB operations are listed below, but you can find further information in the [dbx package godoc](#).



For more details and examples how to interact with Record and Collection models programmatically you could also check [Collection operations](#) and [Record operations](#) sections.

- [Executing queries](#)
- [Binding parameters](#)
- [Query builder](#)
 - [Select\(\)](#), [AndSelect\(\)](#), [Distinct\(\)](#).
 - [From\(\)](#).
 - [Join\(\)](#).
 - [Where\(\)](#), [AndWhere\(\)](#), [OrWhere\(\)](#).
 - [OrderBy\(\)](#), [AndOrderBy\(\)](#).
 - [GroupBy\(\)](#), [AndGroupBy\(\)](#).
 - [Having\(\)](#), [AndHaving\(\)](#), [OrHaving\(\)](#).
 - [Limit\(\)](#).
 - [Offset\(\)](#).
- [Transaction](#)

Executing queries

To execute DB queries you can start with the `NewQuery("...")` statement and then call one of:

- `Execute()` - for any query statement that is not meant to retrieve data:

```
res, err := app.DB().
    NewQuery("DELETE FROM articles WHERE
status = 'archived'").
    Execute()
```

- `One()` - to populate a single row into a struct:

```
type User struct {
    Id      string          `db:"id"
    json:"id"`
    Status  bool
    `db:"status" json:"status"`
    Age     int             `db:"age"
    json:"age"`
    Roles   types.JSONArray[string] `db:"roles"
    json:"roles"`
}

user := User{}

err := app.DB().
    NewQuery("SELECT id, status, age, roles
FROM users WHERE id=1").
    One(&user)
```

- `All()` - to populate multiple rows into a slice of structs:

```
type User struct {
    Id      string          `db:"id"
    json:"id"`
    Status  bool
    `db:"status" json:"status"`
    Age     int             `db:"age"
    json:"age"`
    Roles   types.JSONArray[string] `db:"roles"
    json:"roles"`
}

users := []User{}

err := app.DB().
    NewQuery("SELECT id, status, age, roles
FROM users LIMIT 100").
    All(&users)
```

Binding parameters

To prevent SQL injection attacks, you should use named parameters for any expression value that comes from user input. This could be done using the named `{:paramName}` placeholders in your SQL statement and then define the parameter values for the query with `Bind(params)`. For example:

```
type Post struct {
    Name      string      `db:"name"
    json:"name"`
    Created   types.DateTime `db:"created"
    json:"created"`
}

posts := []Post{}

err := app.DB().
    NewQuery("SELECT name, created FROM posts
WHERE created >= {:from} and created <= {:to}").
    Bind(dbx.Params{
        "from": "2023-06-25 00:00:00.000Z",
        "to":   "2023-06-28 23:59:59.999Z",
    }).
    All(&posts)
```

Query builder

Instead of writing plain SQLs, you can also compose SQL statements programmatically using the db query builder. Every SQL keyword has a corresponding query building method. For example, `SELECT` corresponds to `Select()`, `FROM` corresponds to `From()`, `WHERE` corresponds to `Where()`, and so on.

```
users := []struct {
    Id      string `db:"id" json:"id"`
    Email   string `db:"email" json:"email"`
}{}

app.DB().
    Select("id", "email").
    From("users").
    AndWhere(dbx.Like("email", "example.com")).
    Limit(100).
    OrderBy("created ASC").
    All(&users)
```


Select(), AndSelect(), Distinct()

The `Select(...cols)` method initializes a `SELECT` query builder. It accepts a list of the column names to be selected. To add additional columns to an existing select query, you can call `AndSelect()`.

To select distinct rows, you can call `Distinct(true)`.

```
app.DB().
    Select("id", "avatar as image").
    AndSelect("(firstName || ' ' || lastName) as
fullName").
    Distinct(true)
    ...
```

From()

The `From(...tables)` method specifies which tables to select from (plain table names are automatically quoted).

```
app.DB().
    Select("table1.id", "table2.name").
    From("table1", "table2")
    ...
```

Join()

The `Join(type, table, on)` method specifies a `JOIN` clause. It takes 3 parameters:

- `type` - join type string like `INNER JOIN`, `LEFT JOIN`, etc.
- `table` - the name of the table to be joined
- `on` - optional `dbx.Expression` as an `ON` clause

For convenience, you can also use the shortcuts

`InnerJoin(table, on)`, `LeftJoin(table, on)`,
`RightJoin(table, on)` to specify `INNER JOIN`, `LEFT JOIN`
and `RIGHT JOIN`, respectively.

```
app.DB().
    Select("users.*").
    From("users").
    InnerJoin("profiles",
dbx.NewExp("profiles.user_id = users.id")).
    Join("FULL OUTER JOIN", "department",
dbx.NewExp("department.id = {:id}"), dbx.Params{
```

```
"id": "someId" })))  
...
```

Where(), AndWhere(), OrWhere()

The `Where(exp)` method specifies the `WHERE` condition of the query.

You can also use `AndWhere(exp)` or `OrWhere(exp)` to append additional one or more conditions to an existing `WHERE` clause. Each where condition accepts a single `dbx.Expression` (see below for full list).

```
/*  
SELECT users.*  
FROM users  
WHERE id = "someId" AND  
      status = "public" AND  
      name like "%john%" OR  
      (  
        role = "manager" AND  
        fullTime IS TRUE AND  
        experience > 10  
      )  
*/  
app.DB().  
  Select("users.*").  
  From("users").  
  Where(dbx.NewExp("id = {:id}", dbx.Params{  
    "id": "someId" }))).  
  AndWhere(dbx.HashExp{"status": "public"}).  
  AndWhere(dbx.Like("name", "john")).  
  OrWhere(dbx.And(  
    dbx.HashExp{  
      "role": "manager",  
      "fullTime": true,  
    },  
    dbx.NewExp("experience > {:exp}",  
      dbx.Params{ "exp": 10 })  
  ))  
  ...
```

The following `dbx.Expression` methods are available:

- `dbx.NewExp(raw, optParams)`
Generates an expression with the specified raw query fragment. Use the `optParams` to bind `dbx.Params` to the expression.

```
dbx.NewExp("status = 'public'")
dbx.NewExp("total > {:min} AND total <
{:max}", dbx.Params{ "min": 10, "max": 30 })
```

- `dbx.HashExp{k:v}`

Generates a hash expression from a map whose keys are DB column names which need to be filtered according to the corresponding values.

```
// slug = "example" AND active IS TRUE AND
tags in ("tag1", "tag2", "tag3") AND parent IS
NULL
dbx.HashExp{
  "slug": "example",
  "active": true,
  "tags": []any{"tag1", "tag2", "tag3"},
  "parent": nil,
}
```

- `dbx.Not(exp)`

Negates a single expression by wrapping it with `NOT()`.

```
// NOT(status = 1)
dbx.Not(dbx.NewExp("status = 1"))
```

- `dbx.And(...exps)`

Creates a new expression by concatenating the specified ones with `AND`.

```
// (status = 1 AND username like "%john%")
dbx.And(
  dbx.NewExp("status = 1"),
  dbx.Like("username", "john"),
)
```

- `dbx.Or(...exps)`

Creates a new expression by concatenating the specified ones with `OR`.

```
// (status = 1 OR username like "%john%")
dbx.Or(
  dbx.NewExp("status = 1"),
  dbx.Like("username", "john")
)
```

- `dbx.In(col, ...values)`

Generates an `IN` expression for the specified column and the list of allowed values.

```
// status IN ("public", "reviewed")
dbx.In("status", "public", "reviewed")
```

- `dbx.NotIn(col, ...values)`

Generates an `NOT IN` expression for the specified column and the list of allowed values.

```
// status NOT IN ("public", "reviewed")
dbx.NotIn("status", "public", "reviewed")
```

- `dbx.Like(col, ...values)`

Generates a `LIKE` expression for the specified column and the possible strings that the column should be like. If multiple values are present, the column should be like **all** of them.

By default, each value will be surrounded by `"%"` to enable partial matching. Special characters like `"%"`, `"\"`, `"_"` will also be properly escaped. You may call

`Escape(...pairs)` and/or `Match(left, right)` to change the default behavior.

```
// name LIKE "%test1%" AND name LIKE "%test2%"
dbx.Like("name", "test1", "test2")

// name LIKE "test1%"
dbx.Like("name", "test1").Match(false, true)
```

- `dbx.NotLike(col, ...values)`

Generates a `NOT LIKE` expression in similar manner as `Like()`.

```
// name NOT LIKE "%test1%" AND name NOT LIKE "%test2%"
dbx.NotLike("name", "test1", "test2")

// name NOT LIKE "test1%"
dbx.NotLike("name", "test1").Match(false, true)
```

- `dbx.OrLike(col, ...values)`

This is similar to `Like()` except that the column must be

one of the provided values, aka. multiple values are concatenated with `OR` instead of `AND`.

```
// name LIKE "%test1%" OR name LIKE "%test2%"
dbx.OrLike("name", "test1", "test2")

// name LIKE "test1%" OR name LIKE "test2%"
dbx.OrLike("name", "test1",
"test2").Match(false, true)
```

- `dbx.OrNotLike(col, ...values)`

This is similar to `NotLike()` except that the column must not be one of the provided values, aka. multiple values are concatenated with `OR` instead of `AND`.

```
// name NOT LIKE "%test1%" OR name NOT LIKE
"%test2%"
dbx.OrNotLike("name", "test1", "test2")

// name NOT LIKE "test1%" OR name NOT LIKE
"test2%"
dbx.OrNotLike("name", "test1",
"test2").Match(false, true)
```

- `dbx.Exists(exp)`

Prefix with `EXISTS` the specified expression (usually a subquery).

```
// EXISTS (SELECT 1 FROM users WHERE status =
'active')
dbx.Exists(dbx.NewExp("SELECT 1 FROM users
WHERE status = 'active'))
```

- `dbx.NotExists(exp)`

Prefix with `NOT EXISTS` the specified expression (usually a subquery).

```
// NOT EXISTS (SELECT 1 FROM users WHERE
status = 'active')
dbx.NotExists(dbx.NewExp("SELECT 1 FROM users
WHERE status = 'active'))
```

- `dbx.Between(col, from, to)`

Generates a `BETWEEN` expression with the specified range.

```
// age BETWEEN 3 and 99
dbx.Between("age", 3, 99)
```

- `dbx.NotBetween(col, from, to)`

Generates a `NOT BETWEEN` expression with the specified range.

```
// age NOT BETWEEN 3 and 99
dbx.NotBetween("age", 3, 99)
```

OrderBy(), AndOrderBy()

The `OrderBy(...cols)` specifies the `ORDER BY` clause of the query.

A column name can contain `"ASC"` or `"DESC"` to indicate its ordering direction.

You can also use `AndOrderBy(...cols)` to append additional columns to an existing `ORDER BY` clause.

```
app.DB().
  Select("users.*").
  From("users").
  OrderBy("created ASC", "updated DESC").
  AndOrderBy("title ASC")
  ...
```

GroupBy(), AndGroupBy()

The `GroupBy(...cols)` specifies the `GROUP BY` clause of the query.

You can also use `AndGroupBy(...cols)` to append additional columns to an existing `GROUP BY` clause.

```
app.DB().
  Select("users.*").
  From("users").
  GroupBy("department", "level")
  ...
```

Having(), AndHaving(), OrHaving()

The `Having(exp)` specifies the `HAVING` clause of the query.

Similarly to `Where(exp)`, it accepts a single `dbx.Expression` (see all available expressions listed above).

You can also use `AndHaving(exp)` or `OrHaving(exp)` to append additional one or more conditions to an existing `HAVING` clause.

```
app.DB().
    Select("users.*").
    From("users").
    GroupBy("department", "level").
    Having(dbx.NewExp("sum(level) > {:sum}",
dbx.Params{ sum: 10 })))
...
```

Limit()

The `Limit(number)` method specifies the `LIMIT` clause of the query.

```
app.DB().
    Select("users.*").
    From("users").
    Limit(30)
...
```

Offset()

The `Offset(number)` method specifies the `OFFSET` clause of the query. Usually used together with `Limit(number)`.

```
app.DB().
    Select("users.*").
    From("users").
    Offset(5).
    Limit(30)
...
```

Transaction

To execute multiple queries in a transaction you can use

`app.RunInTransaction(fn)` .

The DB operations are persisted only if the transaction returns `nil`.

It is safe to nest `RunInTransaction` calls as long as you use the callback's `txApp` argument.

Inside the transaction function always use its `txApp` argument and not the original `app` instance because we allow only a single



writer/transaction at a time and it could result in a deadlock.

To avoid performance issues, try to minimize slow/long running tasks such as sending emails, connecting to external services, etc. as part of the transaction.

```
err := app.RunInTransaction(func(txApp core.App)
error {
    // update a record
    record, err :=
txApp.FindRecordById("articles", "RECORD_ID")
    if err != nil {
        return err
    }
    record.Set("status", "active")
    if err := txApp.Save(record); err != nil {
        return err
    }

    // run a custom raw query (doesn't fire event
hooks)
    rawQuery := "DELETE FROM articles WHERE status
= 'pending'"
    if _, err :=
txApp.DB().NewQuery(rawQuery).Execute(); err !=
nil {
        return err
    }

    return nil
})
```

[← Prev: Routing](#)

[Next: Record operations →](#)

[FAQ](#)

[JavaScript SDK](#)

[PocketBase](#)

[Discussions](#)

[Dart SDK](#)

[Documentation](#)



[Introduction](#)[Going to production](#)[Web APIs reference](#)[Extend with Go](#)[Extend with JavaScript](#)[Go Overview](#)[Go Event hooks](#)[Go Routing](#)[Go Database](#)[Go Record operations](#)[Go Collection operations](#)[Go Migrations](#)[Go Jobs scheduling](#)[Go Sending emails](#)[Go Rendering templates](#)[Go Console commands](#)[Go Realtime messaging](#)[Go Filesystem](#)[Go Logging](#)[Go Testing](#)[Go Miscellaneous](#)

Extend with Go - Record operations

The most common task when using PocketBase as framework probably would be querying and working with your collection records.

You could find detailed documentation about all the supported Record model methods in `core.Record` but below are some examples with the most common ones.

- [Set field value](#)
- [Get field value](#)
- [Auth accessors](#)
- [Copies](#)
- [Hide/Unhide fields](#)
- [Fetch records](#)
 - [Fetch single record](#)
 - [Fetch multiple records](#)
 - [Fetch auth records](#)
 - [Custom record query](#)
- [Create new record](#)
 - [Create new record programmatically](#)
 - [Intercept create request](#)
- [Update existing record](#)
 - [Update existing record programmatically](#)
 - [Intercept update request](#)
- [Delete record](#)
- [Transaction](#)
- [Programmatically expanding relations](#)
- [Check if record can be accessed](#)
- [Generating and validating tokens](#)

Set field value

```
// sets the value of a single record field
// (field type specific modifiers are also
supported)
record.Set("title", "example")
record.Set("users+", "6jyr1y02438et52") // append
to existing value

// populates a record from a data map
// (calls Set for each entry of the map)
record.Load(data)
```

Get field value

```
// retrieve a single record field value
// (field specific modifiers are also supported)
record.Get("someField")           // -> any
(without cast)
record.GetBool("someField")       // -> cast to
bool
record.GetString("someField")     // -> cast to
string
record.GetInt("someField")        // -> cast to
int
record.GetFloat("someField")      // -> cast to
float64
record.GetDateTime("someField")   // -> cast to
types.DateTime
record.GetStringSlice("someField") // -> cast to
[]string

// retrieve the new uploaded files
// (e.g. for inspecting and modifying the file(s)
before save)
record.GetUnsavedFiles("someFileField")

// unmarshal a single "json" field value into the
provided result
record.UnmarshalJSONField("someJSONField",
&result)

// retrieve a single or multiple expanded data
record.ExpandedOne("author")      // ->
nil|*core.Record
record.ExpandedAll("categories") // ->
[]*core.Record

// export all the public safe record fields as
map[string]any
```

```
// (note: "json" type field values are exported as
types.JSONRaw bytes slice)
record.PublicExport()
```

Auth accessors

```
record.IsSuperuser() // alias for
record.Collection().Name == "_superusers"
```

```
record.Email() // alias for
record.Get("email")
record.SetEmail(email) // alias for
record.Set("email", email)
```

```
record.Verified() // alias for
record.Get("verified")
record.SetVerified(false) // alias for
record.Set("verified", false)
```

```
record.TokenKey() // alias for
record.Get("tokenKey")
record.SetTokenKey(key) // alias for
record.Set("tokenKey", key)
record.RefreshTokenKey() // alias for
record.Set("tokenKey:autogenerate", "")
```

```
record.ValidatePassword(pass)
record.SetPassword(pass) // alias for
record.Set("password", pass)
record.SetRandomPassword() // sets
cryptographically random 30 characters string as
password
```

Copies

```
// returns a shallow copy of the current record
model populated
// with its ORIGINAL db data state and everything
else reset to the defaults
// (usually used for comparing old and new field
values)
record.Original()
```

```
// returns a shallow copy of the current record
model populated
// with its LATEST data state and everything else
```

```

reset to the defaults
// (aka. no expand, no custom fields and with
default visibility flags)
record.Fresh()

// returns a shallow copy of the current record
model populated
// with its ALL collection and custom fields data,
expand and visibility flags
record.Clone()

```

Hide/Unhide fields

Collection fields can be marked as "Hidden" from the Dashboard to prevent regular user access to the field values.

Record models provide an option to further control the fields serialization visibility in addition to the "Hidden" fields option using the `record.Hide(fieldNames...)` and `record.Unhide(fieldNames...)` methods.

Often the `Hide/Unhide` methods are used in combination with the `OnRecordEnrich` hook invoked on every record enriching (list, view, create, update, realtime change, etc.). For example:

```

app.OnRecordEnrich("articles").BindFunc(func(e
*core.RecordEnrichEvent) error {
    // dynamically show/hide a record field
    depending on whether the current
    // authenticated user has a certain "role" (or
    any other field constraint)
    if e.RequestInfo.Auth == nil ||
        (!e.RequestInfo.Auth.IsSuperuser() &&
e.RequestInfo.Auth.GetString("role") != "staff") {
        e.Record.Hide("someStaffOnlyField")
    }

    return e.Next()
})

```



For custom fields, not part of the record collection schema, it is required to call explicitly `record.WithCustomData(true)` to allow them in the public serialization.

Fetch records

Fetch single record

All single record retrieval methods return `nil` and `sql.ErrNoRows` error if no record is found.

```
// retrieve a single "articles" record by its id
record, err := app.FindRecordById("articles",
    "RECORD_ID")

// retrieve a single "articles" record by a single
key-value pair
record, err :=
app.FindFirstRecordByData("articles", "slug",
    "test")

// retrieve a single "articles" record by a string
filter expression
// (NB! use "{:placeholder}" to safely bind
untrusted user input parameters)
record, err := app.FindFirstRecordByFilter(
    "articles",
    "status = 'public' && category = {:category}",
    dbx.Params{ "category": "news" },
)
```

Fetch multiple records

All multiple records retrieval methods return empty slice and `nil` error if no records are found.

```
// retrieve multiple "articles" records by their
ids
records, err := app.FindRecordsByIds("articles",
    []string{"RECORD_ID1", "RECORD_ID2"})

// retrieve the total number of "articles" records
in a collection with optional dbx expressions
totalPending, err := app.CountRecords("articles",
    dbx.HashExp{"status": "pending"})

// retrieve multiple "articles" records with
optional dbx expressions
records, err := app.FindAllRecords("articles",
    dbx.NewExp("LOWER(username) = {:username}",
    dbx.Params{"username": "John.Doe"}),
    dbx.HashExp{"status": "pending"},
)
```

```
// retrieve multiple paginated "articles" records
// by a string filter expression
// (NB! use ":{placeholder}" to safely bind
// untrusted user input parameters)
records, err := app.FindRecordsByFilter(
    "articles",
// collection
    "status = 'public' && category = {category}",
// filter
    "-published",
// sort
    10,
// limit
    0,
// offset
    dbx.Params{ "category": "news" },
// optional filter params
)
```

Fetch auth records

```
// retrieve a single auth record by its email
user, err := app.FindAuthRecordByEmail("users",
    "test@example.com")

// retrieve a single auth record by JWT
// (you could also specify an optional list of
// accepted token types)
user, err :=
    app.FindAuthRecordByToken("YOUR_TOKEN",
    core.TokenTypeAuth)
```

Custom record query

In addition to the above query helpers, you can also create custom Record queries using `RecordQuery(collection)` method. It returns a SELECT DB builder that can be used with the same methods described in the [Database guide](#).

```
import (
    "github.com/pocketbase/dbx"
    "github.com/pocketbase/pocketbase/core"
)

...

func FindActiveArticles(app core.App)
([]*core.Record, error) {
```

```

records := []*core.Record{}

err := app.RecordQuery("articles").
    AndWhere(dbx.HashExp{"status": "active"}).
    OrderBy("published DESC").
    Limit(10).
    All(&records)

if err != nil {
    return nil, err
}

return records, nil
}

```

Create new record

Create new record programmatically

```

import (
    "github.com/pocketbase/pocketbase/core"

    "github.com/pocketbase/pocketbase/tools/filesystem"
)

...

collection, err :=
app.FindCollectionByNameOrId("articles")
if err != nil {
    return err
}

record := core.NewRecord(collection)

record.Set("title", "Lorem ipsum")
record.Set("active", true)

// field type specific modifiers can also be used
record.Set("slug:autogenerate", "post-")

// new files must be one or a slice of
// *filesystem.File values
//
// note1: see all factories in
// https://pkg.go.dev/github.com/pocketbase/pocketbase/tools/filesystem#File
// note2: for reading files from a request event

```



```

you can also use e.FindUploadedFiles("fileKey")
f1, _ :=
filesystem.NewFileFromPath("/local/path/to/file1.t
xt")
f2, _ := filesystem.NewFileFromBytes([]byte{"test
content"}, "file2.txt")
f3, _ :=
filesystem.NewFileFromURL(context.Background(),
"https://example.com/file3.pdf")
record.Set("documents", []*filesystem.File{f1, f2,
f3})

// validate and persist
// (use SaveNoValidate to skip fields validation)
err = app.Save(record);
if err != nil {
    return err
}

```

Intercept create request

```

import (
    "github.com/pocketbase/pocketbase/core"
)

...

app.OnRecordCreateRequest("articles").BindFunc(func
c(e *core.RecordRequestEvent) error {
    // ignore for superusers
    if e.HasSuperuserAuth() {
        return e.Next()
    }

    // overwrite the submitted "status" field
    value
    e.Record.Set("status", "pending")

    // or you can also prevent the create event by
    returning an error
    status := e.Record.GetString("status")
    if (status != "pending" &&
        // guest or not an editor
        (e.Auth == nil || e.Auth.GetString("role")
!= "editor")) {
        return e.BadRequestError("Only editors can
set a status different from pending", nil)
    }
}

```

```
    return e.Next()
})
```

Update existing record

Update existing record programmatically

```
record, err := app.FindRecordById("articles",
"RECORD_ID")
if err != nil {
    return err
}

record.Set("title", "Lorem ipsum")

// delete existing record files by specifying
their file names
record.Set("documents-",
[]string{"file1_abc123.txt", "file3_abc123.txt"})

// append one or more new files to the already
uploaded list
//
// note1: see all factories in
https://pkg.go.dev/github.com/pocketbase/pocketbas
e/tools/filesystem#File
// note2: for reading files from a request event
you can also use e.FindUploadedFiles("fileKey")
f1, _ :=
filesystem.NewFileFromPath("/local/path/to/file1.t
xt")
f2, _ := filesystem.NewFileFromBytes([]byte{"test
content"}, "file2.txt")
f3, _ :=
filesystem.NewFileFromURL(context.Background(),
"https://example.com/file3.pdf")
record.Set("documents+", []*filesystem.File{f1,
f2, f3})

// validate and persist
// (use SaveNoValidate to skip fields validation)
err = app.Save(record);
if err != nil {
    return err
}
```

Intercept update request

```

import (
    "github.com/pocketbase/pocketbase/core"
)

...

app.OnRecordUpdateRequest("articles").Add(func(e
*core.RecordRequestEvent) error {
    // ignore for superusers
    if e.HasSuperuserAuth() {
        return e.Next()
    }

    // overwrite the submitted "status" field
    value
    e.Record.Set("status", "pending")

    // or you can also prevent the update event by
    returning an error
    status := e.Record.GetString("status")
    if (status != "pending" &&
        // guest or not an editor
        (e.Auth == nil || e.Auth.GetString("role")
        != "editor")) {
        return e.BadRequestError("Only editors can
        set a status different from pending", nil)
    }

    return e.Next()
})

```

Delete record

```

record, err := app.FindRecordById("articles",
"RECORD_ID")
if err != nil {
    return err
}

err = app.Delete(record)
if err != nil {
    return err
}

```

Transaction

To execute multiple queries in a transaction you can use

`app.RunInTransaction(fn)` .

The DB operations are persisted only if the transaction returns `nil`.

It is safe to nest `RunInTransaction` calls as long as you use the callback's `txApp` argument.



Inside the transaction function always use its `txApp` argument and not the original `app` instance because we allow only a single writer/transaction at a time and it could result in a deadlock.

To avoid performance issues, try to minimize slow/long running tasks such as sending emails, connecting to external services, etc. as part of the transaction.

```
import (
    "github.com/pocketbase/pocketbase/core"
)

...

titles := []string{"title1", "title2", "title3"}

collection, err :=
app.FindCollectionByNameOrId("articles")
if err != nil {
    return err
}

// create new record for each title
app.RunInTransaction(func(txApp core.App) error {
    for _, title := range titles {
        record := core.NewRecord(collection)
        record.Set("title", title)

        if err := txApp.Save(record); err != nil {
            return err
        }
    }

    return nil
})
```

Programmatically expanding relations

To expand record relations programmatically you can use

`app.ExpandRecord(record, expands, optFetchFunc)` for single or

`app.ExpandRecords(records, expands, optFetchFunc)` for multiple records.

Once loaded, you can access the expanded relations via

`record.ExpandedOne(relName)` or

`record.ExpandedAll(relName)`.

For example:

```
record, err :=
app.FindFirstRecordByData("articles", "slug",
"lorem-ipsu")
if err != nil {
    return err
}

// expand the "author" and "categories" relations
errs := app.ExpandRecord(record,
[]string{"author", "categories"}, nil)
if len(errs) > 0 {
    return fmt.Errorf("failed to expand: %v",
errs)
}

// print the expanded records
log.Println(record.ExpandedOne("author"))
log.Println(record.ExpandedAll("categories"))
```

Check if record can be accessed

To check whether a custom client request or user can access a single record, you can use the

`app.CanAccessRecord(record, requestInfo, rule)` method.

Below is an example of creating a custom route to retrieve a single article and checking if the request satisfy the View API rule of the record collection:

```
package main

import (
```

```

    "log"
    "net/http"

    "github.com/pocketbase/pocketbase"
    "github.com/pocketbase/pocketbase/core"
)

func main() {
    app := pocketbase.New()

    app.OnServe().BindFunc(func(se
*core.ServeEvent) error {
        se.Router.GET("/articles/{slug}", func(e
*core.RequestEvent) error {
            slug := e.Request.PathValue("slug")

            record, err :=
e.App.FindFirstRecordByData("articles", "slug",
slug)

            if err != nil {
                return e.NotFoundError("Missing or
invalid slug", err)
            }

            info, err := e.RequestInfo()
            if err != nil {
                return e.BadRequestError("Failed
to retrieve request info", err)
            }

            canAccess, err :=
e.App.CanAccessRecord(record, info,
record.Collection().ViewRule)
            if !canAccess {
                return e.ForbiddenError("", err)
            }

            return e.JSON(http.StatusOK, record)
        })

        return se.Next()
    })

    if err := app.Start(); err != nil {
        log.Fatal(err)
    }
}

```

Generating and validating tokens

PocketBase Web APIs are fully stateless (aka. there are no sessions in the traditional sense) and an auth record is considered authenticated if the submitted request contains a valid `Authorization: TOKEN` header (see also [Builtin auth middlewares](#) and [Retrieving the current auth state from a route](#)).

If you want to issue and verify manually a record JWT (auth, verification, password reset, etc.), you could do that using the record token type specific methods:

```
token, err := record.NewAuthToken()

token, err := record.NewVerificationToken()

token, err := record.NewPasswordResetToken()

token, err := record.NewEmailChangeToken(newEmail)

token, err := record.NewFileToken() // for
protected files

token, err :=
record.NewStaticAuthToken(optCustomDuration) //
nonrenewable auth token
```

Each token type has its own secret and the token duration is managed via its type related collection auth option (*the only exception is `NewStaticAuthToken`*).

To validate a record token you can use the `app.FindAuthRecordByToken` method. The token related auth record is returned only if the token is not expired and its signature is valid.

Here is an example how to validate an auth token:

```
record, err :=
app.FindAuthRecordByToken("YOUR_TOKEN",
core.TokenTypeAuth)
```

Next: Collection operations →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



© 2023-2026 PocketBase The Gopher artwork is from [marcusolsson/gophers](https://github.com/marcusolsson/gophers)

Crafted by Gani

[Introduction](#)[Going to production](#)[Web APIs reference](#)Extend with
GoExtend with
JavaScript[Go Overview](#)[Go Event hooks](#)[Go Routing](#)[Go Database](#)[Go Record operations](#)[Go Collection operations](#)[Go Migrations](#)[Go Jobs scheduling](#)[Go Sending emails](#)[Go Rendering templates](#)[Go Console commands](#)[Go Realtime messaging](#)[Go Filesystem](#)[Go Logging](#)[Go Testing](#)[Go Miscellaneous](#)

Extend with Go - Collection operations

Collections are usually managed via the Dashboard interface, but there are some situations where you may want to create or edit a collection programmatically (usually as part of a [DB migration](#)). You can find all available Collection related operations and methods in `core.App` and `core.Collection`, but below are listed some of the most common ones:

- [Fetch collections](#)
 - [Fetch single collection](#)
 - [Fetch multiple collections](#)
 - [Custom collection query](#)
- [Collection properties](#)
- [Field definitions](#)
- [Create new collection](#)
- [Update existing collection](#)
- [Delete collection](#)

Fetch collections

Fetch single collection

All single collection retrieval methods return `nil` and `sql.ErrNoRows` error if no collection is found.

```
collection, err :=  
app.FindCollectionByNameOrId("example")
```

Fetch multiple collections

All multiple collections retrieval methods return empty slice and `nil` error if no collections are found.

```
allCollections, err := app.FindAllCollections()  
  
authAndViewCollections, err :=
```

```
app.FindAllCollections(core.CollectionTypeAuth,
core.CollectionTypeView)
```

Custom collection query

In addition to the above query helpers, you can also create custom Collection queries using `CollectionQuery()` method. It returns a SELECT DB builder that can be used with the same methods described in the [Database guide](#).

```
import (
    "github.com/pocketbase/dbx"
    "github.com/pocketbase/pocketbase/core"
)

...

func FindSystemCollections(app core.App)
([]*core.Collection, error) {
    collections := []*core.Collection{}

    err := app.CollectionQuery().
        AndWhere(dbx.HashExp{"system": true}).
        OrderBy("created DESC").
        All(&collections)

    if err != nil {
        return nil, err
    }

    return collections, nil
}
```

Collection properties

```
Id      string
Name    string
Type    string // "base", "view", "auth"
System  bool // !prevent collection rename,
deletion and rules change of internal collections
like _superusers
Fields  core.FieldsList
Indexes types.JSONArray[string]
Created types.DateTime
Updated types.DateTime

// CRUD rules
```

```

ListRule      *string
ViewRule      *string
CreateRule    *string
UpdateRule    *string
DeleteRule    *string

// "view" type specific options
// (see
https://github.com/pocketbase/pocketbase/blob/master/core/collection\_model\_view\_options.go)
ViewQuery     string

// "auth" type specific options
// (see
https://github.com/pocketbase/pocketbase/blob/master/core/collection\_model\_auth\_options.go)
AuthRule      *string
ManageRule    *string
AuthAlert     core.AuthAlertConfig
OAuth2        core.OAuth2Config
PasswordAuth  core.PasswordAuthConfig
MFA           core.MFAConfig
OTP           core.OTPConfig
AuthToken     core.TokenConfig
PasswordResetToken core.TokenConfig
EmailChangeToken core.TokenConfig
VerificationToken core.TokenConfig
FileToken     core.TokenConfig
VerificationTemplate core.EmailTemplate
ResetPasswordTemplate core.EmailTemplate
ConfirmEmailChangeTemplate core.EmailTemplate

```

Field definitions

- `core.BoolField` ↗
- `core.NumberField` ↗
- `core.TextField` ↗
- `core.EmailField` ↗
- `core.URLField` ↗
- `core.EditorField` ↗
- `core.DateField` ↗
- `core.AutodateField` ↗
- `core.SelectField` ↗
- `core.FileField` ↗

- `core.RelationField` ↗
- `core.JSONField` ↗
- `core.GeoPointField` ↗

Create new collection

```
import (
    "github.com/pocketbase/pocketbase/core"
    "github.com/pocketbase/pocketbase/tools/types"
)

...

// core.NewAuthCollection("example")
// core.NewViewCollection("example")
collection := core.NewBaseCollection("example")

// set rules
collection.ViewRule =
types.Pointer("@request.auth.id != ''")
collection.CreateRule =
types.Pointer("@request.auth.id != '' &&
@request.body.user = @request.auth.id")
collection.UpdateRule = types.Pointer(`
    @request.auth.id != '' &&
    user = @request.auth.id &&
    (@request.body.user:isset = false ||
@request.body.user = @request.auth.id)
`)

// add text field
collection.Fields.Add(&core.TextField{
    Name:      "title",
    Required:  true,
    Max:       100,
})

// add relation field
usersCollection, err :=
app.FindCollectionByNameOrId("users")
if err != nil {
    return err
}
collection.Fields.Add(&core.RelationField{
    Name:      "user",
    Required:  true,
    Max:       100,
    CascadeDelete: true,
    CollectionId: usersCollection.Id,
```

```

}))

// add autodate/timestamp fields (created/updated)
collection.Fields.Add(&core.AutodateField{
    Name:      "created",
    OnCreate:  true,
})
collection.Fields.Add(&core.AutodateField{
    Name:      "updated",
    OnCreate:  true,
    OnUpdate:  true,
})

// or: collection.Indexes = []string{"CREATE
// UNIQUE INDEX idx_example_user ON example (user)"}
collection.AddIndex("idx_example_user", true,
    "user", "")

// validate and persist
// (use SaveNoValidate to skip fields validation)
err = app.Save(collection)
if err != nil {
    return err
}

```

Update existing collection

```

import (
    "github.com/pocketbase/pocketbase/core"
    "github.com/pocketbase/pocketbase/tools/types"
)

...

collection, err :=
app.FindCollectionByNameOrId("example")
if err != nil {
    return err
}

// change rule
collection.DeleteRule =
types.Pointer("@request.auth.id != ''")

// add new editor field
collection.Fields.Add(&core.EditorField{
    Name:      "description",
    Required:  true,
})

```

```
// change existing field
// (returns a pointer and direct modifications are
// allowed without the need of reinsert)
titleField :=
collection.Fields.GetByName("title").
(*core.TextField)
titleField.Min = 10

// or: collection.Indexes =
append(collection.Indexes, "CREATE INDEX
idx_example_title ON example (title)")
collection.AddIndex("idx_example_title", false,
"title", "")

// validate and persist
// (use SaveNoValidate to skip fields validation)
err = app.Save(collection)
if err != nil {
    return err
}
```

Delete collection

```
collection, err :=
app.FindCollectionByNameOrId("example")
if err != nil {
    return err
}

err = app.Delete(collection)
if err != nil {
    return err
}
```

[← Prev: Record operations](#)

[Next: Migrations →](#)

Discussions

Dart SDK

Documentation



[Introduction](#)[Going to production](#)[Web APIs reference](#)[Extend with Go](#)[Extend with JavaScript](#)[Go Overview](#)[Go Event hooks](#)[Go Routing](#)[Go Database](#)[Go Record operations](#)[Go Collection operations](#)[Go Migrations](#)[Go Jobs scheduling](#)[Go Sending emails](#)[Go Rendering templates](#)[Go Console commands](#)[Go Realtime messaging](#)[Go Filesystem](#)[Go Logging](#)[Go Testing](#)[Go Miscellaneous](#)

Extend with Go - Migrations

PocketBase comes with a builtin DB and data migration utility, allowing you to version your DB structure, create collections programmatically, initialize default settings, etc.

Because the migrations are regular Go functions, besides applying schema changes, they could be used also to adjust existing data to fit the new schema or any other app specific logic that you want to run only once.

And as a bonus, being `.go` files also ensure that the migrations will be embedded seamlessly in your final executable.

- [Quick setup](#)
 - [0. Register the migrate command](#)
 - [1. Create new migration](#)
 - [2. Load migrations](#)
 - [3. Run migrations](#)
- [Collections snapshot](#)
- [Migrations history](#)
- [Examples](#)
 - [Executing raw SQL statements](#)
 - [Initialize default application settings](#)
 - [Creating initial superuser](#)
 - [Creating collection programmatically](#)

Quick setup

0. Register the migrate command

You can find all available config options in the `migratecmd` subpackage.

```
// main.go
package main

import (
    "log"
```



```

    "github.com/pocketbase/pocketbase"

    "github.com/pocketbase/pocketbase/plugins/migratecmd"

    "github.com/pocketbase/pocketbase/tools/osutils"

    // enable once you have at least one migration
    // _ "yourpackage/migrations"
)

func main() {
    app := pocketbase.New()

    migratecmd.MustRegister(app, app.RootCmd,
        migratecmd.Config{
            // enable auto creation of migration files
            // when making collection changes in the Dashboard
            // (the IsProbablyGoRun check is to enable
            // it only during development)
            Automigrate: osutils.IsProbablyGoRun(),
        })

    if err := app.Start(); err != nil {
        log.Fatal(err)
    }
}

```

1. Create new migration

To create a new blank migration you can run `migrate create`.

```

// Since the "create" command makes sense only
// during development,
// it is expected the user to be in the app
// working directory
// and to be using "go run"

```

```

[root@dev app]$ go run . migrate create
"your_new_migration"

```

```

// migrations/1655834400_your_new_migration.go
package migrations

import (
    "github.com/pocketbase/pocketbase/core"
    m
    "github.com/pocketbase/pocketbase/migrations"

```

```
)

func init() {
    m.Register(func(app core.App) error {
        // add up queries...

        return nil
    }, func(app core.App) error {
        // add down queries...

        return nil
    })
}
```

The above will create a new blank migration file inside the default command `migrations` directory.

Each migration file should have a single

`m.Register(upFunc, downFunc)` call.

In the migration file, you are expected to write your "upgrade" code in the `upFunc` callback.

The `downFunc` is optional and it should contain the "downgrade" operations to revert the changes made by the `upFunc`.

Both callbacks accept a transactional `core.App` instance.



You can explore the [Database guide](#), [Collection operations](#) and [Record operations](#) for more details how to interact with the database. You can also find [some examples](#) further below in this guide.

2. Load migrations

To make your application aware of the registered migrations, you have to import the above `migrations` package in one of your `main` package files:

```
package main

import _ "yourpackage/migrations"

// ...
```

3. Run migrations

New unapplied migrations are automatically executed when the application server starts, aka. on `serve`.

Alternatively, you can also apply new migrations manually by running `migrate up`.

To revert the last applied migration(s), you can run

`migrate down [number]`.

When manually applying or reverting migrations, the `serve` process needs to be restarted so that it can refresh its cached collections state.

Collections snapshot

The `migrate collections` command generates a full snapshot of your current collections configuration without having to type it manually. Similar to the `migrate create` command, this will generate a new migration file in the `migrations` directory.

```
// Since the "collections" command makes sense
// only during development,
// it is expected the user to be in the app
// working directory
// and to be using "go run"
```

```
[root@dev app]$ go run . migrate collections
```

By default the collections snapshot is imported in *extend* mode, meaning that collections and fields that don't exist in the snapshot are preserved. If you want the snapshot to *delete* missing collections and fields, you can edit the generated file and change the last argument of

`ImportCollectionsByMarshaledJSON` method to `true`.

Migrations history

All applied migration filenames are stored in the internal `_migrations` table.

During local development often you might end up making various collection changes to test different approaches.

When `Automigrate` is enabled this could lead in a migration history with unnecessary intermediate steps that may not be wanted in the final migration history.

To avoid the clutter and to prevent applying the intermediate steps in production, you can remove (or squash) the

unnecessary migration files manually and then update the local migrations history by running:

```
[root@dev app]$ go run . migrate history-sync
```

The above command will remove any entry from the `_migrations` table that doesn't have a related migration file associated with it.

Examples

Executing raw SQL statements

```
// migrations/1687801090_set_pending_status.go
package migrations

import (
    "github.com/pocketbase/pocketbase/core"
    m
    "github.com/pocketbase/pocketbase/migrations"
)

// set a default "pending" status to all empty
// status articles
func init() {
    m.Register(func(app core.App) error {
        _, err := app.DB().NewQuery("UPDATE
articles SET status = 'pending' WHERE status =
''").Execute()
        return err
    }, nil)
}
```

Initialize default application settings

```
// migrations/1687801090_initial_settings.go
package migrations

import (
    "github.com/pocketbase/pocketbase/core"
    m
    "github.com/pocketbase/pocketbase/migrations"
)

func init() {
    m.Register(func(app core.App) error {
```

```

        settings := app.Settings()

        // for all available settings fields you
        could check
        //
        https://github.com/pocketbase/pocketbase/blob/develop/core/settings_model.go#L121-L130
        settings.Meta.AppName = "test"
        settings.Meta.AppURL =
        "https://example.com"
        settings.Logs.MaxDays = 2
        settings.Logs.LogAuthId = true
        settings.Logs.LogIP = false

        return app.Save(settings)
    }, nil)
}

```

Creating initial superuser

For all supported record methods, you can refer to [Record operations](#).

You can also create the initial super user using the

`./pocketbase superuser create EMAIL PASS` command.

```

// migrations/1687801090_initial_superuser.go
package migrations

import (
    "github.com/pocketbase/pocketbase/core"
    m
    "github.com/pocketbase/pocketbase/migrations"
)

func init() {
    m.Register(func(app core.App) error {
        superusers, err :=
        app.FindCollectionByNameOrId(core.CollectionNameSuperusers)
        if err != nil {
            return err
        }

        record := core.NewRecord(superusers)

        // note: the values can be eventually
        loaded via os.Getenv(key)
        // or from a special local config file
        record.Set("email", "test@example.com")
        record.Set("password", "1234567890")
    })
}

```

```

        return app.Save(record)
    }, func(app core.App) error { // optional
revert operation
        record, _ :=
app.FindAuthRecordByEmail(core.CollectionNameSuper
users, "test@example.com")
        if record == nil {
            return nil // probably already deleted
        }

        return app.Delete(record)
    })
}

```

Creating collection programmatically

For all supported collection methods, you can refer to [Collection operations](#).

```

//
migrations/1687801090_create_clients_collection.go
package migrations

import (
    "github.com/pocketbase/pocketbase/core"
    "github.com/pocketbase/pocketbase/tools/types"

    m
    "github.com/pocketbase/pocketbase/migrations"
)

func init() {
    m.Register(func(app core.App) error {
        // init a new auth collection with the
        default system fields and auth options
        collection :=
core.NewAuthCollection("clients")

        // restrict the list and view rules for
        record owners
        collection.ListRule = types.Pointer("id =
@request.auth.id")
        collection.ViewRule = types.Pointer("id =
@request.auth.id")

        // add extra fields in addition to the
        default ones
        collection.Fields.Add(
            &core.TextField{
                Name:      "company",

```

```

        Required: true,
        Max:      100,
    },
    &core.URLField{
        Name:      "website",
        Presentable: true,
    },
)

// disable password auth and enable OTP
only
collection.PasswordAuth.Enabled = false
collection.OTP.Enabled = true

collection.AddIndex("idx_clients_company",
false, "company", "")

return app.Save(collection)
}, func(app core.App) error { // optional
revert operation
    collection, err :=
app.FindCollectionByNameOrId("clients")
    if err != nil {
        return err
    }

    return app.Delete(collection)
})
}

```

← [Prev: Collection operations](#)

[Next: Jobs scheduling](#) →

[FAQ](#)

[JavaScript SDK](#)

[PocketBase](#)

[Discussions](#)

[Dart SDK](#)

[Documentation](#)



[Introduction](#)[Going to production](#)[Web APIs reference](#)[Extend with Go](#)[Extend with JavaScript](#)[Go Overview](#)[Go Event hooks](#)[Go Routing](#)[Go Database](#)[Go Record operations](#)[Go Collection operations](#)[Go Migrations](#)[Go Jobs scheduling](#)[Go Sending emails](#)[Go Rendering templates](#)[Go Console commands](#)[Go Realtime messaging](#)[Go Filesystem](#)[Go Logging](#)[Go Testing](#)[Go Miscellaneous](#)

Extend with Go - Jobs scheduling

If you have tasks that need to be performed periodically, you could set up crontab-like jobs with the builtin `app.Cron()` (it returns an app scoped `cron.Cron` value).

The jobs scheduler is started automatically on app `serve`, so all you have to do is register a handler with `app.Cron().Add(id, cronExpr, handler)` or `app.Cron().MustAdd(id, cronExpr, handler)` (the latter panic if the cron expression is not valid).

Each scheduled job runs in its own goroutine and must have:

- **id** - identifier for the scheduled job; could be used to replace or remove an existing job
- **cron expression** - e.g. `0 0 * * *` (supports numeric list, steps, ranges or macros)
- **handler** - the function that will be executed every time when the job runs

Here is one minimal example:

```
// main.go
package main

import (
    "log"

    "github.com/pocketbase/pocketbase"
)

func main() {
    app := pocketbase.New()

    // prints "Hello!" every 2 minutes
    app.Cron().MustAdd("hello", "*/2 * * * *",
func() {
    log.Println("Hello!")
})

    if err := app.Start(); err != nil {
        log.Fatal(err)
    }
}
```

To remove already registered cron job you can call

`app.Cron().Remove(id)` ↗

All registered app level cron jobs can be also previewed and triggered from the *Dashboard > Settings > Crons* section.



Keep in mind that the `app.Cron()` is also used for running the system scheduled jobs like the logs cleanup or auto backups (the jobs id is in the format `__pb*__`) and replacing these system jobs or calling `RemoveAll()` / `Stop()` could have unintended side-effects.

If you want more advanced control you can initialize your own cron instance independent from the application via `cron.New()`.

← Prev: Migrations

Next: Sending emails →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



[Introduction](#)[Going to production](#)[Web APIs reference](#)Extend with
GoExtend with
JavaScript[Go Overview](#)[Go Event hooks](#)[Go Routing](#)[Go Database](#)[Go Record operations](#)[Go Collection operations](#)[Go Migrations](#)[Go Jobs scheduling](#)[Go Sending emails](#)[Go Rendering templates](#)[Go Console commands](#)[Go Realtime messaging](#)[Go Filesystem](#)[Go Logging](#)[Go Testing](#)[Go Miscellaneous](#)

Extend with Go - Sending emails

PocketBase provides a simple abstraction for sending emails via the `app.NewMailClient()` factory.

Depending on your configured mail settings (*Dashboard > Settings > Mail settings*) it will use the `sendmail` command or a SMTP client.

- [Send custom email](#)
- [Overwrite system emails](#)

Send custom email

You can send your own custom email from anywhere within the app (hooks, middlewares, routes, etc.) by using `app.NewMailClient().Send(message)`. Here is an example of sending a custom email after user registration:

```
// main.go
package main

import (
    "log"
    "net/mail"

    "github.com/pocketbase/pocketbase"
    "github.com/pocketbase/pocketbase/core"
    "github.com/pocketbase/pocketbase/tools/mailer"
)

func main() {
    app := pocketbase.New()

    app.OnRecordCreateRequest("users").BindFunc(func(e
    *core.RecordRequestEvent) error {
        if err := e.Next(); err != nil {
            return err
        }

        message := &mailer.Message{
            From: mail.Address{
```

```

        Address:
e.App.Settings().Meta.SenderAddress,
        Name:
e.App.Settings().Meta.SenderName,
    },
    To:      []mail.Address{{Address:
e.Record.Email()}},
    Subject: "YOUR_SUBJECT...",
    HTML:    "YOUR_HTML_BODY...",
    // bcc, cc, attachments and custom
    headers are also supported...
    }

    return e.App.NewMailClient().Send(message)
})

if err := app.Start(); err != nil {
    log.Fatal(err)
}
}

```

Overwrite system emails

If you want to overwrite the default system emails for forgotten password, verification, etc., you can adjust the default templates available from the *Dashboard > Collections > Edit collection > Options*.

Alternatively, you can also apply individual changes by binding to one of the [mailer hooks](#). Here is an example of appending a Record field value to the subject using the

`OnMailerRecordPasswordResetSend` hook:

```

// main.go
package main

import (
    "log"

    "github.com/pocketbase/pocketbase"
    "github.com/pocketbase/pocketbase/core"
)

func main() {
    app := pocketbase.New()

    app.OnMailerRecordPasswordResetSend("users").BindFunc(
        func(e *core.MailerRecordEvent) error {

```

```
// modify the subject
e.Message.Subject += (" " +
e.Record.GetString("name"))

return e.Next()
})

if err := app.Start(); err != nil {
    log.Fatal(err)
}
}
```

← [Prev: Jobs scheduling](#)

[Next: Rendering templates](#) →

[FAQ](#)

[JavaScript SDK](#)

PocketBase

[Discussions](#)

[Dart SDK](#)

[Documentation](#)



[Introduction](#)[Going to production](#)[Web APIs reference](#)Extend with
GoExtend with
JavaScript[Go Overview](#)[Go Event hooks](#)[Go Routing](#)[Go Database](#)[Go Record operations](#)[Go Collection operations](#)[Go Migrations](#)[Go Jobs scheduling](#)[Go Sending emails](#)[Go Rendering templates](#)[Go Console commands](#)[Go Realtime messaging](#)[Go Filesystem](#)[Go Logging](#)[Go Testing](#)[Go Miscellaneous](#)

Extend with Go - Rendering templates

- [Overview](#)
- [Example HTML page with layout](#)

Overview

A common task when creating custom routes or emails is the need of generating HTML output.

There are plenty of Go template-engines available that you can use for this, but often for simple cases the Go standard library `html/template` package should work just fine.

To make it slightly easier to load template files concurrently and on the fly, PocketBase also provides a thin wrapper around the standard library in the

github.com/pocketbase/pocketbase/tools/template utility package.

```
import
    "github.com/pocketbase/pocketbase/tools/template"

data := map[string]any{"name": "John"}

html, err := template.NewRegistry().LoadFiles(
    "views/base.html",
    "views/partial1.html",
    "views/partial2.html",
).Render(data)
```

The general flow when working with composed and nested templates is that you create "base" template(s) that defines various placeholders using the `{{template "placeholderName" .}}` or `{{block "placeholderName" .}}default...{{end}}` actions.

Then in the partials, you define the content for those placeholders using the

`{{define "placeholderName"}}custom...{{end}}` action.

The dot object (`.`) in the above represents the data passed to the templates via the `Render(data)` method.

By default the templates apply contextual (HTML, JS, CSS, URI) auto escaping so the generated template content should be injection-safe. To render raw/verbatim trusted content in the templates you can use the builtin `raw` function (e.g. `{{.content|raw}}`).



For more information about the template syntax please refer to the [html/template](#) and [text/template](#) package godocs. **Another great resource is also the Hashicorp's [Learn Go Template Syntax](#) tutorial.**

Example HTML page with layout

Consider the following app directory structure:

```
myapp/  
  views/  
    layout.html  
    hello.html  
  main.go
```

We define the content for `layout.html` as:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <title>{{block "title" .}}Default app  
  title{{end}}</title>  
</head>  
<body>  
  Header...  
  
  {{block "body" .}}  
    Default app body...  
  {{end}}  
  
  Footer...  
</body>  
</html>
```

We define the content for `hello.html` as:

```

{{define "title"}}
    Page 1
{{end}}

{{define "body"}}
    <p>Hello from {{.name}}</p>
{{end}}

```

Then to output the final page, we'll register a custom `/hello/:name` route:

```

// main.go
package main

import (
    "log"
    "net/http"

    "github.com/pocketbase/pocketbase"
    "github.com/pocketbase/pocketbase/core"

    "github.com/pocketbase/pocketbase/tools/template"
)

func main() {
    app := pocketbase.New()

    app.OnServe().BindFunc(func(se
    *core.ServeEvent) error {
        // this is safe to be used by multiple
        goroutines
        // (it acts as store for the parsed
        templates)
        registry := template.NewRegistry()

        se.Router.GET("/hello/{name}", func(e
        *core.RequestEvent) error {
            name := e.Request.PathValue("name")

            html, err := registry.LoadFiles(
                "views/layout.html",
                "views/hello.html",
            ).Render(map[string]any{
                "name": name,
            })

            if err != nil {
                // or redirect to a dedicated 404
                return e.NotFoundError("", err)
            }
        })
    })
}

```

HTML page


```
    }

    return e.HTML(http.StatusOK, html)
  })

  return se.Next()
})

if err := app.Start(); err != nil {
  log.Fatal(err)
}
}
```

← [Prev: Sending emails](#)

[Next: Console commands](#) →

[FAQ](#)

[JavaScript SDK](#)

PocketBase

[Discussions](#)

[Dart SDK](#)

[Documentation](#)



[Introduction](#)[Going to production](#)[Web APIs reference](#)Extend with
GoExtend with
JavaScript[Go Overview](#)[Go Event hooks](#)[Go Routing](#)[Go Database](#)[Go Record operations](#)[Go Collection operations](#)[Go Migrations](#)[Go Jobs scheduling](#)[Go Sending emails](#)[Go Rendering templates](#)[Go Console commands](#)[Go Realtime messaging](#)[Go Filesystem](#)[Go Logging](#)[Go Testing](#)[Go Miscellaneous](#)

Extend with Go - Console commands

You can register custom console commands using `app.RootCmd.AddCommand(cmd)`, where `cmd` is a cobra command.

Here is an example:

```
package main

import (
    "log"

    "github.com/pocketbase/pocketbase"
    "github.com/spf13/cobra"
)

func main() {
    app := pocketbase.New()

    app.RootCmd.AddCommand(&cobra.Command{
        Use: "hello",
        Run: func(cmd *cobra.Command, args
[]string) {
            log.Println("Hello world!")
        },
    })

    if err := app.Start(); err != nil {
        log.Fatal(err)
    }
}
```

To run the command you can build your Go application and execute:

```
# or "go run main.go hello"
./myapp hello
```



Keep in mind that the console commands execute in their own separate app process and run independently from the main `serve` command

[Go](#) Record proxy

(aka. hook and realtime events between different processes are not shared with one another).

[← Prev: Rendering templates](#)

[Next: Realtime messaging →](#)

[FAQ](#)

[JavaScript SDK](#)

PocketBase

[Discussions](#)

[Dart SDK](#)

[Documentation](#)



[Introduction](#)[Going to production](#)[Web APIs reference](#)Extend with
GoExtend with
JavaScript[Go Overview](#)[Go Event hooks](#)[Go Routing](#)[Go Database](#)[Go Record operations](#)[Go Collection operations](#)[Go Migrations](#)[Go Jobs scheduling](#)[Go Sending emails](#)[Go Rendering templates](#)[Go Console commands](#)[Go Realtime messaging](#)[Go Filesystem](#)[Go Logging](#)[Go Testing](#)[Go Miscellaneous](#)

Extend with Go - Realtime messaging

By default PocketBase sends realtime events only for Record create/update/delete operations (*and for the OAuth2 auth redirect*), but you are free to send custom realtime messages to the connected clients via the `app.SubscriptionsBroker()` instance.

`app.SubscriptionsBroker().Clients()` returns all connected `subscriptions.Client` indexed by their unique connection id.

`app.SubscriptionsBroker().ChunkedClients(size)` is similar but returns the result as a chunked slice allowing you to split the iteration across several goroutines (usually combined with `errgroup`).

The current auth record associated with a client could be accessed through

```
client.Get(api.RealtimeClientAuthKey)
```



Note that a single authenticated user could have more than one active realtime connection (aka. multiple clients). This could happen for example when opening the same app in different tabs, browsers, devices, etc.

Below you can find a minimal code sample that sends a JSON payload to all clients subscribed to the "example" topic:

```
func notify(app core.App, subscription string,
data any) error {
    rawData, err := json.Marshal(data)
    if err != nil {
        return err
    }

    message := subscriptions.Message{
        Name: subscription,
        Data: rawData,
    }

    group := new(errgroup.Group)
```

```

    chunks :=
app.SubscriptionsBroker().ChunkedClients(300)

    for _, chunk := range chunks {
        group.Go(func() error {
            for _, client := range chunk {
                if
!client.HasSubscription(subscription) {
                    continue
                }

                client.Send(message)
            }

            return nil
        })
    }

    return group.Wait()
}

err := notify(app, "example",
map[string]any{"test": 123})
if err != nil {
    return err
}

```

From the client-side, users can listen to the custom subscription topic by doing something like:

JavaScript

Dart

```

import PocketBase from 'pocketbase';

const pb = new
PocketBase('http://127.0.0.1:8090');

...

await pb.realtime.subscribe('example', (e) => {
    console.log(e)
})

```

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



[Introduction](#)[Going to production](#)[Web APIs reference](#)Extend with
GoExtend with
JavaScript[Go Overview](#)[Go Event hooks](#)[Go Routing](#)[Go Database](#)[Go Record operations](#)[Go Collection operations](#)[Go Migrations](#)[Go Jobs scheduling](#)[Go Sending emails](#)[Go Rendering templates](#)[Go Console commands](#)[Go Realtime messaging](#)[Go Filesystem](#)[Go Logging](#)[Go Testing](#)[Go Miscellaneous](#)

Extend with Go - Filesystem

PocketBase comes with a thin abstraction between the local filesystem and S3.

To configure which one will be used you can adjust the storage settings from *Dashboard > Settings > Files storage* section.

The filesystem abstraction can be accessed programmatically via the `app.NewFilesystem()` method.

Below are listed some of the most common operations but you can find more details in the `filesystem` subpackage.



Always make sure to call `Close()` at the end for both the created filesystem instance and the retrieved file readers to prevent leaking resources.

- [Reading files](#)
- [Saving files](#)
- [Deleting files](#)

Reading files

To retrieve the file content of a single stored file you can use `GetReader(key)`.

Note that file keys often contain a **prefix** (aka. the "path" to the file). For record files the full key is `collectionId/recordId/filename`.

To retrieve multiple files matching a specific *prefix* you can use `List(prefix)`.

The below code shows a minimal example how to retrieve a single record file and copy its content into a `bytes.Buffer`.

```
record, err := app.FindAuthRecordByEmail("users",
    "test@example.com")
if err != nil {
    return err
}

// construct the full file key by concatenating
```

```

the record storage path with the specific filename
avatarKey := record.BaseFilePath() + "/" +
record.GetString("avatar")

// initialize the filesystem
fsys, err := app.NewFilesystem()
if err != nil {
    return err
}
defer fsys.Close()

// retrieve a file reader for the avatar key
r, err := fsys.GetReader(avatarKey)
if err != nil {
    return err
}
defer r.Close()

// do something with the reader...
content := new(bytes.Buffer)
_, err = io.Copy(content, r)
if err != nil {
    return err
}

```

Saving files

There are several methods to save (*aka. write/upload*) files depending on the available file content source:

- `Upload([]byte, key) ↗`
- `UploadFile(*filesystem.File, key) ↗`
- `UploadMultipart(*multipart.FileHeader, key) ↗`

Most users rarely will have to use the above methods directly because for collection records the file persistence is handled transparently when saving the record model (it will also perform size and MIME type validation based on the collection `file` field options). For example:

```

record, err := app.FindRecordById("articles",
"RECORD_ID")
if err != nil {
    return err
}

// Other available File factories
// - filesystem.NewFileFromBytes(data, name)

```



```
// - filesystem.NewFileFromURL(ctx, url)
// - filesystem.NewFileFromMultipart(mh)
f, err :=
filesystem.NewFileFromPath("/local/path/to/file")

// set new file (can be single *filesystem.File or
multiple []*filesystem.File)
// (if the record has an old file it is
automatically deleted on successful Save)
record.Set("yourFileField", f)

err = app.Save(record)
if err != nil {
    return err
}
```

Deleting files

Files can be deleted from the storage filesystem using

Delete(key) .

Similar to the previous section, most users rarely will have to use the `Delete` file method directly because for collection records the file deletion is handled transparently when removing the existing filename from the record model (this also ensures that the db entry referencing the file is also removed).

For example:

```
record, err := app.FindRecordById("articles",
"RECORD_ID")
if err != nil {
    return err
}

// if you want to "reset" a file field (aka.
deleting the associated single or multiple files)
// you can set it to nil
record.Set("yourFileField", nil)

// OR if you just want to remove individual
file(s) from a multiple file field you can use the
"-" modifier
// (the value could be a single filename string or
slice of filename strings)
record.Set("yourFileField-",
"example_52iWbGinWd.txt")

err = app.Save(record)
if err != nil {
```

```
    return err  
}
```

← Prev: Realtime messaging

Next: Logging →

[FAQ](#)

[JavaScript SDK](#)

PocketBase

[Discussions](#)

[Dart SDK](#)

[Documentation](#)



Extend with Go - Logging

`app.Logger()` provides access to a standard `slog.Logger` implementation that writes any logs into the database so that they can be later explored from the PocketBase *Dashboard* > *Logs* section.

For better performance and to minimize blocking on hot paths, logs are written with debounce and on batches:



- 3 seconds after the last debounced log write
- when the batch threshold is reached (currently 200)
- right before app termination to attempt saving everything from the existing logs queue

- Log methods
 - Debug(message, attrs...)
 - Info(message, attrs...)
 - Warn(message, attrs...)
 - Error(message, attrs...)
 - With(attrs...)
 - WithGroup(name)
- Logs settings
- Custom log queries
- Intercepting logs write

Log methods

All standard `slog.Logger` methods are available but below is a list with some of the most notable ones.

`Debug(message, attrs...)`

✔ Going to production

☰ Web APIs reference

Extend with
Go

Extend with
JavaScript

Go Overview

Go Event hooks

Go Routing

Go Database

Go Record operations

Go Collection operations

Go Migrations

Go Jobs scheduling

Go Sending emails

Go Rendering templates

Go Console commands

Go Realtime messaging

Go Filesystem

Go **Logging**

Go Testing

Go Miscellaneous

Go Record proxy

```
app.Logger().Debug("Debug message!")
```

```
app.Logger().Debug(  
  "Debug message with attributes!",  
  "name", "John Doe",  
  "id", 123,  
)
```

Info(message, attrs...)

```
app.Logger().Info("Info message!")
```

```
app.Logger().Info(  
  "Info message with attributes!",  
  "name", "John Doe",  
  "id", 123,  
)
```

Warn(message, attrs...)

```
app.Logger().Warn("Warning message!")
```

```
app.Logger().Warn(  
  "Warning message with attributes!",  
  "name", "John Doe",  
  "id", 123,  
)
```

Error(message, attrs...)

```
app.Logger().Error("Error message!")
```

```
app.Logger().Error(  
  "Error message with attributes!",  
  "id", 123,  
  "error", err,  
)
```

With(attrs...)

`With(attrs...)` creates a new local logger that will "inject" the specified attributes with each following log.

```
l := app.Logger().With("total", 123)
```

```
// results in log with data {"total": 123}
```

```
l.Info("message A")
```

```
// results in log with data {"total": 123, "name":  
"john"}
```

```
l.Info("message B", "name", "john")
```

WithGroup(name)

`WithGroup(name)` creates a new local logger that wraps all logs attributes under the specified group name.

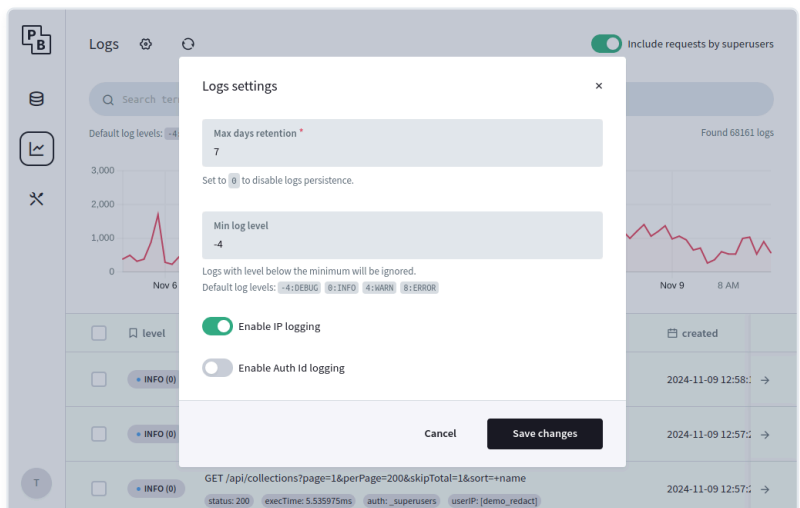
```
l := app.Logger().WithGroup("sub")
```

```
// results in log with data {"sub": { "total": 123  
}}
```

```
l.Info("message A", "total", 123)
```

Logs settings

You can control various log settings like logs retention period, minimal log level, request IP logging, etc. from the logs settings panel:



Custom log queries

The logs are usually meant to be filtered from the UI but if you want to programmatically retrieve and filter the stored logs you can make use of the `app.LogQuery()` query builder method. For example:

```
logs := []*core.Log{}
```

```
// see https://pocketbase.io/docs/go-  
database/#query-builder  
err := app.LogQuery().  
    // target only debug and info logs  
    AndWhere(dbx.In("level", -4, 0)).  
    // the data column is serialized json object  
    and could be anything  
    AndWhere(dbx.NewExp("json_extract(data,  
'$.type') = 'request'")).  
    OrderBy("created DESC").  
    Limit(100).  
    All(&logs)
```

Intercepting logs write

If you want to modify the log data before persisting in the database or to forward it to an external system, then you can listen for changes of the `_logs` table by attaching to the [base model hooks](#). For example:

```
app.OnModelCreate(core.LogsTableName).BindFunc(func  
c(e *core.ModelEvent) error {  
    l := e.Model.(*core.Log)  
  
    fmt.Println(l.Id)  
    fmt.Println(l.Created)  
    fmt.Println(l.Level)  
    fmt.Println(l.Message)  
    fmt.Println(l.Data)  
  
    return e.Next()  
})
```

← Prev: Filesystem

Next: Testing →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



Extend with Go - Testing

PocketBase exposes several test mocks and stubs (eg.

`tests.TestApp`, `tests.ApiScenario`,

`tests.MockMultipartData`, etc.) to help you write unit and integration tests for your app.

You could find more information in the

[github.com/pocketbase/pocketbase/tests](https://github.com/pocketbase/pocketbase/tree/main/tests) sub package, but here is a simple example.

- 1. Setup
- 2. Prepare test data
- 3. Integration test

1. Setup

Let's say that we have a custom API route `GET /my/hello` that requires superuser authentication:

```
// main.go
package main

import (
    "log"
    "net/http"

    "github.com/pocketbase/pocketbase"
    "github.com/pocketbase/pocketbase/apis"
    "github.com/pocketbase/pocketbase/core"
)

func bindAppHooks(app core.App) {
    app.OnServe().BindFunc(func(se
    *core.ServeEvent) error {
        se.Router.Get("/my/hello", func(e
    *core.RequestEvent) error {
            return e.JSON(http.StatusOK, "Hello
world!")
        }).Bind/apis.RequireSuperuserAuth()

    return se.Next()
})
```


✓ Going to production

☰ Web APIs reference

Extend with
Go

Extend with
JavaScript

Go Overview

Go Event hooks

Go Routing

Go Database

Go Record operations

Go Collection operations

Go Migrations

Go Jobs scheduling

Go Sending emails

Go Rendering templates

Go Console commands

Go Realtime messaging

Go Filesystem

Go Logging

Go **Testing**

Go Miscellaneous

Go Record proxy

```
app := pocketbase.New()

bindAppHooks(app)

if err := app.Start(); err != nil {
    log.Fatal(err)
}
```

2. Prepare test data

Now we have to prepare our test/mock data. There are several ways you can approach this, but the easiest one would be to start your application with a custom `test_pb_data` directory, e.g.:

```
./pocketbase serve --dir="./test_pb_data" --
automigrate=0
```

Go to your browser and create the test data via the Dashboard (both collections and records). Once completed you can stop the server (you could also commit `test_pb_data` to your repo).

3. Integration test

To test the example endpoint, we want to:

- ensure it handles only GET requests
- ensure that it can be accessed only by superusers
- check if the response body is properly set

Below is a simple integration test for the above test cases. We'll also use the test data created in the previous step.

```
// main_test.go
package main

import (
    "net/http"
    "testing"
```

```

    "github.com/pocketbase/pocketbase/core"
    "github.com/pocketbase/pocketbase/tests"
)

const testDataDir = "./test_pb_data"

func generateToken(collectionNameOrId string,
email string) (string, error) {
    app, err := tests.NewTestApp(testDataDir)
    if err != nil {
        return "", err
    }
    defer app.Cleanup()

    record, err :=
app.FindAuthRecordByEmail(collectionNameOrId,
email)
    if err != nil {
        return "", err
    }

    return record.NewAuthToken()
}

func TestHelloEndpoint(t *testing.T) {
    recordToken, err := generateToken("users",
"test@example.com")
    if err != nil {
        t.Fatal(err)
    }

    superuserToken, err :=
generateToken(core.CollectionNameSuperusers,
"test@example.com")
    if err != nil {
        t.Fatal(err)
    }

    // set up the test ApiScenario app instance
    setupTestApp := func(t testing.TB)
*tests.TestApp {
        testApp, err :=
tests.NewTestApp(testDataDir)
        if err != nil {
            t.Fatal(err)
        }
        // no need to cleanup since
scenario.Test() will do that for us
        // defer testApp.Cleanup()

        bindAppHooks(testApp)

```

```

        return testApp
    }

    scenarios := []tests.ApiScenario{
        {
            Name:          "try with different
http method, e.g. POST",
            Method:         http.MethodPost,
            URL:             "/my/hello",
            ExpectedStatus:  405,
            ExpectedContent: []string{"\"data\":
{}"},
            TestAppFactory:    setupTestApp,
        },
        {
            Name:          "try as guest (aka.
no Authorization header)",
            Method:         http.MethodGet,
            URL:             "/my/hello",
            ExpectedStatus:  401,
            ExpectedContent: []string{"\"data\":
{}"},
            TestAppFactory:    setupTestApp,
        },
        {
            Name:          "try as authenticated app
user",
            Method:         http.MethodGet,
            URL:             "/my/hello",
            Headers:         map[string]string{
                "Authorization": recordToken,
            },
            ExpectedStatus:  401,
            ExpectedContent: []string{"\"data\":
{}"},
            TestAppFactory:    setupTestApp,
        },
        {
            Name:          "try as authenticated
superuser",
            Method:         http.MethodGet,
            URL:             "/my/hello",
            Headers:         map[string]string{
                "Authorization": superuserToken,
            },
            ExpectedStatus:  200,
            ExpectedContent: []string{"Hello
world!"},
            TestAppFactory:    setupTestApp,
        },
    }
}

```

```
for _, scenario := range scenarios {  
    scenario.Test(t)  
}  
}
```

← Prev: Logging

Next: Miscellaneous →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



[Introduction](#)[Going to production](#)[Web APIs reference](#)[Extend with Go](#)[Extend with JavaScript](#)[Go Overview](#)[Go Event hooks](#)[Go Routing](#)[Go Database](#)[Go Record operations](#)[Go Collection operations](#)[Go Migrations](#)[Go Jobs scheduling](#)[Go Sending emails](#)[Go Rendering templates](#)[Go Console commands](#)[Go Realtime messaging](#)[Go Filesystem](#)[Go Logging](#)[Go Testing](#)[Go Miscellaneous](#)

Extend with Go - Miscellaneous

- [app.Store\(\)](#)
- [Security helpers](#)
 - [Generating random strings](#)
 - [Compare strings with constant time](#)
 - [AES Encrypt/Decrypt](#)

app.Store()

app.Store() returns a concurrent-safe application memory store that you can use to store anything for the duration of the application process (e.g. cache, config flags, etc.).

You can find more details about the available store methods in the **store.Store** documentation but the most commonly used ones are **Get(key)**, **Set(key, value)** and **GetOrSet(key, setFunc)**.

```
app.Store().Set("example", 123)

v1 := app.Store().Get("example").(int) // 123

v2 := app.Store().GetOrSet("example2", func() any {
    // this setter is invoked only once unless
    // "example2" is removed
    // (e.g. suitable for instantiating
    // singletons)
    return 456
}).(int) // 456
```

Keep in mind that the application store is also used internally usually with **pb*** prefixed keys (e.g. the collections cache is stored under the **pbAppCachedCollections** key) and changing these system keys or calling **RemoveAll()** / **Reset()** could have unintended side-effects.



If you want more advanced control you can initialize your own store independent from the application instance via `store.New[K, T](nil)`.

Security helpers

Below are listed some of the most commonly used security helpers but you can find detailed documentation for all available methods in the `security` subpackage.

Generating random strings

```
secret := security.RandomString(10) // e.g.
a35Vdb10Z4
```

```
secret := security.RandomStringWithAlphabet(5,
"1234567890") // e.g. 33215
```

Compare strings with constant time

```
isEqual := security.Equal(hash1, hash2)
```

AES Encrypt/Decrypt

```
// must be random 32 characters string
const key = "7Jgctik6116z8k3SUDxzZdmsR0Qtq9TA"

encrypted, err := security.Encrypt([]byte("test"),
key)
if err != nil {
    return err
}

decrypted := security.Decrypt(encrypted, key) //
[]byte("test")
```

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



[Introduction](#)[✔ Going to production](#)[☰ Web APIs reference](#)[Extend with
Go](#)[Extend with
JavaScript](#)[Go Overview](#)[Go Event hooks](#)[Go Routing](#)[Go Database](#)[Go Record operations](#)[Go Collection operations](#)[Go Migrations](#)[Go Jobs scheduling](#)[Go Sending emails](#)[Go Rendering templates](#)[Go Console commands](#)[Go Realtime messaging](#)[Go Filesystem](#)[Go Logging](#)[Go Testing](#)[Go Miscellaneous](#)

Extend with Go - Record proxy

The available `core.Record` and its [helpers](#) are usually the recommended way to interact with your data, but in case you want a typed access to your record fields you can create a helper struct that embeds `core.BaseRecordProxy` (which implements the `core.RecordProxy` interface) and define your collection fields as getters and setters.

By implementing the `core.RecordProxy` interface you can use your custom struct as part of a `RecordQuery` result like a regular record model. In addition, every DB change through the proxy struct will trigger the corresponding record validations and hooks. This ensures that other parts of your app, including 3rd party plugins, that don't know or use your custom struct will still work as expected.

Below is a sample `Article` record proxy implementation:

```
// article.go
package main

import (
    "github.com/pocketbase/pocketbase/core"
    "github.com/pocketbase/pocketbase/tools/types"
)

// ensures that the Article struct satisfy the
// core.RecordProxy interface
var _ core.RecordProxy = (*Article)(nil)

type Article struct {
    core.BaseRecordProxy
}

func (a *Article) Title() string {
    return a.GetString("title")
}

func (a *Article) SetTitle(title string) {
    a.Set("title", title)
}

func (a *Article) Slug() string {
    return a.GetString("slug")
}
```



```

}

func (a *Article) SetSlug(slug string) {
    a.Set("slug", slug)
}

func (a *Article) Created() types.DateTime {
    return a.GetDateTime("created")
}

func (a *Article) Updated() types.DateTime {
    return a.GetDateTime("updated")
}

```

Accessing and modifying the proxy records is the same as for the regular records. Continuing with the above `Article` example:

```

func FindArticleBySlug(app core.App, slug string)
(*Article, error){
    article := &Article{}

    err := app.RecordQuery("articles").
        AndWhere(dbx.NewExp("LOWER(slug)={:slug}",
        dbx.Params{
            "slug": strings.ToLower(slug), // case
insensitive match
        }))).
        Limit(1).
        One(article)

    if err != nil {
        return nil, err
    }

    return article, nil
}

...

article, err := FindArticleBySlug(app, "example")
if err != nil {
    return err
}

// change the title
article.SetTitle("Lorem ipsum...")

// persist the change while also triggering the
original record validations and hooks

```

```
err = app.Save(article)
if err != nil {
    return err
}
```

If you have an existing `*core.Record` value you can also load it into your proxy using the `SetProxyRecord` method:

```
// fetch regular record
record, err := app.FindRecordById("articles",
"RECORD_ID")
if err != nil {
    return err
}

// load into proxy
article := &Article{}
article.SetProxyRecord(record)
```

[← Prev: Miscellaneous](#)

[FAQ](#)

[JavaScript SDK](#)

[PocketBase](#)

[Discussions](#)

[Dart SDK](#)

[Documentation](#)



[Introduction](#)[Going to production](#)[Web APIs reference](#)[Extend with Go](#)[Extend with JavaScript](#)[JS Overview](#)[JS Event hooks](#)[JS Routing](#)[JS Database](#)[JS Record operations](#)[JS Collection operations](#)[JS Migrations](#)[JS Jobs scheduling](#)[JS Sending emails](#)[JS Rendering templates](#)[JS Console commands](#)[JS Sending HTTP requests](#)[JS Realtime messaging](#)[JS Filesystem](#)[JS Logging](#)

Extend with JavaScript - Event hooks

You can extend the default PocketBase behavior with custom server-side code using the exposed JavaScript app event hooks.

Throwing an error or not calling `e.next()` inside a handler function stops the hook execution chain.



All hook handler functions share the same `function(e){}` signature and expect the user to call `e.next()` if they want to proceed with the execution chain.

- [App hooks](#)
- [Mailer hooks](#)
- [Realtime hooks](#)
- [Record model hooks](#)
- [Collection model hooks](#)
- [Request hooks](#)
- [Base model hooks](#)

App hooks

[onBootstrap](#)[onSettingsReload](#)[onBackupCreate](#)[onBackupRestore](#)[onTerminate](#)

Mailer hooks

onMailerSend



onMailerRecordAuthAlertSend



onMailerRecordPasswordResetSend



onMailerRecordVerificationSend



onMailerRecordEmailChangeSend



onMailerRecordOTPSend



Realtime hooks

onRealtimeConnectRequest



onRealtimeSubscribeRequest



onRealtimeMessageSend



Record model hooks



These are lower level Record model hooks and could be triggered from anywhere (custom console command, scheduled cron job, when calling `e.save(record)`, etc.) and therefore they have no access to the request context!

If you want to intercept the builtin Web APIs and to access their request body, query parameters, headers or the request auth state, then please use the designated Record ***Request** hooks.

onRecordEnrich



onRecordValidate



Record model create hooks

onRecordCreate



onRecordCreateExecute



onRecordAfterCreateSuccess



onRecordAfterCreateError



Record model update hooks

onRecordUpdate



onRecordUpdateExecute



onRecordAfterUpdateSuccess



onRecordAfterUpdateError



Record model delete hooks

onRecordDelete



onRecordDeleteExecute



onRecordAfterDeleteSuccess



onRecordAfterDeleteError



Collection model hooks

These are lower level Collection model hooks and could be triggered from anywhere (custom console command, scheduled cron job, when calling



`e.save(collection)`, etc.) and therefore they have no access to the request context!

If you want to intercept the builtin Web APIs and to access their request body, query parameters, headers or the request auth state, then please use the designated Collection ***Request** hooks .

onCollectionValidate



Collection mode create hooks

onCollectionCreate



onCollectionCreateExecute



onCollectionAfterCreateSuccess



onCollectionAfterCreateError



Collection mode update hooks

onCollectionUpdate



onCollectionUpdateExecute



onCollectionAfterUpdateSuccess



onCollectionAfterUpdateError



Collection mode delete hooks

onCollectionDelete



onCollectionDeleteExecute



onCollectionAfterDeleteSuccess



onCollectionAfterDeleteError



Request hooks



The request hooks are triggered only when the corresponding API request endpoint is accessed.

Record CRUD request hooks

onRecordsListRequest



onRecordViewRequest



onRecordCreateRequest



onRecordUpdateRequest



onRecordDeleteRequest



Record auth request hooks

onRecordAuthRequest



onRecordAuthRefreshRequest



onRecordAuthWithPasswordRequest



onRecordAuthWithOAuth2Request



onRecordRequestPasswordResetRequest



onRecordConfirmPasswordResetRequest



onRecordRequestVerificationRequest



onRecordConfirmVerificationRequest



onRecordRequestEmailChangeRequest	▼
-----------------------------------	---



onRecordConfirmEmailChangeRequest	▼
-----------------------------------	---



onRecordRequestOTPRequest	▼
---------------------------	---



onRecordAuthWithOTPRequest	▼
----------------------------	---



Batch request hooks

onBatchRequest	▼
----------------	---



File request hooks

onFileDownloadRequest	▼
-----------------------	---



onFileTokenRequest	▼
--------------------	---



Collection request hooks

onCollectionsListRequest	▼
--------------------------	---



onCollectionViewRequest	▼
-------------------------	---



onCollectionCreateRequest	▼
---------------------------	---



onCollectionUpdateRequest	▼
---------------------------	---



onCollectionDeleteRequest	▼
---------------------------	---



onCollectionsImportRequest	▼
----------------------------	---



Settings request hooks

onSettingsListRequest	▼
-----------------------	---



onSettingsUpdateRequest	▼
-------------------------	---



Base model hooks

The Model hooks are fired for all PocketBase structs that implements the Model DB interface - Record, Collection, Log, etc.



For convenience, if you want to listen to only the Record or Collection DB model events without doing manual type assertion, you can use the **onRecord*** and **onCollection*** proxy hooks above.

onModelValidate



Base model create hooks

onModelCreate



onModelCreateExecute



onModelAfterCreateSuccess



onModelAfterCreateError



Base model update hooks

onModelUpdate



onModelUpdateExecute



onModelAfterUpdateSuccess



onModelAfterUpdateError



Base model delete hooks

onModelDelete



onModelDeleteExecute

onModelAfterDeleteSuccess

onModelAfterDeleteError

← Prev: Overview

Next: Routing →

[Introduction](#)[Going to production](#)[Web APIs reference](#)Extend with
GoExtend with
JavaScript[JS Overview](#)[JS Event hooks](#)[JS Routing](#)[JS Database](#)[JS Record operations](#)[JS Collection operations](#)[JS Migrations](#)[JS Jobs scheduling](#)[JS Sending emails](#)[JS Rendering templates](#)[JS Console commands](#)[JS Sending HTTP requests](#)[JS Realtime messaging](#)[JS Filesystem](#)[JS Logging](#)

Extend with JavaScript - Routing

You can register custom routes and middlewares by using the top-level `routerAdd()` and `routerUse()` functions.

- **Routes**

- [Registering new routes](#)
- [Path parameters and matching rules](#)
- [Reading path parameters](#)
- [Retrieving the current auth state](#)
- [Reading query parameters](#)
- [Reading request headers](#)
- [Writing response headers](#)
- [Retrieving uploaded files](#)
- [Reading request body](#)
- [Writing response body](#)
- [Reading the client IP](#)
- [Request store](#)

- **Middleware**

- [Registering middlewares](#)
- [Builtin middlewares](#)
- [Default globally registered middlewares](#)

- **Error response**

- **Helpers**

- [Serving static directory](#)
- [Auth response](#)
- [Enrich record\(s\)](#)

- **[Sending request to custom routes using the SDKs](#)**

Routes

Registering new routes

Every route has a path, handler function and eventually middlewares attached to it. For example:

```
// register "GET /hello/{name}" route (allowed for
everyone)
routerAdd("GET", "/hello/{name}", (e) => {
  let name = e.request.pathValue("name")

  return e.json(200, { "message": "Hello " +
name })
})

// register "POST /api/myapp/settings" route
(allowed only for authenticated users)
routerAdd("POST", "/api/myapp/settings", (e) => {
  // do something ...
  return e.json(200, {"success": true})
}, $apis.requireAuth())
```

Path parameters and matching rules

Because PocketBase routing is based on top of the Go standard router mux, we follow the same pattern matching rules. Below you could find a short overview but for more details please refer to [net/http.ServeMux](#).

In general, a route pattern looks like `[METHOD] [HOST]/[PATH]`.

Route paths can include parameters in the format

`{paramName}`.

You can also use `{paramName...}` format to specify a parameter that targets more than one path segment.

A pattern ending with a trailing slash `/` acts as anonymous wildcard and matches any requests that begins with the defined route. If you want to have a trailing slash but to indicate the end of the URL then you need to end the path with the special `{$}` parameter.



If your route path starts with `/api/` consider combining it with your unique app name like `/api/myapp/...` to avoid collisions with system routes.

Here are some examples:

```
// match "GET example.com/index.html"
routerAdd("GET", "example.com/index.html", ...)
```

```
// match "GET /index.html" (for any host)
routerAdd("GET", "/index.html", ...)

// match "GET /static/", "GET /static/a/b/c",
etc.
routerAdd("GET", "/static/", ...)

// match "GET /static/", "GET /static/a/b/c",
etc.
// (similar to the above but with a named
wildcard parameter)
routerAdd("GET", "/static/{path...}", ...)

// match only "GET /static/" (if no "/static" is
registered, it is 301 redirected)
routerAdd("GET", "/static/{${}}", ...)

// match "GET /customers/john", "GET
/customers/jane", etc.
routerAdd("GET", "/customers/{name}", ...)
```

In the following examples `e` is usually `core.RequestEvent` [↗](#) value.

Reading path parameters

```
let id = e.request.pathValue("id")
```

Retrieving the current auth state

The request auth state can be accessed (or set) via the `RequestEvent.auth` field.

```
let authRecord = e.auth

let isGuest = !e.auth

// the same as "e.auth?.isSuperuser()"
let isSuperuser = e.hasSuperuserAuth()
```

Alternatively you could also access the request data from the summarized request info instance (*usually used in hooks like the*

`onRecordEnrich` *where there is no direct access to the request*)

.

```
let info = e.requestInfo()

let authRecord = info.auth

let isGuest = !info.auth

// the same as "info.auth?.isSuperuser()"
let isSuperuser = info.hasSuperuserAuth()
```

Reading query parameters

```
// retrieve the first value of the "search" query
param
let search = e.request.url.query().get("search")

// or via the parsed request info
let search = e.requestInfo().query["search"]

// in case of array query params (e.g.
search=123&search=456)
let query =
JSON.parse(toString(e.request.url.query())) || {}
let arr = query.search; // ["123", "456"]
```

Reading request headers

```
let token = e.request.header.get("Some-Header")

// or via the parsed request info
// (the header value is always normalized per the
@request.headers.* API rules format)
let token = e.requestInfo().headers["some_header"]
```

Writing response headers

```
e.response.header().set("Some-Header", "123")
```

Retrieving uploaded files

```
// retrieve the uploaded files and parse the found
multipart data into a ready-to-use
[]*filesystem.File
let files = e.findUploadedFiles("document")
```

```
// or retrieve the raw single multipart/form-data
file and header
let [mf, mh] = e.request.formFile("document")
```

Reading request body

Body parameters can be read either via `e.bindBody` OR through the parsed request info.

```
// retrieve the entire raw body as string
console.log(toString(e.request.body))

// read the body fields via the parsed request
object
let body = e.requestInfo().body
console.log(body.title)

// OR read/scan the request body fields into a
typed object
const data = new DynamicModel({
  // describe the fields to read (used also as
  initial values)
  someTextField: "",
  someIntValue: 0,
  someFloatValue: -0,
  someBoolField: false,
  someArrayField: [],
  someObjectField: {}, // object props are
  accessible via .get(key)
})
e.bindBody(data)
console.log(data.sometextField)
```

Writing response body

```
// send response with JSON body
// (it also provides a generic response fields
picker/filter if the "fields" query parameter is
set)
e.json(200, {"name": "John"})

// send response with string body
e.string(200, "Lorem ipsum...")

// send response with HTML body
// (check also the "Rendering templates" section)
e.html(200, "<h1>Hello!</h1>")
```

```
// redirect
e.redirect(307, "https://example.com")

// send response with no body
e.noContent(204)

// serve a single file
e.fileFS($os.dirFS("."), "example.txt")

// stream the specified reader
e.stream(200, "application/octet-stream", reader)

// send response with blob (bytes array) body
e.blob(200, "application/octet-stream", [ ... ])
```

Reading the client IP

```
// The IP of the last client connecting to your
// server.
// The returned IP is safe and can be always
// trusted.
// When behind a reverse proxy (e.g. nginx) this
// method returns the IP of the proxy.
//
// (/jsvm/interfaces/core.RequestEvent.html#remoteIP)
let ip = e.remoteIP()

// The "real" IP of the client based on the
// configured Settings.trustedProxy header(s).
// If such headers are not set, it fallbacks to
// e.remoteIP().
//
// (/jsvm/interfaces/core.RequestEvent.html#realIP)
let ip = e.realIP()
```

Request store

The `core.RequestEvent` comes with a local store that you can use to share custom data between middlewares and the route action.

```
// store for the duration of the request
e.set("someKey", 123)

// retrieve later
let val = e.get("someKey") // 123
```


Middlewares

Middlewares allow inspecting, intercepting and filtering route requests.

Middlewares can be registered both to a single route (by passing them after the handler) and globally usually by using `routerUse(middleware)`.

Registering middlewares

Here is a minimal example of what a global middleware looks like:

```
// register a global middleware
routerUse((e) => {
  if (e.request.header.get("Something") == "") {
    throw new BadRequestError("Something
header value is missing!")
  }

  return e.next()
})
```

Middleware can be either registered as simple functions (`function(e){}`) or if you want to specify a custom priority and id - as a `Middleware` class instance.

Below is a slightly more advanced example showing all options and the execution sequence:

```
// attach global middleware
routerUse((e) => {
  console.log(1)
  return e.next()
})

// attach global middleware with a custom priority
routerUse(new Middleware((e) => {
  console.log(2)
  return e.next()
}, -1))

// attach middleware to a single route
//
// "GET /hello" should print the sequence: 2,1,3,4
routerAdd("GET", "/hello", (e) => {
  console.log(4)
  return e.string(200, "Hello!")
})
```

```
}, (e) => {
  console.log(3)
  return e.next()
})
```

Builtin middlewares

The global `$apis.*` object exposes several middlewares that you can use as part of your application.

```
// Require the request client to be
// unauthenticated (aka. guest).
$apis.requireGuestOnly()

// Require the request client to be authenticated
// (optionally specify a list of allowed auth
// collection names, default to any).
$apis.requireAuth(optCollectionNames...)

// Require the request client to be authenticated
// as superuser
// (this is an alias for
$apis.requireAuth("_superusers")).
$apis.requireSuperuserAuth()

// Require the request client to be authenticated
// as superuser OR
// regular auth record with id matching the
// specified route parameter (default to "id").
$apis.requireSuperuserOrOwnerAuth(ownerIdParam)

// Changes the global 32MB default request body
// size limit (set it to 0 for no limit).
// Note that system record routes have dynamic
// body size limit based on their collection field
// types.
$apis.bodyLimit(limitBytes)

// Compresses the HTTP response using Gzip
// compression scheme.
$apis.zip()

// Instructs the activity logger to log only
// requests that have failed/returned an error.
$apis.skipSuccessActivityLog()
```

Default globally registered middlewares

The below list is mostly useful for users that may want to plug their own custom middlewares before/after the priority of the default global ones, for example: registering a custom auth loader before the rate limiter with `-1001` so that the rate limit can be applied properly based on the loaded auth state.

All PocketBase applications have the below internal middlewares registered out of the box (*sorted by their priority*):

- **WWW redirect** (id: pbWWWRedirect, priority: -99999)
Performs www -> non-www redirect(s) if the request host matches with one of the values in certificate host policy.
- **CORS** (id: pbCors, priority: -1041)
By default all origins are allowed (PocketBase is stateless and doesn't rely on cookies) but this can be configured with the `--origins` flag.
- **Activity logger** (id: pbActivityLogger, priority: -1040)
Saves request information into the logs auxiliary database.
- **Auto panic recover** (id: pbPanicRecover, priority: -1030)
Default panic-recover handler.
- **Auth token loader** (id: pbLoadAuthToken, priority: -1020)
Loads the auth token from the `Authorization` header and populates the related auth record into the request event (aka. `e.auth`).
- **Security response headers**
(id: pbSecurityHeaders, priority: -1010)
Adds default common security headers (`X-XSS-Protection` , `X-Content-Type-Options` , `X-Frame-Options`) to the response (can be overwritten by other middlewares or from inside the route action).
- **Rate limit** (id: pbRateLimit, priority: -1000)
Rate limits client requests based on the configured app settings (it does nothing if the rate limit option is not enabled).
- **Body limit** (id: pbBodyLimit, priority: -990)
Applies a default max ~32MB request body limit for all custom routes (system record routes have dynamic body size limit based on their collection field types). Can be overwritten on group/route level by simply rebinding the `$apis.bodyLimit(limitBytes)` middleware.

Error response

PocketBase has a global error handler and every returned or thrown `Error` from a route or middleware will be safely converted by default to a generic API error to avoid accidentally leaking sensitive information (the original error will be visible only in the *Dashboard > Logs* or when in `--dev` mode).

To make it easier returning formatted json error responses, PocketBase provides `ApiError` constructor that can be instantiated directly or using the builtin factories.

`ApiError.data` will be returned in the response only if it is a map of `ValidationError` items.

```
// construct ApiError with custom status code and
validation data error
throw new ApiError(500, "something went wrong", {
  "title": new ValidationError("invalid_title",
    "Invalid or missing title"),
})

// if message is empty string, a default one will
be set
throw new BadRequestError(optMessage, optData)
// 400 ApiError
throw new UnauthorizedError(optMessage, optData)
// 401 ApiError
throw new ForbiddenError(optMessage, optData)
// 403 ApiError
throw new NotFoundError(optMessage, optData)
// 404 ApiError
throw new TooManyRequestsError(optMessage,
optData) // 429 ApiError
throw new InternalServerError(optMessage, optData)
// 500 ApiError
```

Helpers

Serving static directory

`$apis.static()` [↗] serves static directory content from `fs.FS` instance.

Expects the route to have a `{path...}` wildcard parameter.

```
// serves static files from the provided dir (if
exists)
routerAdd("GET", "{path...}",
$apis.static($os.dirFS("/path/to/public"), false))
```

Auth response

`$apis.recordAuthResponse()` writes standardized JSON record auth response (aka. token + record data) into the specified request body. Could be used as a return result from a custom auth route.

```
routerAdd("POST", "/phone-login", (e) => {
  const data = new DynamicModel({
    phone: "",
    password: "",
  })
  e.bindBody(data)

  let record =
e.app.findFirstRecordByData("users", "phone",
data.phone)
  if !record.validatePassword(data.password) {
    // return generic 400 error as a basic
    enumeration protection
    throw new BadRequestError("Invalid
credentials")
  }

  return $apis.recordAuthResponse(e, record,
"phone")
})
```

Enrich record(s)

`$apis.enrichRecord()` and `$apis.enrichRecords()` helps parses the request context and enrich the provided record(s) by:

- expands relations (if `defaultExpands` and/or `?expand` query parameter is set)
- ensures that the emails of the auth record and its expanded auth relations are visible only for the current logged superuser, record owner or record with manage access

These helpers are also responsible for triggering the `onRecordEnrich` hook events.

```
routerAdd("GET", "/custom-article", (e) => {
  let records =
e.app.findRecordsByFilter("article", "status =
'active'", "-created", 40, 0)
```

```
// enrich the records with the "categories"
relation as default expand
$apis.enrichRecords(e, records, "categories")

return e.json(200, records)
})
```

Sending request to custom routes using the SDKs

The official PocketBase SDKs expose the internal `send()` method that could be used to send requests to your custom route(s).

JavaScript

Dart

```
import PocketBase from 'pocketbase';

const pb = new
PocketBase('http://127.0.0.1:8090');

await pb.send("/hello", {
  // for other options check
  // https://developer.mozilla.org/en-US/docs/Web/API/fetch#options
  query: { "abc": 123 },
});
```

← Prev: Event hooks

Next: Database →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



[Introduction](#)[Going to production](#)[Web APIs reference](#)Extend with
GoExtend with
JavaScript[JS Overview](#)[JS Event hooks](#)[JS Routing](#)[JS Database](#)[JS Record operations](#)[JS Collection operations](#)[JS Migrations](#)[JS Jobs scheduling](#)[JS Sending emails](#)[JS Rendering templates](#)[JS Console commands](#)[JS Sending HTTP requests](#)[JS Realtime messaging](#)[JS Filesystem](#)[JS Logging](#)

Extend with JavaScript - Database

`$app` is the main interface to interact with your database.

`$app.db()` returns a `dbx.Builder` that can run all kinds of SQL statements, including raw queries.



For more details and examples how to interact with Record and Collection models programmatically you could also check [Collection operations](#) and [Record operations](#) sections.

- [Executing queries](#)
- [Binding parameters](#)
- [Query builder](#)
 - [select\(\)](#), [andSelect\(\)](#), [distinct\(\)](#)
 - [from\(\)](#)
 - [join\(\)](#)
 - [where\(\)](#), [andWhere\(\)](#), [orWhere\(\)](#)
 - [orderBy\(\)](#), [andOrderBy\(\)](#)
 - [groupBy\(\)](#), [andGroupBy\(\)](#)
 - [having\(\)](#), [andHaving\(\)](#), [orHaving\(\)](#)
 - [limit\(\)](#)
 - [offset\(\)](#)
- [Transaction](#)

Executing queries

To execute DB queries you can start with the `newQuery("...")` statement and then call one of:

- `execute()` - for any query statement that is not meant to retrieve data:

```
$app.db()  
  .newQuery("DELETE FROM articles WHERE
```

```
status = 'archived')
    .execute() // throw an error on db failure
```

- `one()` - to populate a single row into `DynamicModel` object:

```
const result = new DynamicModel({
    // describe the shape of the data (used
    // also as initial values)
    // the keys cannot start with underscore
    // and must be a valid Go struct field name
    "id":      "", // or nullString() if
    nullable
    "status":   false, // or nullBool() if
    nullable
    "age":      0, // or nullInt() if
    nullable
    "totalSpent": -0, // or nullFloat() if
    nullable
    "roles":    [], // or nullArray() if
    nullable;
    "meta":     {}, // or nullObject() if
    nullable
})

$app.db()
    .newQuery("SELECT id, status, age,
totalSpent, roles, meta FROM users WHERE
id=1")
    .one(result) // throw an error on db
failure or missing row

console.log(result.id)
```

- `all()` - to populate multiple rows into an array of objects (note that the array must be created with `arrayOf`):

```
const result = arrayOf(new DynamicModel({
    // describe the shape of the data (used
    // also as initial values)
    // the keys cannot start with underscore
    // and must be a valid Go struct field name
    "id":      "" // or nullString() if
    nullable
    "status":   false, // or nullBool() if
    nullable
    "age":      0, // or nullInt() if
    nullable
    "totalSpent": -0, // or nullFloat() if
    nullable
```



```

        "roles":      [],      // or nullArray() if
        nullable
        "meta":       {},      // or nullObject() if
        nullable
    )))

$app.db()
    .newQuery("SELECT id, status, age,
totalSpent, Roles, meta FROM users LIMIT 100")
    .all(result) // throw an error on db
failure

if (result.length > 0) {
    console.log(result[0].id)
}

```

Binding parameters

To prevent SQL injection attacks, you should use named parameters for any expression value that comes from user input. This could be done using the named `{:paramName}` placeholders in your SQL statement and then define the parameter values for the query with `bind(params)`. For example:

```

const result = arrayOf(new DynamicModel({
    "name":      "",
    "created":   "",
}))

$app.db()
    .newQuery("SELECT name, created FROM posts
WHERE created >= {:from} and created <= {:to}")
    .bind({
        "from": "2023-06-25 00:00:00.000Z",
        "to":   "2023-06-28 23:59:59.999Z",
    })
    .all(result)

console.log(result.length)

```

Query builder

Instead of writing plain SQLs, you can also compose SQL statements programmatically using the db query builder. Every SQL keyword has a corresponding query building

method. For example, `SELECT` corresponds to `select()`, `FROM` corresponds to `from()`, `WHERE` corresponds to `where()`, and so on.

```
const result = arrayOf(new DynamicModel({
    "id": "",
    "email": "",
}))

$app.db()
    .select("id", "email")
    .from("users")
    .andWhere($dbx.like("email", "example.com"))
    .limit(100)
    .orderBy("created ASC")
    .all(result)
```

`select()`, `andSelect()`, `distinct()`

The `select(...cols)` method initializes a `SELECT` query builder. It accepts a list of the column names to be selected. To add additional columns to an existing select query, you can call `andSelect()`.

To select distinct rows, you can call `distinct(true)`.

```
$app.db()
    .select("id", "avatar as image")
    .andSelect("(firstName || ' ' || lastName) as fullName")
    .distinct(true)
    ...
```

`from()`

The `from(...tables)` method specifies which tables to select from (plain table names are automatically quoted).

```
$app.db()
    .select("table1.id", "table2.name")
    .from("table1", "table2")
    ...
```

`join()`

The `join(type, table, on)` method specifies a `JOIN` clause. It takes 3 parameters:

- `type` - join type string like `INNER JOIN`, `LEFT JOIN`, etc.
- `table` - the name of the table to be joined
- `on` - optional `dbx.Expression` as an `ON` clause

For convenience, you can also use the shortcuts

`innerJoin(table, on)`, `leftJoin(table, on)`, `rightJoin(table, on)` to specify `INNER JOIN`, `LEFT JOIN` and `RIGHT JOIN`, respectively.

```
$app.db()
  .select("users.*")
  .from("users")
  .innerJoin("profiles",
$dbx.exp("profiles.user_id = users.id"))
  .join("FULL OUTER JOIN", "department",
$dbx.exp("department.id = {:id}", {id: "someId"}))
  ...
```

where(), andWhere(), orWhere()

The `where(exp)` method specifies the `WHERE` condition of the query.

You can also use `andWhere(exp)` or `orWhere(exp)` to append additional one or more conditions to an existing `WHERE` clause. Each where condition accepts a single `dbx.Expression` (see below for full list).

```
/*
SELECT users.*
FROM users
WHERE id = "someId" AND
      status = "public" AND
      name like "%john%" OR
      (
        role = "manager" AND
        fullTime IS TRUE AND
        experience > 10
      )
*/
$app.db()
  .select("users.*")
  .from("users")
  .where($dbx.exp("id = {:id}", { id: "someId"
})))
  .andWhere($dbx.hashExp({ status: "public" })))
  .andWhere($dbx.like("name", "john"))
  .orWhere($dbx.and(
    $dbx.hashExp({
```

```

        role: "manager",
        fullTime: true,
    }},
    $dbx.exp("experience > {:exp}", { exp: 10
})
))
...

```

The following `dbx.Expression` methods are available:

- `$dbx.exp(raw, optParams)`

Generates an expression with the specified raw query fragment. Use the `optParams` to bind parameters to the expression.

```

$dbx.exp("status = 'public'")
$dbx.exp("total > {:min} AND total < {:max}",
{ min: 10, max: 30 })

```

- `$dbx.hashExp(pairs)`

Generates a hash expression from a map whose keys are DB column names which need to be filtered according to the corresponding values.

```

// slug = "example" AND active IS TRUE AND
tags in ("tag1", "tag2", "tag3") AND parent IS
NULL
$dbx.hashExp({
  slug: "example",
  active: true,
  tags: ["tag1", "tag2", "tag3"],
  parent: null,
})

```

- `$dbx.not(exp)`

Negates a single expression by wrapping it with `NOT()`.

```

// NOT(status = 1)
$dbx.not($dbx.exp("status = 1"))

```

- `$dbx.and(...exps)`

Creates a new expression by concatenating the specified ones with `AND`.

```

// (status = 1 AND username like "%john%")
$dbx.and($dbx.exp("status = 1"),
$dbx.like("username", "john"))

```

- `$dbx.or(...exps)`

Creates a new expression by concatenating the specified ones with `OR`.

```
// (status = 1 OR username like "%john%")
$dbx.or($dbx.exp("status = 1"),
$dbx.like("username", "john"))
```

- `$dbx.in(col, ...values)`

Generates an `IN` expression for the specified column and the list of allowed values.

```
// status IN ("public", "reviewed")
$dbx.in("status", "public", "reviewed")
```

- `$dbx.notIn(col, ...values)`

Generates an `NOT IN` expression for the specified column and the list of allowed values.

```
// status NOT IN ("public", "reviewed")
$dbx.notIn("status", "public", "reviewed")
```

- `$dbx.like(col, ...values)`

Generates a `LIKE` expression for the specified column and the possible strings that the column should be like. If multiple values are present, the column should be like **all** of them.

By default, each value will be surrounded by `"%"` to enable partial matching. Special characters like `"%"`, `"\"`, `"_"` will also be properly escaped. You may call

`escape(...pairs)` and/or `match(left, right)` to change the default behavior.

```
// name LIKE "%test1%" AND name LIKE "%test2%"
$dbx.like("name", "test1", "test2")

// name LIKE "test1%"
$dbx.like("name", "test1").match(false, true)
```

- `$dbx.notLike(col, ...values)`

Generates a `NOT LIKE` expression in similar manner as `like()`.

```
// name NOT LIKE "%test1%" AND name NOT LIKE
"%test2%"
$dbx.notLike("name", "test1", "test2")
```

```
// name NOT LIKE "test1%"
$dbx.notLike("name", "test1").match(false,
true)
```

- `$dbx.orLike(col, ...values)`

This is similar to `like()` except that the column must be one of the provided values, aka. multiple values are concatenated with `OR` instead of `AND`.

```
// name LIKE "%test1%" OR name LIKE "%test2%"
$dbx.orLike("name", "test1", "test2")

// name LIKE "test1%" OR name LIKE "test2%"
$dbx.orLike("name", "test1",
"test2").match(false, true)
```

- `$dbx.orNotLike(col, ...values)`

This is similar to `notLike()` except that the column must not be one of the provided values, aka. multiple values are concatenated with `OR` instead of `AND`.

```
// name NOT LIKE "%test1%" OR name NOT LIKE
"%test2%"
$dbx.orNotLike("name", "test1", "test2")

// name NOT LIKE "test1%" OR name NOT LIKE
"test2%"
$dbx.orNotLike("name", "test1",
"test2").match(false, true)
```

- `$dbx.exists(exp)`

Prefix with `EXISTS` the specified expression (usually a subquery).

```
// EXISTS (SELECT 1 FROM users WHERE status =
'active')
$dbx.exists(dbx.exp("SELECT 1 FROM users WHERE
status = 'active'"))
```

- `$dbx.notExists(exp)`

Prefix with `NOT EXISTS` the specified expression (usually a subquery).

```
// NOT EXISTS (SELECT 1 FROM users WHERE
status = 'active')
```

```
$dbx.notExists(dbx.exp("SELECT 1 FROM users  
WHERE status = 'active'"))
```

- `$dbx.between(col, from, to)`

Generates a `BETWEEN` expression with the specified range.

```
// age BETWEEN 3 and 99  
$dbx.between("age", 3, 99)
```

- `$dbx.notBetween(col, from, to)`

Generates a `NOT BETWEEN` expression with the specified range.

```
// age NOT BETWEEN 3 and 99  
$dbx.notBetween("age", 3, 99)
```

orderBy(), andOrderBy()

The `orderBy(...cols)` specifies the `ORDER BY` clause of the query.

A column name can contain `"ASC"` or `"DESC"` to indicate its ordering direction.

You can also use `andOrderBy(...cols)` to append additional columns to an existing `ORDER BY` clause.

```
$app.db()  
  .select("users.*")  
  .from("users")  
  .orderBy("created ASC", "updated DESC")  
  .andOrderBy("title ASC")  
  ...
```

groupBy(), andGroupBy()

The `groupBy(...cols)` specifies the `GROUP BY` clause of the query.

You can also use `andGroupBy(...cols)` to append additional columns to an existing `GROUP BY` clause.

```
$app.db()  
  .select("users.*")  
  .from("users")  
  .groupBy("department", "level")  
  ...
```

having(), andHaving(), orHaving()

The `having(exp)` specifies the `HAVING` clause of the query. Similarly to `where(exp)`, it accepts a single `dbx.Expression` (see all available expressions listed above).

You can also use `andHaving(exp)` or `orHaving(exp)` to append additional one or more conditions to an existing `HAVING` clause.

```
$app.db()
  .select("users.*")
  .from("users")
  .groupBy("department", "level")
  .having($dbx.exp("sum(level) > { :sum }", { sum:
10 })))
  ...
```

limit()

The `limit(number)` method specifies the `LIMIT` clause of the query.

```
$app.db()
  .select("users.*")
  .from("users")
  .limit(30)
  ...
```

offset()

The `offset(number)` method specifies the `OFFSET` clause of the query. Usually used together with `limit(number)`.

```
$app.db()
  .select("users.*")
  .from("users")
  .offset(5)
  .limit(30)
  ...
```

Transaction

To execute multiple queries in a transaction you can use

```
$app.runInTransaction(fn) ↗ .
```


The DB operations are persisted only if the transaction completes without throwing an error.

It is safe to nest `runInTransaction` calls as long as you use the callback's `txApp` argument.



Inside the transaction function always use its `txApp` argument and not the original `$app` instance because we allow only a single writer/transaction at a time and it could result in a deadlock.

To avoid performance issues, try to minimize slow/long running tasks such as sending emails, connecting to external services, etc. as part of the transaction.

```
$app.runInTransaction((txApp) => {  
  // update a record  
  const record =  
    txApp.findRecordById("articles", "RECORD_ID")  
    record.set("status", "active")  
    txApp.save(record)  
  
  // run a custom raw query (doesn't fire event  
  hooks)  
  txApp.db().newQuery("DELETE FROM articles  
  WHERE status = 'pending']").execute()  
})
```

[← Prev: Routing](#)

[Next: Record operations →](#)

[FAQ](#)

[JavaScript SDK](#)

PocketBase

[Discussions](#)

[Dart SDK](#)

[Documentation](#)



[Introduction](#)[Going to production](#)[Web APIs reference](#)Extend with
GoExtend with
JavaScript[JS Overview](#)[JS Event hooks](#)[JS Routing](#)[JS Database](#)[JS Record operations](#)[JS Collection operations](#)[JS Migrations](#)[JS Jobs scheduling](#)[JS Sending emails](#)[JS Rendering templates](#)[JS Console commands](#)[JS Sending HTTP requests](#)[JS Realtime messaging](#)[JS Filesystem](#)[JS Logging](#)

Extend with JavaScript - Record operations

The most common task when extending PocketBase probably would be querying and working with your collection records.

You could find detailed documentation about all the supported Record model methods in `core.Record` type interface but below are some examples with the most common ones.

- [Set field value](#)
- [Get field value](#)
- [Auth accessors](#)
- [Copies](#)
- [Hide/Unhide fields](#)
- [Fetch records](#)
 - [Fetch single record](#)
 - [Fetch multiple records](#)
 - [Fetch auth records](#)
 - [Custom record query](#)
- [Create new record](#)
 - [Create new record programmatically](#)
 - [Intercept create request](#)
- [Update existing record](#)
 - [Update existing record programmatically](#)
 - [Intercept update request](#)
- [Delete record](#)
- [Transaction](#)
- [Programmatically expanding relations](#)
- [Check if record can be accessed](#)
- [Generating and validating tokens](#)

Set field value

```
// sets the value of a single record field
// (field type specific modifiers are also
```

```
supported)
record.set("title", "example")
record.set("users+", "6jyr1y02438et52") // append
to existing value

// populates a record from a data map
// (calls set() for each entry of the map)
record.load(data)
```

Get field value

```
// retrieve a single record field value
// (field specific modifiers are also supported)
record.get("someField") // -> any
(without cast)
record.getBool("someField") // -> cast to
bool
record.getString("someField") // -> cast to
string
record.getInt("someField") // -> cast to
int
record.getFloat("someField") // -> cast to
float64
record.getDateTime("someField") // -> cast to
types.DateTime
record.getStringSlice("someField") // -> cast to
[]string

// retrieve the new uploaded files
// (e.g. for inspecting and modifying the file(s)
before save)
record.getUnsavedFiles("someFileField")

// unmarshal a single json field value into the
provided result
let result = new DynamicModel({ ... })
record.unmarshalJSONField("someJsonField", result)

// retrieve a single or multiple expanded data
record.expandedOne("author") // -> as
null|Record
record.expandedAll("categories") // -> as []Record

// export all the public safe record fields in a
plain object
// (note: "json" type field values are exported as
raw bytes array)
record.publicExport()
```

Auth accessors

```
record.isSuperuser() // alias for
record.collection().name == "_superusers"

record.email() // alias for
record.get("email")
record.setEmail(email) // alias for
record.set("email", email)

record.verified() // alias for
record.get("verified")
record.setVerified(false) // alias for
record.set("verified", false)

record.tokenKey() // alias for
record.get("tokenKey")
record.setTokenKey(key) // alias for
record.set("tokenKey", key)
record.refreshTokenKey() // alias for
record.set("tokenKey:autogenerate", "")

record.validatePassword(pass)
record.setPassword(pass) // alias for
record.set("password", pass)
record.setRandomPassword() // sets
cryptographically random 30 characters string as
password
```

Copies

```
// returns a shallow copy of the current record
model populated
// with its ORIGINAL db data state and everything
else reset to the defaults
// (usually used for comparing old and new field
values)
record.original()

// returns a shallow copy of the current record
model populated
// with its LATEST data state and everything else
reset to the defaults
// (aka. no expand, no custom fields and with
default visibility flags)
record.fresh()
```

```
// returns a shallow copy of the current record
model populated
// with its ALL collection and custom fields data,
expand and visibility flags
record.clone()
```

Hide/Unhide fields

Collection fields can be marked as "Hidden" from the Dashboard to prevent regular user access to the field values.

Record models provide an option to further control the fields serialization visibility in addition to the "Hidden" fields option using the `record.hide(fieldNames...)` and `record.unhide(fieldNames...)` methods.

Often the `hide/unhide` methods are used in combination with the `onRecordEnrich` hook invoked on every record enriching (list, view, create, update, realtime change, etc.). For example:

```
onRecordEnrich((e) => {
  // dynamically show/hide a record field
  depending on whether the current
  // authenticated user has a certain "role" (or
  any other field constraint)
  if (
    !e.requestInfo.auth ||
    (!e.requestInfo.auth.isSuperuser() &&
    e.requestInfo.auth.get("role") != "staff")
  ) {
    e.record.hide("someStaffOnlyField")
  }

  e.next()
}, "articles")
```



For custom fields, not part of the record collection schema, it is required to call explicitly `record.withCustomData(true)` to allow them in the public serialization.

Fetch records

Fetch single record

All single record retrieval methods throw an error if no record is found.

```
// retrieve a single "articles" record by its id
let record = $app.findRecordById("articles",
  "RECORD_ID")

// retrieve a single "articles" record by a single
key-value pair
let record = $app.findFirstRecordByData("articles",
  "slug", "test")

// retrieve a single "articles" record by a string
filter expression
// (NB! use "{:placeholder}" to safely bind
untrusted user input parameters)
let record = $app.findFirstRecordByFilter(
  "articles",
  "status = 'public' && category = {:category}",
  { "category": "news" },
)
```

Fetch multiple records

All multiple records retrieval methods return an empty array if no records are found.

```
// retrieve multiple "articles" records by their
ids
let records = $app.findRecordsByIds("articles",
  ["RECORD_ID1", "RECORD_ID2"])

// retrieve the total number of "articles" records
in a collection with optional dbx expressions
let totalPending = $app.countRecords("articles",
  $dbx.hashExp({"status": "pending"}))

// retrieve multiple "articles" records with
optional dbx expressions
let records = $app.findAllRecords("articles",
  $dbx.exp("LOWER(username) = {:username}",
  {"username": "John.Doe"}),
  $dbx.hashExp({"status": "pending"}),
)

// retrieve multiple paginated "articles" records
by a string filter expression
// (NB! use "{:placeholder}" to safely bind
untrusted user input parameters)
let records = $app.findRecordsByFilter(
```

```

    "articles",
    // collection
    "status = 'public' && category = {:category}",
    // filter
    "-published",
    // sort
    10,
    // limit
    0,
    // offset
    { "category": "news" },
    // optional filter params
  )

```

Fetch auth records

```

// retrieve a single auth record by its email
let user = $app.findAuthRecordByEmail("users",
  "test@example.com")

// retrieve a single auth record by JWT
// (you could also specify an optional list of
// accepted token types)
let user = $app.findAuthRecordByToken("YOUR_TOKEN",
  "auth")

```

Custom record query

In addition to the above query helpers, you can also create custom Record queries using

`$app.recordQuery(collection)` [↗] method. It returns a SELECT DB builder that can be used with the same methods described in the [Database guide](#).

```

function findTopArticle() {
  let record = new Record();

  $app.recordQuery("articles")
    .andWhere($dbx.hashExp({ "status": "active"
  }))
    .orderBy("rank ASC")
    .limit(1)
    .one(record)

  return record
}

let article = findTopArticle()

```


For retrieving **multiple** Record models with the `all()` executor, you can use `arrayOf(new Record)` to create an array placeholder in which to populate the resolved DB result.

```
// the below is identical to
// $app.findRecordsByFilter("articles", "status =
// 'active'", '-published', 10)
// but allows more advanced use cases and filtering
// (aggregations, subqueries, etc.)
function findLatestArticles() {
    let records = arrayOf(new Record);

    $app.recordQuery("articles")
        .andWhere($dbx.hashExp({ "status": "active"
    }))
        .orderBy("published DESC")
        .limit(10)
        .all(records)

    return records
}

let articles = findLatestArticles()
```

Create new record

Create new record programmatically

```
let collection =
$app.findCollectionByNameOrId("articles")

let record = new Record(collection)

record.set("title", "Lorem ipsum")
record.set("active", true)

// field type specific modifiers can also be used
record.set("slug:autogenerate", "post-")

// new files must be one or a slice of
// filesystem.File values
//
// note1: see all factories in
// /jsvm/modules/_filesystem.html
// note2: for reading files from a request event
// you can also use e.findUploadedFiles("fileKey")
let f1 =
$filesystem.fileFromPath("/local/path/to/file1.txt")
```

```

)
let f2 = $filesystem.fileFromBytes("test content",
"file2.txt")
let f3 =
$filesystem.fileFromURL("https://example.com/file3.
pdf")
record.set("documents", [f1, f2, f3])

// validate and persist
// (use saveNoValidate to skip fields validation)
$app.save(record);

```

Intercept create request

```

onRecordCreateRequest((e) => {
  // ignore for superusers
  if (e.hasSuperuserAuth()) {
    return e.next()
  }

  // overwrite the submitted "status" field value
  e.record.set("status", "pending")

  // or you can also prevent the create event by
  returning an error
  let status = e.record.get("status")
  if (
    status !== "pending" &&
    // guest or not an editor
    (!e.auth || e.auth.get("role") !== "editor")
  ) {
    throw new BadRequestError("Only editors can
set a status different from pending")
  }

  e.next()
}, "articles")

```

Update existing record

Update existing record programmatically

```

let record = $app.findRecordById("articles",
"RECORD_ID")

record.set("title", "Lorem ipsum")

// delete existing record files by specifying their

```

```

file names
record.set("documents-", ["file1_abc123.txt",
"file3_abc123.txt"])

// append one or more new files to the already
uploaded list
//
// note1: see all factories in
/jsvm/modules/_filesystem.html
// note2: for reading files from a request event
you can also use e.findUploadedFiles("fileKey")
let f1 =
$filesystem.fileFromPath("/local/path/to/file1.txt"
)
let f2 = $filesystem.fileFromBytes("test content",
"file2.txt")
let f3 =
$filesystem.fileFromURL("https://example.com/file3.
pdf")
record.set("documents+", [f1, f2, f3])

// validate and persist
// (use saveNoValidate to skip fields validation)
$app.save(record);

```

Intercept update request

```

onRecordUpdateRequest((e) => {
  // ignore for superusers
  if (e.hasSuperuserAuth()) {
    return e.next()
  }

  // overwrite the submitted "status" field value
  e.record.set("status", "pending")

  // or you can also prevent the update event by
  returning an error
  let status = e.record.get("status")
  if (
    status !== "pending" &&
    // guest or not an editor
    (!e.auth || e.auth.get("role") !== "editor")
  ) {
    throw new BadRequestError("Only editors can
set a status different from pending")
  }

  e.next()
}, "articles")

```

Delete record

```
let record = $app.findRecordById("articles",
"RECORD_ID")

$app.delete(record)
```

Transaction

To execute multiple queries in a transaction you can use

```
$app.runInTransaction(fn) ↗
```

The DB operations are persisted only if the transaction completes without throwing an error.

It is safe to nest `runInTransaction` calls as long as you use the callback's `txApp` argument.



Inside the transaction function always use its `txApp` argument and not the original `$app` instance because we allow only a single writer/transaction at a time and it could result in a deadlock.

To avoid performance issues, try to minimize slow/long running tasks such as sending emails, connecting to external services, etc. as part of the transaction.

```
let titles = ["title1", "title2", "title3"]

let collection =
$app.findCollectionByNameOrId("articles")

$app.runInTransaction((txApp) => {
  // create new record for each title
  for (let title of titles) {
    let record = new Record(collection)

    record.set("title", title)

    txApp.save(record)
  }
})
```

Programmatically expanding relations

To expand record relations programmatically you can use

```
$app.expandRecord(record, expands, customFetchFunc) ↗
```

for single or

```
$app.expandRecords(records, expands, customFetchFunc) ↗
```

for multiple records.

Once loaded, you can access the expanded relations via

```
record.expandedOne(relName) ↗ or
```

```
record.expandedAll(relName) ↗ methods.
```

For example:

```
let record = $app.findFirstRecordByData("articles",
  "slug", "lorem-ipsum")

// expand the "author" and "categories" relations
$app.expandRecord(record, ["author", "categories"],
  null)

// print the expanded records
console.log(record.expandedOne("author"))
console.log(record.expandedAll("categories"))
```

Check if record can be accessed

To check whether a custom client request or user can access a single record, you can use the

```
$app.canAccessRecord(record, requestInfo, rule) ↗
```

method.

Below is an example of creating a custom route to retrieve a single article and checking if the request satisfy the View API rule of the record collection:

```
routerAdd("GET", "/articles/{slug}", (e) => {
  let slug = e.request.pathValue("slug")

  let record =
    e.app.findFirstRecordByData("articles", "slug",
    slug)

  let canAccess = e.app.canAccessRecord(record,
    e.requestInfo(), record.collection().viewRule)
  if (!canAccess) {
    throw new ForbiddenError()
```

```

    }

    return e.json(200, record)
  })

```

Generating and validating tokens

PocketBase Web APIs are fully stateless (aka. there are no sessions in the traditional sense) and an auth record is considered authenticated if the submitted request contains a valid `Authorization: TOKEN` header (see also [Builtin auth middlewares](#) and [Retrieving the current auth state from a route](#)).

If you want to issue and verify manually a record JWT (auth, verification, password reset, etc.), you could do that using the record token type specific methods:

```

let token = record.newAuthToken()

let token = record.newVerificationToken()

let token = record.newPasswordResetToken()

let token = record.newEmailChangeToken(newEmail)

let token = record.newFileToken() // for protected files

let token =
  record.newStaticAuthToken(optCustomDuration) //
  nonrenewable auth token

```

Each token type has its own secret and the token duration is managed via its type related collection auth option (*the only exception is* `newStaticAuthToken`).

To validate a record token you can use the `$app.findAuthRecordByToken` method. The token related auth record is returned only if the token is not expired and its signature is valid.

Here is an example how to validate an auth token:

```

let record =
  $app.findAuthRecordByToken("YOUR_TOKEN", "auth")

```

← Prev: Database

Next: Collection operations →

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



[Introduction](#)[Going to production](#)[Web APIs reference](#)[Extend with Go](#)[Extend with JavaScript](#)[JS Overview](#)[JS Event hooks](#)[JS Routing](#)[JS Database](#)[JS Record operations](#)[JS Collection operations](#)[JS Migrations](#)[JS Jobs scheduling](#)[JS Sending emails](#)[JS Rendering templates](#)[JS Console commands](#)[JS Sending HTTP requests](#)[JS Realtime messaging](#)[JS Filesystem](#)[JS Logging](#)

Extend with JavaScript - Collection operations

Collections are usually managed via the Dashboard interface, but there are some situations where you may want to create or edit a collection programmatically (usually as part of a [DB migration](#)). You can find all available Collection related operations and methods in `$app` and `Collection`, but below are listed some of the most common ones:

- [Fetch collections](#)
 - [Fetch single collection](#)
 - [Fetch multiple collections](#)
 - [Custom collection query](#)
- [Field definitions](#)
- [Create new collection](#)
- [Update existing collection](#)
- [Delete collection](#)

Fetch collections

Fetch single collection

All single collection retrieval methods throw an error if no collection is found.

```
let collection =  
$app.findCollectionByNameOrId("example")
```

Fetch multiple collections

All multiple collections retrieval methods return an empty array if no collections are found.

```
let allCollections = $app.findAllCollections(/*  
optional types */)

// only specific types
```



```
let authAndViewCollections =
  $app.findAllCollections("auth", "view")
```

Custom collection query

In addition to the above query helpers, you can also create custom Collection queries using `$app.collectionQuery()` method. It returns a SELECT DB builder that can be used with the same methods described in the [Database guide](#).

```
let collections = arrayOf(new Collection)

$app.collectionQuery().
  andWhere($dbx.hashExp({"viewRule": null})).
  orderBy("created DESC").
  all(collections)
```

Field definitions



All collection fields (*with exception of the `JSONField`*) are non-nullable and use a zero-default for their respective type as fallback value when missing.

- `new BoolField({ ... })`
- `new NumberField({ ... })`
- `new TextField({ ... })`
- `new EmailField({ ... })`
- `new URLField({ ... })`
- `new EditorField({ ... })`
- `new DateField({ ... })`
- `new AutodateField({ ... })`
- `new SelectField({ ... })`
- `new FileField({ ... })`
- `new RelationField({ ... })`
- `new JSONField({ ... })`
- `new GeoPointField({ ... })`

Create new collection

```
// missing default options, system fields like id,
// email, etc. are initialized automatically
// and will be merged with the provided
// configuration
let collection = new Collection({
  type:      "base", // base | auth | view
  name:      "example",
  listRule:  null,
  viewRule:  "@request.auth.id != ''",
  createRule: "",
  updateRule: "@request.auth.id != ''",
  deleteRule: null,
  fields: [
    {
      name:      "title",
      type:      "text",
      required:  true,
      max:      10,
    },
    {
      name:      "user",
      type:      "relation",
      required:  true,
      maxSelect: 1,
      collectionId: "ae40239d2bc4477",
      cascadeDelete: true,
    },
  ],
  indexes: [
    "CREATE UNIQUE INDEX idx_user ON example
    (user)"
  ],
})

// validate and persist
// (use saveNoValidate to skip fields validation)
$app.save(collection)
```

Update existing collection

```
let collection =
$app.findCollectionByNameOrId("example")

// change the collection name
collection.name = "example_update"
```

```
// add new editor field
collection.fields.add(new EditorField({
  name:      "description",
  required: true,
}))

// change existing field
// (returns a pointer and direct modifications are
// allowed without the need of reinsert)
let titleField =
collection.fields.getByName("title")
titleField.min = 10

// or: collection.indexes.push("CREATE INDEX
idx_example_title ON example (title)")
collection.addIndex("idx_example_title", false,
"title", "")

// validate and persist
// (use saveNoValidate to skip fields validation)
$app.save(collection)
```

Delete collection

```
let collection =
$app.findCollectionByNameOrId("example")

$app.delete(collection)
```

[← Prev: Record operations](#)

[Next: Migrations →](#)

[FAQ](#)

[JavaScript SDK](#)

PocketBase

[Discussions](#)

[Dart SDK](#)

[Documentation](#)



[Introduction](#)[Going to production](#)[Web APIs reference](#)[Extend with
Go](#)[Extend with
JavaScript](#)[JS Overview](#)[JS Event hooks](#)[JS Routing](#)[JS Database](#)[JS Record operations](#)[JS Collection operations](#)[JS Migrations](#)[JS Jobs scheduling](#)[JS Sending emails](#)[JS Rendering templates](#)[JS Console commands](#)[JS Sending HTTP requests](#)[JS Realtime messaging](#)[JS Filesystem](#)[JS Logging](#)

Extend with JavaScript - Migrations

PocketBase comes with a builtin DB and data migration utility, allowing you to version your DB structure, create collections programmatically, initialize default settings and/or run anything that needs to be executed only once.

The user defined migrations are located in `pb_migrations` directory (it can be changed using the `--migrationsDir` flag) and each unapplied migration inside it will be executed automatically in a transaction on `serve` (or on `migrate up`).

The generated migrations are safe to be committed to version control and can be shared with your other team members.

- [Automigrate](#)
- [Creating migrations](#)
 - [Migration file](#)
- [Collections snapshot](#)
- [Migrations history](#)
- [Examples](#)
 - [Executing raw SQL statements](#)
 - [Initialize default application settings](#)
 - [Creating initial superuser](#)
 - [Creating collection programmatically](#)

Automigrate

The prebuilt executable has the `--automigrate` flag enabled by default, meaning that every collection configuration change from the Dashboard (or Web API) will generate the related migration file automatically for you.

Creating migrations

To create a new blank migration you can run `migrate create`.

```
[root@dev app]$ ./pocketbase migrate create
"your_new_migration"
```

```
// pb_migrations/1687801097_your_new_migration.js
migrate((app) => {
  // add up queries...
}, (app) => {
  // add down queries...
})
```

New migrations are applied automatically on `serve`.

Optionally, you could apply new migrations manually by running `migrate up`.

To revert the last applied migration(s), you could run

```
migrate down [number].
```

When manually applying or reverting migrations, the `serve` process needs to be restarted so that it can refresh its cached collections state.

Migration file

Each migration file should have a single

```
migrate(upFunc, downFunc) call.
```

In the migration file, you are expected to write your "upgrade" code in the `upFunc` callback.

The `downFunc` is optional and it should contain the "downgrade" operations to revert the changes made by the `upFunc`.

Both callbacks accept a transactional `app` instance.

Collections snapshot

The `migrate collections` command generates a full snapshot of your current collections configuration without having to type it manually. Similar to the `migrate create` command, this will generate a new migration file in the `pb_migrations` directory.

```
[root@dev app]$ ./pocketbase migrate collections
```

By default the collections snapshot is imported in *extend* mode, meaning that collections and fields that don't exist in the

snapshot are preserved. If you want the snapshot to *delete* missing collections and fields, you can edit the generated file and change the last argument of `importCollections` to `true`.

Migrations history

All applied migration filenames are stored in the internal `_migrations` table.

During local development often you might end up making various collection changes to test different approaches.

When `--automigrate` is enabled (*which is the default*) this could lead in a migration history with unnecessary intermediate steps that may not be wanted in the final migration history.

To avoid the clutter and to prevent applying the intermediate steps in production, you can remove (or squash) the unnecessary migration files manually and then update the local migrations history by running:

```
[root@dev app]$ ./pocketbase migrate history-sync
```

The above command will remove any entry from the `_migrations` table that doesn't have a related migration file associated with it.

Examples

Executing raw SQL statements

```
// pb_migrations/1687801090_set_pending_status.js

migrate((app) => {
  app.db().newQuery("UPDATE articles SET status
= 'pending' WHERE status = ''").execute()
})
```

Initialize default application settings

```
// pb_migrations/1687801090_initial_settings.js

migrate((app) => {
  let settings = app.settings()
```

```

    // for all available settings fields you could
    check
    // /jsvm/interfaces/core.Settings.html
    settings.meta.appName = "test"
    settings.meta.appURL = "https://example.com"
    settings.logs.maxDays = 2
    settings.logs.logAuthId = true
    settings.logs.logIP = false

    app.save(settings)
  })

```

Creating initial superuser

For all supported record methods, you can refer to [Record operations](#).

```

// pb_migrations/1687801090_initial_superuser.js

migrate((app) => {
  let superusers =
  app.findCollectionByNameOrId("_superusers")

  let record = new Record(superusers)

  // note: the values can be eventually loaded
  via $os.getenv(key)
  // or from a special local config file
  record.set("email", "test@example.com")
  record.set("password", "1234567890")

  app.save(record)
}, (app) => { // optional revert operation
  try {
    let record =
    app.findAuthRecordByEmail("_superusers",
    "test@example.com")
    app.delete(record)
  } catch {
    // silent errors (probably already
    deleted)
  }
})

```

Creating collection programmatically

For all supported collection methods, you can refer to [Collection operations](#).


```
//
migrations/1687801090_create_clients_collection.js

migrate((app) => {
  // missing default options, system fields like
  id, email, etc. are initialized automatically
  // and will be merged with the provided
  configuration
  let collection = new Collection({
    type:      "auth",
    name:      "clients",
    listRule:  "id = @request.auth.id",
    viewRule:  "id = @request.auth.id",
    fields: [
      {
        type:      "text",
        name:      "company",
        required:  true,
        max:       100,
      },
      {
        name:      "url",
        type:      "url",
        presentable: true,
      },
    ],
    passwordAuth: {
      enabled: false,
    },
    otp: {
      enabled: true,
    },
    indexes: [
      "CREATE INDEX idx_clients_company ON
clients (company)"
    ],
  })

  app.save(collection)
}, (app) => {
  let collection =
app.findCollectionByNameOrId("clients")
  app.delete(collection)
})
```

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



[Introduction](#)[Going to production](#)[Web APIs reference](#)Extend with
GoExtend with
JavaScript[JS Overview](#)[JS Event hooks](#)[JS Routing](#)[JS Database](#)[JS Record operations](#)[JS Collection operations](#)[JS Migrations](#)[JS Jobs scheduling](#)[JS Sending emails](#)[JS Rendering templates](#)[JS Console commands](#)[JS Sending HTTP requests](#)[JS Realtime messaging](#)[JS Filesystem](#)[JS Logging](#)

Extend with JavaScript - Jobs scheduling

If you have tasks that need to be performed periodically, you could set up crontab-like jobs with

```
cronAdd(id, expr, handler).
```

Each scheduled job runs in its own goroutine as part of the `serve` command process and must have:

- **id** - identifier for the scheduled job; could be used to replace or remove an existing job
- **cron expression** - e.g. `0 0 * * *` (supports numeric list, steps, ranges or macros)
- **handler** - the function that will be executed every time when the job runs

Here is an example:

```
// prints "Hello!" every 2 minutes
cronAdd("hello", "*/2 * * * *", () => {
  console.log("Hello!")
})
```

To remove a single registered cron job you can call

```
cronRemove(id).
```

All registered app level cron jobs can be also previewed and triggered from the *Dashboard > Settings > Crons* section.

[← Prev: Migrations](#)[Next: Sending emails →](#)

FAQ

JavaScript SDK

PocketBase

Discussions

Dart SDK

Documentation



[Introduction](#)[Going to production](#)[Web APIs reference](#)Extend with
GoExtend with
JavaScript[JS Overview](#)[JS Event hooks](#)[JS Routing](#)[JS Database](#)[JS Record operations](#)[JS Collection operations](#)[JS Migrations](#)[JS Jobs scheduling](#)[JS Sending emails](#)[JS Rendering templates](#)[JS Console commands](#)[JS Sending HTTP requests](#)[JS Realtime messaging](#)[JS Filesystem](#)[JS Logging](#)

Extend with JavaScript - Sending emails

PocketBase provides a simple abstraction for sending emails via the `$app.newMailClient()` helper.

Depending on your configured mail settings (*Dashboard > Settings > Mail settings*) it will use the `sendmail` command or a SMTP client.

- [Send custom email](#)
- [Overwrite system emails](#)

Send custom email

You can send your own custom emails from everywhere within the app (hooks, middlewares, routes, etc.) by using `$app.newMailClient().send(message)`. Here is an example of sending a custom email after user registration:

```
onRecordCreateRequest((e) => {
  e.next()

  const message = new MailerMessage({
    from: {
      address:
e.app.settings().meta.senderAddress,
      name:
e.app.settings().meta.senderName,
    },
    to:      [{address: e.record.email()}],
    subject: "YOUR_SUBJECT...",
    html:    "YOUR_HTML_BODY...",
    // bcc, cc and custom headers are also
supported...
  })

  e.app.newMailClient().send(message)
}, "users")
```

Overwrite system emails

If you want to overwrite the default system emails for forgotten password, verification, etc., you can adjust the default templates available from the *Dashboard > Collections > Edit collection > Options*.

Alternatively, you can also apply individual changes by binding to one of the mailer hooks. Here is an example of appending a Record field value to the subject using the

`onMailerRecordPasswordResetSend` hook:

```
onMailerRecordPasswordResetSend((e) => {  
  // modify the subject  
  e.message.subject += (" " +  
    e.record.get("name"))  
  
  e.next()  
})
```

[← Prev: Jobs scheduling](#)

[Next: Rendering templates →](#)

[FAQ](#)

[JavaScript SDK](#)

PocketBase

[Discussions](#)

[Dart SDK](#)

[Documentation](#)



[Introduction](#)[Going to production](#)[Web APIs reference](#)Extend with
GoExtend with
JavaScript[JS Overview](#)[JS Event hooks](#)[JS Routing](#)[JS Database](#)[JS Record operations](#)[JS Collection operations](#)[JS Migrations](#)[JS Jobs scheduling](#)[JS Sending emails](#)[JS Rendering templates](#)[JS Console commands](#)[JS Sending HTTP requests](#)[JS Realtime messaging](#)[JS Filesystem](#)[JS Logging](#)

Extend with JavaScript - Rendering templates

- [Overview](#)
- [Example HTML page with layout](#)

Overview

A common task when creating custom routes or emails is the need of generating HTML output. To assist with this, PocketBase provides the global `$template` helper for parsing and rendering HTML templates.

```
const html = $template.loadFiles(  
  `${__hooks}/views/base.html`,  
  `${__hooks}/views/partial1.html`,  
  `${__hooks}/views/partial2.html`,  
).render(data)
```

The general flow when working with composed and nested templates is that you create "base" template(s) that defines various placeholders using the `{{template "placeholderName" .}}` or `{{block "placeholderName" .}}default...{{end}}` actions.

Then in the partials, you define the content for those placeholders using the

`{{define "placeholderName"}}custom...{{end}}` action.

The dot object (`.`) in the above represents the data passed to the templates via the `render(data)` method.

By default the templates apply contextual (HTML, JS, CSS, URI) auto escaping so the generated template content should be injection-safe. To render raw/verbatim trusted content in the templates you can use the builtin `raw` function (e.g. `{{.content|raw}}`).

For more information about the template syntax please refer to the [html/template](#) and [text/template](#) package godocs. **Another great**



resource is also the Hashicorp's [Learn Go Template Syntax](#) tutorial.

Example HTML page with layout

Consider the following app directory structure:

```
myapp/  
  pb_hooks/  
    views/  
      layout.html  
      hello.html  
    main.pb.js  
  pocketbase
```

We define the content for `layout.html` as:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <title>{{block "title" .}}Default app  
title{{end}}</title>  
</head>  
<body>  
  Header...  
  
  {{block "body" .}}  
    Default app body...  
  {{end}}  
  
  Footer...  
</body>  
</html>
```

We define the content for `hello.html` as:

```
{{define "title"}}  
  Page 1  
{{end}}  
  
{{define "body"}}  
  <p>Hello from {{.name}}</p>  
{{end}}
```

Then to output the final page, we'll register a custom

`/hello/:name` route:


```
routerAdd("get", "/hello/{name}", (e) => {
  const name = e.request.pathValue("name")

  const html = $template.loadFiles(
    `${__hooks}/views/layout.html`,
    `${__hooks}/views/hello.html`,
  ).render({
    "name": name,
  })

  return e.html(200, html)
})
```

← [Prev: Sending emails](#)

[Next: Console commands](#) →

[FAQ](#)

[JavaScript SDK](#)

PocketBase

[Discussions](#)

[Dart SDK](#)

[Documentation](#)

